Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Introduction to MongoDB (Architecture, Features, and Core Concepts)

🎯 Objective

To understand what MongoDB is, its architecture, key design principles, and how it differs from traditional relational databases like SQL Server or Oracle. This module builds the foundational knowledge needed for all MongoDB administration topics.

◆ 1. What is MongoDB?

MongoDB is a **NoSQL**, **document-oriented** database designed for:

- **High performance**
- **High availability**
- **Horizontal scalability**

Instead of storing data in rows and columns (like RDBMS), MongoDB stores it in **JSON-like documents** (BSON format).

◆ 2. MongoDB Key Characteristics

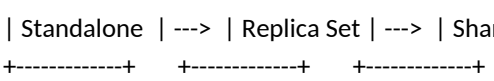
Feature	Description
Document Model	Data is stored as key–value pairs in flexible JSON-like documents.
Schema-less	Each document can have a different structure — no rigid schema.
Horizontal Scalability	Supports **sharding** for distributed scaling across servers.
High Availability	Uses **replica sets** for automatic failover and redundancy.
Powerful Query Language	Rich query capabilities with aggregation pipelines.
Cross-platform	Runs on Linux, Windows, macOS, and in containers.

◆ 3. MongoDB Architecture Overview

A MongoDB deployment consists of the following components:

Component	Description
mongod	Primary database server process handling data requests and replication.
mongos	Query router used in sharded clusters to distribute operations.
Config Servers	Store metadata for sharded clusters.
Replica Sets	Groups of mongod instances maintaining the same data set for HA.
Clients / Drivers	Applications connect to MongoDB through supported drivers (Python, Java, .NET, etc.).

Visual Architecture (Standalone → Replica → Sharded):



◆ 4. MongoDB vs Relational Databases

Concept	MongoDB	RDBMS (SQL Server / Oracle)
Data Storage	Collections & Documents	Tables & Rows
Schema	Dynamic / Flexible	Fixed Schema
Query Language	MQL (Mongo Query Language)	SQL
Joins	Embedded Documents / $lookup	JOIN
Scaling	Horizontal (Sharding)	Vertical (Add resources)
Transactions	Supported (ACID since v4.0)	Supported
Backup Tool	mongodump, mongorestore	BACKUP DATABASE
Replication	Replica Sets	Log Shipping / Always On
Failover	Automatic	Manual or semi-automatic

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

◆ 5. MongoDB Ecosystem Components

Tool / Service	Purpose
MongoDB Compass	GUI for visualization, queries, and schema exploration
MongoDB Atlas	Fully managed cloud-based MongoDB service
mongosh	Modern Mongo shell for administration
Ops Manager	On-premises management and monitoring platform
Atlas CLI	Command-line tool to manage cloud resources

◆ 6. MongoDB Editions

Edition	Description
Community Edition	Free, open-source version with full features
Enterprise Edition	Commercial version with security, monitoring, and support
Atlas Cloud	Fully managed MongoDB-as-a-Service by MongoDB Inc.

◆ 7. BSON (Binary JSON)

- MongoDB stores documents in **BSON** — a binary form of JSON that adds:
  - **Type support** (e.g., dates, binary, decimals)
  - **Efficient encoding/decoding**
  - **Indexing optimization**

Example:

```
{
  "_id": ObjectId("671859bc9d3f2a4f5f0f1c20"),
  "name": "John Doe",
  "age": 32,
  "skills": ["MongoDB", "Node.js", "DevOps"]
}
```

◆ 8. MongoDB Core Components Summary

Component	Description
Database	Container for collections
Collection	Group of related documents
Document	JSON-like object (similar to a row in SQL)
Field	Key-value pair (similar to a column)
Index	Improves query performance
Replica Set	Ensures redundancy and HA
Shard	Enables horizontal scaling

◆ 9. MongoDB Advantages

✔ Flexible schema design
✔ Built-in replication and sharding
✔ Powerful aggregation framework
✔ Cross-platform and cloud-ready
✔ Easy integration with modern apps and APIs

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

◆ 10. MongoDB Limitations

⚠ Complex joins (compared to SQL)
⚠ Higher memory usage for large workloads
⚠ Requires careful shard key selection
⚠ Limited multi-document ACID before v4.0

◆ 11. Hands-On Lab Checklist

Exercise	Task	Validation Command / Step
Lab 1	Install MongoDB on Linux or Windows	mongod --version
Lab 2	Connect using Mongo Shell	mongosh
Lab 3	Create a database and collection	use testDB; db.createCollection("students")
Lab 4	Insert sample data	db.students.insertOne({name: "Alice", age: 25})
Lab 5	Retrieve data	db.students.find().pretty()
Lab 6	Explore MongoDB Compass	View collections and documents
Lab 7	Verify installation directories	/var/lib/mongo, /etc/mongod.conf

□ Summary

By completing this module, you now understand:

- What MongoDB is and how it differs from RDBMS
- MongoDB's architecture and ecosystem components
- Basic MongoDB commands and structure
- The core foundation for replication, sharding, and performance tuning in later modules

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Replication — Setup, Failover, and Administration

Replication is the foundation of **high availability (HA)** in MongoDB. It ensures your data remains accessible and consistent, even if one server goes down.

1 What Is Replication in MongoDB?

Replication means maintaining **identical copies of data** across multiple MongoDB servers (called *replica set members*).

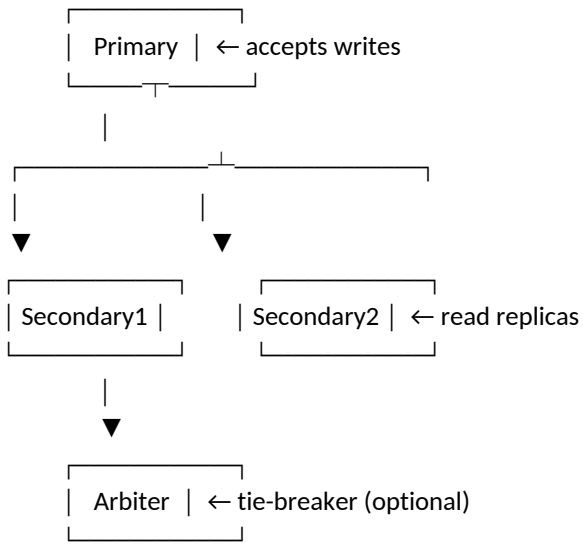
Key Benefits:

- ✔ High Availability (automatic failover)
- ✔ Data Redundancy
- ✔ Disaster Recovery
- ✔ Read scalability (secondary reads)
- ✔ Backup from secondary nodes (no primary load)

2 Replica Set Components

Role	Description
Primary	Receives all write operations and replicates data to secondaries.
Secondary	Copies data from the primary's oplog and stays synchronized.
Arbiter (optional)	Participates in elections but doesn't store data. Helps maintain an odd number of voting members.

Example Architecture



3 Steps to Set Up a Replica Set (Hands-On)

Step 1 — Prepare Servers

You need **3 or more MongoDB instances** (can be on the same host for lab setup):

Node	Hostname	Port
Node1	mongo1	27017
Node2	mongo2	27018
Node3	mongo3	27019

Each instance should have a unique **data path** and **port**.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Step 2 — Create Config Files

/etc/mongod1.conf

```
storage:
  dbPath: /data/mongo1
net:
  bindIp: 0.0.0.0
  port: 27017
replication:
  replSetName: rs0
```

/etc/mongod2.conf

```
storage:
  dbPath: /data/mongo2
net:
  bindIp: 0.0.0.0
  port: 27018
replication:
  replSetName: rs0
```

/etc/mongod3.conf

```
storage:
  dbPath: /data/mongo3
net:
  bindIp: 0.0.0.0
  port: 27019
replication:
  replSetName: rs0
```

Step 3 — Start Each Instance

```
sudo systemctl start mongod1
sudo systemctl start mongod2
sudo systemctl start mongod3
```

Or directly:

```
mongod --config /etc/mongod1.conf &
mongod --config /etc/mongod2.conf &
mongod --config /etc/mongod3.conf &
```

Step 4 — Initialize Replica Set

Connect to **one node**:

```
mongosh --port 27017
```

Then initialize:

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "mongo1:27017" },
    { _id: 1, host: "mongo2:27018" },
    { _id: 2, host: "mongo3:27019" }
  ]
})
```

Check status:

```
rs.status()
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

✔ One node becomes **PRIMARY**, others become **SECONDARY**.

▯? Step 5 — Verify Replication

Insert sample data:

```
use testdb
db.users.insertOne({ name: "Alice", role: "DBA" })
```

Then connect to a secondary (read-only by default):

```
mongosh --port 27018
```

Enable secondary reads:

```
rs.slaveOk()
db.users.find()
```

You'll see the same document — replication is working 🎉

💣 4 Automatic Failover Demo

1. Stop the primary:
   ```
   sudo systemctl stop mongod1
   ```
2. Wait ~10 seconds — one of the secondaries becomes **new PRIMARY**.
3. Run:
   ```
   rs.status()
   ```
4. Restart the original primary; it rejoins as a **secondary**.

5 Replica Set Administration Commands

Task	Command
Check status	rs.status()
Add member	rs.add("mongo4:27020")
Remove member	rs.remove("mongo3:27019")
Reconfigure set	rs.reconfig(config)
Step down primary	rs.stepDown()
Sync from specific node	rs.syncFrom("mongo2:27018")

6 Maintenance Tips

Task	Best Practice
Backups	Run mongodump from a secondary
Monitoring	Use db.printReplicationInfo() and rs.printSlaveReplicationInfo()
Elections	Keep odd number of voting members
Disk layout	Use SSDs for oplog-heavy workloads
Oplog size	Increase oplog for write-heavy workloads (--oplogSize)
Security	Use keyFile authentication between nodes

7 Hands-On Validation Checklist

Validation Step	Command	Expected Output
Check primary	rs.status()	One node shows "stateStr" : "PRIMARY"
Data replicated	db.collection.find() on secondary	Data visible
Failover tested	Stop primary	New node elected
Sync delay	rs.printSlaveReplicationInfo()	Seconds behind master shown
Backup from secondary	mongodump --readPreference secondary	Successful dump

8 Replica Set Architecture Summary

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Environment	Members	Use Case
1 Primary + 2 Secondary	Basic HA	Standard 3-node cluster
2 Secondary + 1 Arbiter	Space-saving	Lightweight setup
5 Members	Enterprise-grade	Large-scale systems
Hidden Member	Backup-only node	Offload reporting/backup

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Sharding — Configuration, Scaling, and Administration

Sharding allows MongoDB to **scale horizontally** by distributing data across multiple servers.

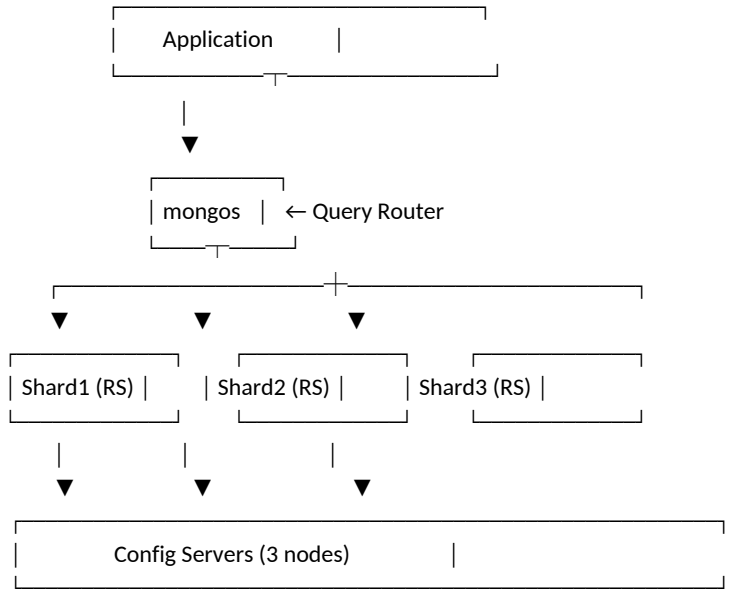
1 What Is Sharding?

Sharding = Partitioning data across multiple servers (shards) so that large datasets and high-throughput workloads can be handled efficiently. It's MongoDB's way of scaling **out** instead of **up**.

2 Sharded Cluster Components

Component	Description
Shard	A data-bearing node or replica set that stores part of the collection data.
Config Server (configsvr)	Stores cluster metadata, sharding configuration, and chunk distribution.
Query Router (mongos)	Routes client requests to the appropriate shard(s). Acts as a load balancer.

Example Architecture



3 Sharded Cluster Setup (Hands-On)

We'll build:

- **3 Config Servers**
- **3 Shards (replica sets)**
- **1 mongos router**

Step 1 — Start Config Servers

Create folders:

```
mkdir -p /data/config1 /data/config2 /data/config3
```

Start instances:

```
mongod --configsvr --replSet configReplSet --dbpath /data/config1 --port 27019 --bind_ip localhost &
mongod --configsvr --replSet configReplSet --dbpath /data/config2 --port 27020 --bind_ip localhost &
mongod --configsvr --replSet configReplSet --dbpath /data/config3 --port 27021 --bind_ip localhost &
```

Initiate replica set:

```
mongosh --port 27019
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
rs.initiate({
  _id: "configReplSet",
  configsvr: true,
  members: [
    { _id: 0, host: "localhost:27019" },
    { _id: 1, host: "localhost:27020" },
    { _id: 2, host: "localhost:27021" }
  ]
})
```

Step 2 — Start Shard Replica Sets

Each shard will be a 3-node replica set:

Shard1

```
mongod --shardsvr --replSet shard1 --dbpath /data/shard1a --port 27030 --bind_ip localhost &
mongod --shardsvr --replSet shard1 --dbpath /data/shard1b --port 27031 --bind_ip localhost &
mongod --shardsvr --replSet shard1 --dbpath /data/shard1c --port 27032 --bind_ip localhost &
```

Initialize shard1:

```
rs.initiate({
  _id: "shard1",
  members: [
    { _id: 0, host: "localhost:27030" },
    { _id: 1, host: "localhost:27031" },
    { _id: 2, host: "localhost:27032" }
  ]
})
```

Repeat for **shard2** and **shard3** (ports 27040–27042, 27050–27052).

Step 3 — Start mongos Router

```
mongos --configdb configReplSet/localhost:27019,localhost:27020,localhost:27021 --port 27017 --bind_ip localhost &
```

Step 4 — Connect to mongos

```
mongosh --port 27017
```

Add shards:

```
sh.addShard("shard1/localhost:27030,localhost:27031,localhost:27032")
sh.addShard("shard2/localhost:27040,localhost:27041,localhost:27042")
sh.addShard("shard3/localhost:27050,localhost:27051,localhost:27052")
```

Check:

```
sh.status()
```

4️⃣ Enable Sharding for a Database and Collection

Enable for DB:

```
sh.enableSharding("salesDB")
```

Choose a **shard key** (must be indexed):

```
db.sales.createIndex({ customer_id: 1 })
sh.shardCollection("salesDB.sales", { customer_id: 1 })
```

✔ Now MongoDB distributes data across shards automatically.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

5 Types of Sharding

Type	Description	Use Case
Ranged Sharding	Divides data into contiguous ranges based on the shard key.	Sequential data like time-series, IDs
Hashed Sharding	Hashes the shard key to randomize distribution.	Evenly spread writes, avoids hotspots
Zone Sharding	Assigns specific ranges to specific shards.	Geo-distributed clusters

6 Monitoring and Administration

Command	Purpose
sh.status()	Show cluster status
db.printShardingStatus()	Detailed distribution info
balancerStatus()	Check balancer activity
sh.stopBalancer() / sh.startBalancer()	Control balancer
db.collection.getShardDistribution()	See per-shard chunk sizes
sh.moveChunk()	Manually move chunks between shards

7 Maintenance Best Practices

Task	Best Practice
Choose Shard Key	Pick a field with high cardinality & good distribution
Backups	Use mongodump per shard, or cluster-wide backups
Indexing	Ensure shard key fields are indexed
Monitor Balancer	Prevent over-sharding and hotspots
Networking	Low latency between shards & config servers
Security	Enable keyFile authentication between cluster components

8 Hands-On Validation Checklist

Validation Step	Command	Expected Output
Shards added	sh.status()	Lists all shards
Database sharded	sh.enableSharding("salesDB")	Success message
Collection sharded	sh.shardCollection("salesDB.sales", {customer_id:1})	Success
Data distributed	db.sales.getShardDistribution()	Documents spread across shards
Balancer active	sh.isBalancerRunning()	true when active

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Security, Backup, and Monitoring Best Practices

This module focuses on how to **protect MongoDB environments**, **secure data in transit and at rest**, **perform consistent backups**, and **monitor clusters** for proactive maintenance.

1️⃣ MongoDB Security Overview

MongoDB's security is built on four pillars:

Pillar	Description
Authentication	Verifies who can connect
Authorization	Controls what actions are allowed
Encryption	Protects data in transit and at rest
Auditing	Tracks access and configuration changes

🔐 2️⃣ Authentication Methods

Method	Description	Common Use Case
SCRAM (Default)	Username/password stored in system.users	General deployments
x.509 Certificates	SSL-based mutual authentication	Enterprise/Cluster-to-Cluster
LDAP / Kerberos	Centralized auth integration	Enterprise SSO environments
KeyFile Authentication	Shared secret for replica sets / sharded clusters	Internal node-to-node authentication

⚙️ Enable Authentication (Standalone Example)

Step 1 — Start MongoDB without auth

```
mongod --port 27017 --dbpath /data/db --bind_ip 0.0.0.0
```

Step 2 — Create the first admin user

```
use admin
db.createUser({
  user: "rootAdmin",
  pwd: "StrongPassword123!",
  roles: [ { role: "root", db: "admin" } ]
})
```

Step 3 — Restart MongoDB with authentication

```
mongod --auth --port 27017 --dbpath /data/db --bind_ip 0.0.0.0
```

Step 4 — Connect as admin

```
mongosh -u rootAdmin -p StrongPassword123! --authenticationDatabase admin
```

✔ Authentication enabled.

3️⃣ Role-Based Access Control (RBAC)

Roles define what a user can do.

Common Built-in Roles

Role	Description
read	Read-only access to database
readWrite	Read and write access
dbAdmin	Database admin tasks (indexes, stats)
userAdmin	Manage users and roles
clusterAdmin	Manage cluster-wide operations
backup / restore	Backup and restore rights
root	Superuser (full control)

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Example: Create a read-only user

```
use salesDB
db.createUser({
  user: "readonlyUser",
  pwd: "Password123",
  roles: [{ role: "read", db: "salesDB" }]
})
```

🔐 4 Encryption in MongoDB

🔒 Data in Transit — TLS/SSL

1 Generate certificates:

```
openssl req -newkey rsa:4096 -new -x509 -days 365 -keyout mongodb.key -out mongodb.crt
cat mongodb.key mongodb.crt > mongodb.pem
```

2 Update config:

```
net:
  tls:
    mode: requireTLS
    certificateKeyFile: /etc/ssl/mongodb.pem
```

3 Restart mongod:

```
sudo systemctl restart mongod
```

🔒 Data at Rest — Encryption at Storage Level

MongoDB Enterprise supports **Encrypted Storage Engine**.

```
security:
  enableEncryption: true
  encryptionKeyFile: /etc/mongodb-keyfile
```

Generate key file:

```
openssl rand -base64 32 > /etc/mongodb-keyfile
chmod 600 /etc/mongodb-keyfile
```

5 Backup and Restore Strategies

MongoDB supports several backup methods:

Type	Tool / Method	Description
Logical Backup	mongodump / mongorestore	BSON dumps of collections and metadata
Physical Backup	File-system level snapshots	Faster, consistent with replica sets
Cloud Backup	MongoDB Atlas Backup	Managed incremental backups
Hot Backup	Backup from secondary	No impact on primary performance

? Example 1 — Full Logical Backup

```
mongodump --host localhost --port 27017 --out /backups/mongobackup_$(date +%F)
```

? Example 2 — Restore Backup

```
mongorestore --host localhost --port 27017 /backups/mongobackup_2025-10-14
```

? Example 3 — Backup Specific Database

```
mongodump --db salesDB --out /backups/salesDB
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

6 Replica Set Backup Best Practice

✔ Always take backups from **SECONDARY** to reduce load:

mongodump --readPreference secondary --oplog --out /backups/replica_set_backup

--oplog ensures a consistent point-in-time snapshot.

7 Monitoring MongoDB

MongoDB provides native and third-party monitoring tools.

Tool	Description
mongostat	Real-time DB metrics (ops/sec, latency)
mongotop	Time spent reading/writing per collection
serverStatus	Built-in health metrics
Atlas Monitoring	Graphical monitoring (cloud)
Prometheus + Grafana	Open-source monitoring stack
Cloud Manager / Ops Manager	Enterprise-grade management suite

 Example: Monitor Ops in Real Time

mongostat --host localhost --port 27017 --rowcount 10

 Example: Collection-Level Metrics

db.collection.stats()

 Example: System Health Snapshot

db.serverStatus()

8 Auditing in MongoDB

Auditing tracks security events like login, CRUD, and configuration changes.

Available in **MongoDB Enterprise**.

Example Config:

```
auditLog:
  destination: file
  format: BSON
  path: /var/log/mongodb/auditLog.bson
```

9 Security and Backup Validation Checklist

Validation Step	Command	Expected Output
Authentication enabled	mongo --eval "db.runCommand({ connectionStatus: 1 })"	“authInfo” field present
Encrypted connections	db.serverCmdLineOpts()	--tlsMode or requireTLS visible
Backup success	Check /backups/ directory	BSON files created
Restore test	mongorestore --dryRun	Restore validation message
Roles verified	db.getUsers()	Users with roles visible
Audit log active	Check /var/log/mongodb/auditLog.bson	Entries logged
Monitoring live	mongostat / Grafana dashboard	Metrics streaming

10 MongoDB Security Hardening Checklist

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Area	Recommendation
Network Access	Bind MongoDB to specific IPs only
Firewall Rules	Allow only trusted application servers
TLS Everywhere	Use SSL/TLS for all connections
KeyFile Authentication	Required for internal cluster comms
Role-based Access	Follow least-privilege principle
Disable HTTP Interface	Never expose port 28017 publicly
Change Default Ports	Use non-standard ports in production
Rotate Keys	Rotate key files & passwords periodically
Patch Regularly	Keep MongoDB and OS updated
Enable Auditing	For compliance (SOC2, HIPAA, etc.)

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Disaster Recovery (DR) & Maintenance Automation

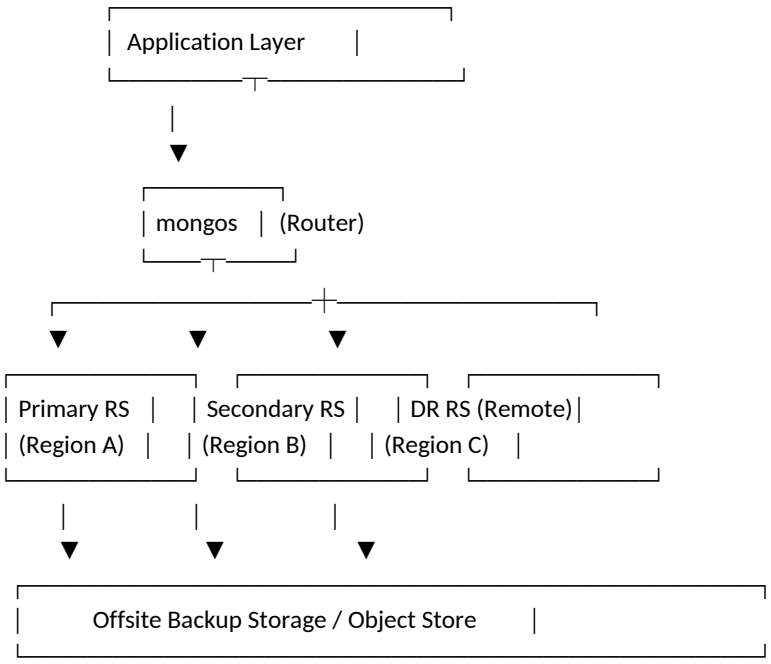
Disaster Recovery (DR) ensures that your MongoDB data and services remain **available, consistent, and restorable** even in the event of hardware failure, network outage, or data corruption.

1 MongoDB Disaster Recovery Fundamentals

DR Component	Description
High Availability (HA)	Automatically handled by Replica Sets
Backups & Snapshots	Protect against logical corruption or accidental deletion
Disaster Recovery Site	Geo-distributed replica or backup
Failover Testing	Regular simulation of outages to validate resiliency
Automation & Monitoring	Early alerts and automated fixes to reduce downtime

2 DR Architecture Overview

Example Multi-Region DR Setup



3 DR Strategies in MongoDB

Strategy	Description	Use Case
Replica Set Replication	Synchronous data replication for HA	Local failover
Delayed Secondary	Secondary node with intentional delay	Ransomware / accidental delete recovery
Hot Standby Cluster (Geo)	Replica set in another region	Regional failover
Snapshot-Based DR	EBS / LVM / Filesystem snapshots	Fast full restoration
Backup & Restore	Logical or physical backups	Long-term retention
Cloud Backup (Atlas / Ops Manager)	Continuous incremental backup	Enterprise-grade DR

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

4️⃣ Configure a Delayed Secondary (for Logical Recovery)

A delayed node can "lag behind" by N hours, allowing rollback of accidental deletions.

Example (delay by 1 hour):

```
cfg = rs.conf()
cfg.members[2].priority = 0
cfg.members[2].hidden = true
cfg.members[2].slaveDelay = 3600
rs.reconfig(cfg)
```

✔ Node 2 will now always be **1 hour behind primary**, providing a rollback window.

5️⃣ Offsite Backup Strategy

Perform regular **encrypted backups** to offsite or cloud storage.

```
mongodump --host primary.mydomain.com \
  --authenticationDatabase admin \
  --gzip --archive=/backups/mongo_$(date +%F).gz
aws s3 cp /backups/mongo_$(date +%F).gz s3://company-dr-bucket/
```

Set up a **daily cron job**:

```
0 2 * * * /usr/bin/mongodump --archive=/backups/mongo_$(date +%F).gz --gzip && aws s3 sync /backups s3://mongo-dr/
```

6️⃣ Disaster Recovery Testing

Regularly test your DR plans to verify reliability.

Test Type	Description	Frequency
Replica Failover	Stop the primary node and verify automatic election	Monthly
Restore Test	Restore backups to a staging cluster	Quarterly
Network Partition Test	Simulate inter-region outage	Biannually
Oplog Replay	Verify recovery from delayed secondary	Quarterly

7️⃣ Automating Maintenance & Health Checks

Automate daily DBA tasks via scripts or cron jobs.

 Example: Health Check Script (mongo_healthcheck.sh)

```
#!/bin/bash
DATE=$(date +%F_%T)
LOG="/var/log/mongo_health_${DATE}.log"

echo "=== MongoDB Health Check: ${DATE} ===" >> $LOG

mongo --eval "
print('--- Server Status ---');
var s = db.serverStatus();
printjson({uptime: s.uptime, connections: s.connections.current, opcounters: s.opcounters});
print('--- Replica Set Status ---');
printjson(rs.status());
print('--- Database Stats ---');
db.adminCommand({listDatabases: 1}).databases.forEach(function(d){print(d.name);});
" >> $LOG
```

Schedule with cron:

```
0 6 * * * /scripts/mongo_healthcheck.sh
```

 Example: Auto-Restart Service on Failure

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Use a **systemd unit override**:

```
[Service]
Restart=always
RestartSec=5
```

Or Linux-level watchdog:

```
systemctl enable mongod
systemctl is-failed mongod && systemctl restart mongod
```

8 Automate Backup Retention Policy

Auto-clean old backups to save storage:

```
find /backups/ -type f -mtime +7 -delete
```

Or keep only last 3 successful archives:

```
ls -1t /backups/*.gz | tail -n +4 | xargs rm -f
```

9 Monitoring for DR Failures

Set up alerts to detect replication lag or node failure.

Metric	Command / Source	Alert Threshold
Replication Lag	rs.printSlaveReplicationInfo()	> 30 seconds
Primary Availability	rs.status().members.stateStr	No PRIMARY present
Disk Space	df -h or db.serverStatus().wiredTiger.cache	< 20% free
Oplog Window	db.printReplicationInfo()	< 30 min coverage
Backup Age	ls -l /backups	Older than 24h

Integrate with:

- **Prometheus + Grafana**
- **CloudWatch**
- **Ops Manager Alerts**
- **Email/SMS scripts**

10 DR Validation Checklist

Validation Step	Command / Action	Expected Result
Backup created	ls /backups	Latest backup present
Restore test	mongorestore --archive=...	Data restored successfully
Failover test	sudo systemctl stop mongod_primary	New primary elected
Lag monitor	rs.printSlaveReplicationInfo()	Lag < threshold
Delayed node check	rs.status()	One node shows SECONDARY + hidden:true
Offsite copy	aws s3 ls s3://mongo-dr	Files present
Auto health log	/var/log/mongo_health_*.log	Daily logs generated

11 Example: Full DR Workflow

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

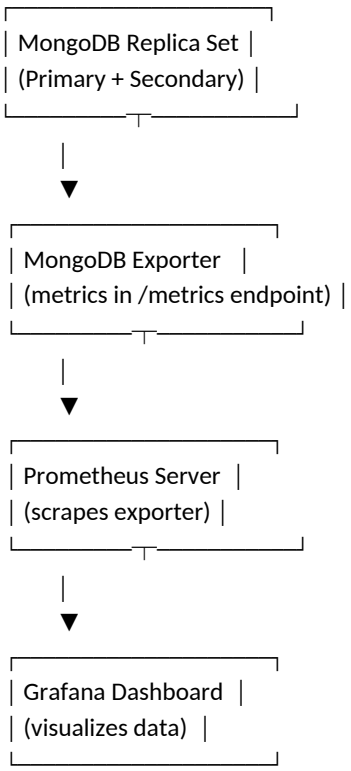
Step	Action	Outcome
1	Regular daily backups (logical or snapshot)	Data protected
2	Continuous replication to secondary/DR site	Real-time sync
3	Health & lag monitoring scripts	Early alerting
4	Test restore every quarter	Confirms data integrity
5	Auto cleanup of old backups	Optimized storage
6	Automated failover	Minimal downtime

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Real-Time Performance Dashboard (Prometheus + Grafana Integration).
This is where you'll learn how to **visualize, monitor, and alert** on MongoDB performance in real time using **open-source monitoring stacks** — exactly what's used in production at top tech companies.

Module 7: MongoDB Real-Time Performance Dashboard (Prometheus + Grafana Integration)
MongoDB produces a wealth of performance metrics — from query latency to replication lag.
This module shows how to **collect, store, and visualize** those metrics using **Prometheus + Grafana** and MongoDB's own **exporters**.

Architecture Overview



Install Prometheus MongoDB Exporter
The **MongoDB Exporter** exposes internal metrics to Prometheus.
Step 1: Install Exporter

```
wget https://github.com/percona/mongodb_exporter/releases/latest/download/mongodb_exporter.tar.gz
tar -xvf mongodb_exporter.tar.gz
sudo mv mongodb_exporter /usr/local/bin/
```

Step 2: Create MongoDB monitoring user

```
use admin
db.createUser({
  user: "metricsUser",
  pwd: "StrongPassword123",
  roles: [ { role: "clusterMonitor", db: "admin" } ]
})
```

Step 3: Run the Exporter

```
mongodb_exporter --mongodb.uri="mongodb://metricsUser:StrongPassword123@localhost:27017/admin"
```

The exporter listens by default on http://localhost:9216/metrics.
✅ You can test it:

```
curl http://localhost:9216/metrics | head
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

3️⃣ Install Prometheus

Step 1: Download and extract

wget https://github.com/prometheus/prometheus/releases/latest/download/prometheus.tar.gz
tar -xvf prometheus.tar.gz
cd prometheus

Step 2: Edit Prometheus config (prometheus.yml)

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: "mongodb"
    static_configs:
      - targets: ["localhost:9216"]
```

Step 3: Start Prometheus

./prometheus --config.file=prometheus.yml

✔ Open Prometheus UI → http://localhost:9090

You can now **query metrics** like:

- mongodb_connections
- mongodb_op_counters_total
- mongodb_repl_lag_seconds

4️⃣ Install Grafana

Step 1: Install Grafana (on RHEL / Ubuntu)

sudo yum install grafana -y
sudo systemctl enable --now grafana-server

Grafana runs on:

👉 http://localhost:3000

(Default login: admin / admin)

5️⃣ Connect Prometheus to Grafana

1. Log into Grafana
2. Navigate to: ⚙ **Configuration → Data Sources → Add data source**
3. Choose **Prometheus**
4. URL: http://localhost:9090
5. Click **Save & Test**

✔ If successful → Grafana can now query Prometheus metrics.

6️⃣ Import MongoDB Dashboard Template

Grafana has community dashboards ready for MongoDB.

Example Dashboard ID (from Grafana.com):

👉 2583 — *MongoDB Overview by Percona*

Steps:

1. Go to Grafana → "+" → "Import"
2. Enter Dashboard ID 2583
3. Select Prometheus as the data source
4. Click **Import**

✔ You'll get a full visual dashboard with:

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

- **Connections / Opcounters**
- **Replication Lag**
- **Memory usage**
- **Lock %**
- **Index misses**
- **Query performance**

7 Custom MongoDB Metrics (Optional)

You can create **custom metrics** using aggregation queries and push them via Pushgateway.

Example: track slow queries count

```
db.system.profile.aggregate([
 { $match: { millis: { $gt: 1000 } } },
 { $count: "slowQueries" }
])
```

You can push the result to Prometheus Pushgateway via script:

```
echo "mongo_slow_queries $(mongo --quiet --eval '...')" | curl --data-binary @- http://pushgateway:9091/metrics/job/mongo
```

8 Alerting Rules in Prometheus

Edit prometheus.yml to include alerting conditions:

```
rule_files:
 - "alerts.yml"
```

Example **alerts.yml**:

```
groups:
 - name: mongodb_alerts
   rules:
    - alert: MongoDBInstanceDown
      expr: up{job="mongodb"} == 0
      for: 1m
      labels:
        severity: critical
      annotations:
        description: "MongoDB instance is down!"
        summary: "MongoDB Down"

    - alert: HighReplicationLag
      expr: mongodb_mongod_replset_member_optime_lag > 30
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: "Replication lag > 30s"
```

9 Integrate Alerts (Email / Slack / Teams)

You can use **Alertmanager** with Prometheus for notifications.

Example Slack webhook config (alertmanager.yml):

```
receivers:
 - name: "slack-notifications"
   slack_configs:
    - send_resolved: true


channel: "#db-alerts"
      username: "MongoDB Monitor"
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
    api_url: "https://hooks.slack.com/services/XXXX/YYYY/ZZZZ"
```

✔ Send alerts to:

- Slack
- Microsoft Teams (via webhook)
- Email / SMS / PagerDuty

📊 10 Sample Dashboard Metrics (Key KPIs)

Metric	Prometheus Query	Alert Threshold
Connections	mongodb_connections{state="active"}	> 80% max
Replication Lag	mongodb_mongod_replset_member_optime_lag	> 30s
Oplog Size	mongodb_mongod_replset_oplog_size_bytes	< 500MB
Lock Percentage	mongodb_mongod_locks_time_locked_global_microseconds_total	> 50%
Slow Queries	mongo_slow_queries	> 10/min
Disk Free %	Node Exporter metric: node_filesystem_free_bytes	< 20%
CPU Load	node_load1	> 70% utilization

☑ 11 Validation Checklist

Step	Validation Command / Action	Expected Output
MongoDB exporter	curl http://localhost:9216/metrics	Metrics visible
Prometheus config	http://localhost:9090/targets	MongoDB target UP
Grafana data source	Grafana UI → Test	“Data source is working”
Dashboard import	Dashboard ID 2583	Visuals loaded
Alerts firing	Stop mongod service	Slack/email notification
Replication lag	rs.printSlaveReplicationInfo()	Matches Grafana chart

🧠 12 Best Practices

✔ Use **dedicated node exporters** for system metrics (CPU, disk, RAM)
✔ Retain Prometheus data for **30–90 days** max (rotate older data)
✔ Always **secure Prometheus and Grafana** with authentication and HTTPS
✔ Keep MongoDB exporter **version-matched** with your MongoDB server
✔ Automate dashboard deployment with **Terraform / Ansible**

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Advanced Automation (Shell + Bash + Python for DBAs)

Automation is essential in MongoDB administration for:

✔ Reducing manual errors

✔ Scaling deployments

✔ Enforcing consistency across servers

✔ Scheduling regular maintenance, health checks, and DR tasks

1 Core Automation Layers

Layer	Language	Typical Use Case
Mongo Shell (JavaScript)	Database-level logic (indexes, stats, cleanup)	
Bash Scripts	OS-level automation (backups, restores, health checks, cron jobs)	
Python (pymongo)	Advanced orchestration, reporting, and API integration	

2 Mongo Shell Scripting (JavaScript Automation)

You can use **Mongo Shell scripts (.js)** to automate administrative tasks.

Example 1: Collect Database Stats

```
// db_stats.js
db.adminCommand({ listDatabases: 1 }).databases.forEach(function (d) {
  db = db.getSiblingDB(d.name);
  stats = db.stats();
  print("DB: " + d.name);
  printjson({
    collections: stats.collections,
    objects: stats.objects,
    dataSizeMB: stats.dataSize / 1024 / 1024,
    storageSizeMB: stats.storageSize / 1024 / 1024
  });
});
```

Run:

```
mongo admin db_stats.js > /tmp/db_stats.log
```

Example 2: Index Usage Report

```
// index_usage.js
db.getCollectionNames().forEach(function (coll) {
  var stats = db[coll].aggregate([{ $indexStats: {} }]);
  print("Collection: " + coll);
  stats.forEach(printjson);
});
```

✔ Use this to find unused or slow indexes.

Example 3: Auto Drop Old Logs / Temporary Collections

```
// cleanup.js
db.getCollectionNames().forEach(function (coll) {
  if (coll.startsWith("tmp_") || coll.endsWith("_log")) {
    db[coll].drop();
    print("Dropped: " + coll);
  }
});
```

3 Bash Scripting for OS-level Automation

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Bash is used for **maintenance**, **monitoring**, and **recovery workflows**.

Example 1: Automated Backup

```
#!/bin/bash
DATE=$(date +%F)
BACKUP_DIR="/backups/mongo"
mkdir -p $BACKUP_DIR

mongodump --archive=$BACKUP_DIR/mongo_$DATE.gz --gzip
find $BACKUP_DIR -type f -mtime +7 -delete
echo "Backup complete for $DATE"
```

Schedule in **cron**:

```
0 2 * * * /scripts/mongo_backup.sh
```

Example 2: Replica Set Health Check

```
#!/bin/bash
mongo --quiet --eval '
  var status = rs.status();
  status.members.forEach(function(m) {
    print(m.name + " - " + m.stateStr + " - lag: " + (m.optimeDate ? (new Date() - m.optimeDate)/1000 : "N/A") + "s");
  });
'
```

✔ Output shows current primary, secondaries, and replication lag.

Example 3: Disk Space and Oplog Monitor

```
#!/bin/bash
df -h /data/db
mongo --quiet --eval 'db.printReplicationInfo()'
```

Example 4: Restart MongoDB Automatically If Down

```
#!/bin/bash
if ! pgrep -x "mongod" > /dev/null
then
  echo "$(date): mongod was down, restarting..." >> /var/log/mongo_watchdog.log
  systemctl restart mongod
fi
```

Cron entry:

```
*/5 * * * * /scripts/mongo_watchdog.sh
```

4 Python Automation with PyMongo

Python is ideal for **complex automation**, **report generation**, and **integration with APIs** (like sending alerts to Teams/Slack).

Install PyMongo

```
pip install pymongo
```

Example 1: MongoDB Connection and Stats

```
from pymongo import MongoClient
client = MongoClient("mongodb://localhost:27017/")
for db_name in client.list_database_names():
    db = client[db_name]
    stats = db.command("dbStats")
    print(f"Database: {db_name} - Size: {stats['dataSize']/1024/1024:.2f} MB")
```

Example 2: Automatic Index Report

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
from pymongo import MongoClient
client = MongoClient("mongodb://localhost:27017/")
db = client["admin"]

for coll_name in db.list_collection_names():
    indexes = db[coll_name].index_information()
    print(f"Collection: {coll_name}, Indexes: {list(indexes.keys())}")
```

Example 3: Alert on High Replication Lag

```
from pymongo import MongoClient
import smtplib

client = MongoClient("mongodb://localhost:27017/")
status = client.admin.command("replSetGetStatus")

for member in status["members"]:
    if "SECONDARY" in member["stateStr"]:
        lag = (status["date"] - member["optimeDate"]).total_seconds()
        if lag > 30:
            msg = f"ALERT: {member['name']} lagging by {lag} seconds"
            print(msg)
            # send email alert
```

Example 4: Backup to AWS S3

```
import os
import boto3
from datetime import datetime

DATE = datetime.now().strftime("%Y-%m-%d")
os.system(f"mongodump --archive=/backups/mongo_{DATE}.gz --gzip")

s3 = boto3.client("s3")
s3.upload_file(f"/backups/mongo_{DATE}.gz", "mongo-dr-bucket", f"mongo_{DATE}.gz")
print("Backup uploaded to S3.")
```

5 Centralized Scheduler (Crontab or Airflow)

You can orchestrate all automation scripts using:

- **Linux Crontab** for simple scheduling
- **Apache Airflow** for advanced DAG-based scheduling
- **Ansible / Jenkins** for deployment automation

Example Crontab:

```
0 2 * * * /scripts/mongo_backup.sh
0 6 * * * /scripts/mongo_healthcheck.sh
*/5 * * * * /scripts/mongo_watchdog.sh
```

6 MongoDB DBA Automation Toolkit Summary

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Category	Script Type	Function
Backups	Bash / Python	Automated daily logical dump & upload
Health Checks	Bash / Mongo Shell	Monitor uptime, lag, replication
DR Validation	Bash / Mongo Shell	Verify restores and replication delay
Performance Tuning	Python	Collect metrics, report top queries
Index Management	Mongo Shell	List unused or duplicate indexes
Cleanup	Mongo Shell / Bash	Purge temp collections, old logs
Alerting	Python	Send email/Slack alerts
Scaling Ops	Ansible / Airflow	Rolling updates, cluster expansion

7 Validation Checklist

Checkpoint	Validation Command / Output	Expected Result
Scripts executable	chmod +x /scripts/*.sh	No permission errors
Crontab entries	crontab -l	Correct schedule shown
Mongo Shell automation	mongo admin cleanup.js	Tasks executed
Python scripts	python backup_s3.py	File uploaded
Backup restore	mongorestore --archive=...	Data restored successfully
Health logs	/var/log/mongo_health_*.log	Daily log files present

8 Best Practices for MongoDB Automation

✔ Always run scripts as **non-root** MongoDB user
✔ Store credentials securely (use .env files or Vault)
✔ Log all automation actions to /var/log/mongodb/
✔ Test automation in **staging** before applying to production
✔ Use **Git** for version control of scripts
✔ Implement **idempotent logic** (safe to run multiple times)
✔ Document and schedule **DR drills** regularly

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Most critical parts of MongoDB administration — securing your databases against unauthorized access, data theft, and configuration leaks. This module covers **role-based access control (RBAC)**, **encryption**, **network hardening**, and **audit logging**, all with real-world commands and automation examples.

Module 9: MongoDB Security Hardening & User Management

Objective: Ensure MongoDB deployment is secure at every layer — authentication, authorization, encryption, and auditing.

1 MongoDB Security Layers Overview

Layer	Security Feature	Description
Authentication	SCRAM / x.509 / LDAP	Verify who can connect
Authorization	RBAC (Roles & Privileges)	Control what actions are allowed
Encryption	TLS/SSL + Encryption at Rest	Protect data in motion and at rest
Auditing	Action Logs	Record who did what
Network Hardening	Firewall / Bind IP	Limit access scope
Secrets Management	Vault / Keyfile	Store credentials safely

2 Enable Authentication in MongoDB

By default, MongoDB starts **without authentication**. Always enable it before going production.

Step 1: Edit /etc/mongod.conf

```
security:
  authorization: enabled
```

Restart MongoDB:

```
sudo systemctl restart mongod
```

Step 2: Create Admin User (first connection must be without auth)

```
use admin
db.createUser({
  user: "adminUser",
  pwd: "StrongPassword#123",
  roles: [ { role: "root", db: "admin" } ]
})
```

✔ Now connect with authentication:

```
mongo -u "adminUser" -p "StrongPassword#123" --authenticationDatabase "admin"
```

3 Role-Based Access Control (RBAC)

MongoDB uses **roles** to define what users can do.

Built-in Roles

Role	Description
read	Read-only access to a database
readWrite	Read and write access
dbAdmin	Administer a specific DB
userAdmin	Create/modify users on DB
clusterAdmin	Manage cluster-wide settings
backup / restore	Perform backup/restore operations
root	Superuser access (all privileges)

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Create Application User

```
use appDB
db.createUser({
  user: "appUser",
  pwd: "AppSecure#321",
  roles: [ { role: "readWrite", db: "appDB" } ]
})
```

✔ Validate:

```
db.getUser("appUser")
```

Custom Role Example

```
use admin
db.createRole({
  role: "readWriteMonitor",
  privileges: [
    { resource: { db: "sales", collection: "" }, actions: [ "find", "insert", "update" ] },
    { resource: { cluster: true }, actions: [ "serverStatus" ] }
  ],
  roles: []
})
```

Assign role:

```
db.grantRolesToUser("appUser", [ { role: "readWriteMonitor", db: "admin" } ])
```

🔒 4 Enable TLS/SSL for Encrypted Connections

Step 1: Generate Certificates

```
openssl req -newkey rsa:4096 -new -x509 -days 365 -nodes -out mongodb.crt -keyout mongodb.key -subj "/CN=yourdomain.com"
cat mongodb.key mongodb.crt > mongodb.pem
```

Step 2: Edit mongod.conf

```
net:
  tls:
    mode: requireTLS
    certificateKeyFile: /etc/ssl/mongodb.pem
```

Restart MongoDB:

```
sudo systemctl restart mongod
```

✔ Verify:

```
openssl s_client -connect localhost:27017
```

🛡 5 Network Hardening

Bind MongoDB to Local Interface

In /etc/mongod.conf:

```
net:
  bindIp: 127.0.0.1
```

Or for a private network:

```
net:
  bindIp: 127.0.0.1,10.0.0.5
```

Firewall Rules

```
sudo ufw allow from 10.0.0.0/24 to any port 27017
sudo ufw deny 27017/tcp
```

✔ Never expose port 27017 directly to the Internet.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

6 Keyfile Authentication (for Replica Sets)

Ensures secure intra-cluster communication.

Step 1: Create keyfile

```
openssl rand -base64 756 > /etc/mongod.keyfile
chmod 600 /etc/mongod.keyfile
chown mongod:mongod /etc/mongod.keyfile
```

Step 2: Update /etc/mongod.conf

```
security:
  keyFile: /etc/mongod.keyfile
```

Restart MongoDB on all replica set members.

All members with the same keyfile will automatically authenticate.

7 Encryption at Rest (Enterprise or Cloud)

Option 1: MongoDB Enterprise (Encrypted Storage Engine)

Enable in mongod.conf:

```
security:
  enableEncryption: true
  encryptionKeyFile: /etc/mongo_encryption.key
```

Generate key:

```
openssl rand -base64 32 > /etc/mongo_encryption.key
chmod 600 /etc/mongo_encryption.key
```

Option 2: Cloud / OS-level Encryption

Use:

- AWS EBS Encryption
- Azure Disk Encryption
- Linux LUKS or dm-crypt

8 Enable Audit Logging

Audit logs record all actions (login, CRUD, role changes, etc.)

Step 1: Edit /etc/mongod.conf

```
auditLog:
  destination: file
  format: JSON
  path: /var/log/mongodb/audit.log
```

Restart MongoDB:

```
sudo systemctl restart mongod
```

✔ Verify:

```
cat /var/log/mongodb/audit.log | jq '.atype'
```

You'll see entries like:

```
"atype": "authenticate"
"atype": "createUser"
"atype": "dropCollection"
```

9 MongoDB Security Checklist

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Area	Command / Action	Validation
Authentication	mongo --authenticationDatabase admin	Login required
Authorization	db.runCommand({connectionStatus:1})	Shows assigned roles
TLS enabled	`netstat -tlpn	grep 27017`
Keyfile	`cat /etc/mongod.keyfile	wc -c`
Firewall	ufw status	Port restricted
Audit logs	cat /var/log/mongodb/audit.log	Activity recorded
Encryption	Check config file	enableEncryption: true

⚡🔟 **Python Security Audit Script Example**

```
from pymongo import MongoClient
import json

client = MongoClient("mongodb://adminUser:StrongPassword#123@localhost:27017/admin")
status = client.admin.command("connectionStatus")

print("User Roles:")
print(json.dumps(status['authInfo']['authenticatedUserRoles'], indent=2))

print("TLS Enabled:", client.admin.command("getCmdLineOpts")['parsed']['net']['tls'])
```

✔ Use this to quickly validate authentication and TLS across servers.

🔐1️⃣1️⃣ **Security Best Practices**

✔ Disable unused databases (test, sampleDB)
✔ Rotate passwords and keyfiles regularly
✔ Restrict OS access to mongod user only
✔ Enable **AppArmor / SELinux** for process isolation
✔ Monitor failed logins (audit.log)
✔ Disable --noauth and --bind_ip_all
✔ Regularly update MongoDB to latest stable patch

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Performance Tuning & Query Optimization, one of the most **critical and advanced** areas for any MongoDB Database Administrator. This module helps you identify performance bottlenecks, optimize queries, design efficient indexes, and fine-tune MongoDB for production-grade workloads.

⚙ Module 10: MongoDB Performance Tuning & Query Optimization (Advanced)

🎯 **Goal:** Detect and eliminate performance bottlenecks by analyzing queries, indexes, memory, and I/O efficiency.

1️⃣ Key Performance Pillars

Area	Description	Tools
Query Efficiency	Optimize query filters, projections, and plans	explain(), profiler, db.currentOp()
Indexing Strategy	Create efficient single and compound indexes	db.collection.getIndexes()
Hardware & Memory	Leverage RAM, CPU cores, and storage	serverStatus, top, free -h
Connection Management	Optimize concurrent operations	Connection pooling
Schema Design	Denormalization, embedding, referencing	Schema analysis tools

🔍2️⃣ Identify Slow Queries

Enable the Database Profiler:

use admin
db.setProfilingLevel(1, { slowms: 100 })
This logs queries taking more than 100 ms.
✔ View slow queries:
use appDB
db.system.profile.find().sort({ ts: -1 }).limit(5).pretty()
✔ Disable after analysis:
db.setProfilingLevel(0)

🧠3️⃣ Use explain() to Analyze Query Plans

Example 1: Basic explain()

db.orders.find({ customerId: 123 }).explain("executionStats")

Key Fields to Watch:

Field	Meaning
executionTimeMillis	Time taken to execute
nReturned	Documents returned
totalDocsExamined	Docs scanned
totalKeysExamined	Index entries scanned
winningPlan.stage	Whether index scan or collection scan

🌤 **Goal:** Keep totalDocsExamined as close as possible to nReturned.

{}4️⃣ Index Tuning

Check existing indexes

db.orders.getIndexes()

Create index

db.orders.createIndex({ customerId: 1, orderDate: -1 })

Drop unused indexes

db.orders.dropIndex("customerId_1_orderDate_-1")

Find unused indexes

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
db.collection.aggregate([
 { $indexStats: {} },
 { $match: { accesses: { ops: { $eq: 0 } } } }
])
```

5️⃣ Compound vs. Multikey Indexes

Index Type	Use Case	Example
Single Field	Simple lookups	{ customerId: 1 }
Compound Index	Multiple filter fields	{ customerId: 1, orderDate: -1 }
Multikey Index	Array fields	{ tags: 1 }

💡 **Tip:** Order matters!
Use the most selective field first in compound indexes.

6️⃣ Query Optimization Examples

✖ Bad Query

```
db.orders.find({ total: { $gt: 100 } })
```

Scans entire collection.

✔ Optimized Query with Index

```
db.orders.createIndex({ total: 1 })
db.orders.find({ total: { $gt: 100 } }).hint({ total: 1 })
```

Covered Queries

When all requested fields are part of the index — no document lookup needed.

```
db.orders.createIndex({ customerId: 1, orderDate: 1, total: 1 })
db.orders.find({ customerId: 123 }, { _id: 0, orderDate: 1, total: 1 }).explain("executionStats")
```

✔ You'll see "indexOnly": true in explain() output.

7️⃣ Query Patterns to Avoid

Pattern	Problem	Fix
$regex without anchor	Full scan	Use anchored regex: ^ABC
$ne or $nin	Non-selective	Avoid unless indexed
$or across many fields	Multiple index scans	Use compound index
Large $in arrays	High memory	Paginate or batch
Unindexed $lookup	Slow joins	Index foreign fields

8️⃣ Optimize Write Performance

Technique	Description
Bulk Writes	Use bulkWrite() instead of many single inserts
Write Concern	Use { w: 1 } for speed, { w: "majority" } for safety
Journaling	Keep enabled, but ensure fast disk I/O
Pre-allocate storage	For predictable I/O performance

Example:

```
db.orders.bulkWrite([
 { insertOne: { document: { orderId: 1, total: 100 } } },
 { insertOne: { document: { orderId: 2, total: 200 } } }
])
```

9️⃣ Hardware & Server Tuning

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Area	Recommendation
RAM	Keep working set (indexes + hot data) in memory
Disk	Use SSDs for data and journaling
CPU	Multi-core (for concurrent queries)
NUMA	Disable for consistent performance
Filesystem	Use XFS over ext4
Swap	Disable swap (prefer large RAM)

Check server status:
db.serverStatus().mem
db.serverStatus().connections

📊 🔟 Monitoring & Profiling Tools

Tool	Purpose
mongostat	Live server metrics
mongotop	Collection read/write timing
Atlas Performance Advisor	Cloud performance insights
db.currentOp()	Active operations
serverStatus()	Overall health snapshot
Profiler	Query-level performance analysis

Example:
mongostat --rowcount 10

1️⃣1️⃣ Connection Pool Tuning (Application Level)

For Node.js (Mongoose):

```
mongoose.connect(uri, {
  maxPoolSize: 100,
  minPoolSize: 10,
  connectTimeoutMS: 5000,
  socketTimeoutMS: 45000
})
```

💡 Tip: Match pool size to number of CPU cores × active clients.

🧠 1️⃣2️⃣ Schema Design Optimization

Pattern	Description	Benefit
Embedding	Store related data in one document	Fast reads
Referencing	Store _id references only	Smaller documents
Bucket Pattern	Group related time-series data	Improves analytics
Subset Pattern	Store only active subset	Faster lookups

Example — **Embedding for Speed:**

```
{
  _id: 1,
  customer: { id: 101, name: "John" },
  orders: [
    { id: 1, total: 200 },
    { id: 2, total: 150 }
  ]
}
```

1️⃣3️⃣ Validation: Performance Audit Checklist

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Area	Validation Command	Expected
Slow queries	db.system.profile.find()	< 100ms ideally
Index scan ratio	explain()	IXSCAN not COLLSCAN
Memory use	db.serverStatus().mem	< 80% utilization
Connections	db.serverStatus().connections	No overload
Disk I/O	iostat -xm 1	Low I/O wait
Index stats	$indexStats	All active
Query pattern	$currentOp()	No blocked ops

🔟4️⃣ Bonus — Automated Performance Report (Python)

```
from pymongo import MongoClient
import pprint

client = MongoClient("mongodb://adminUser:StrongPassword#123@localhost:27017/admin")

server_status = client.admin.command("serverStatus")
profiled_queries = client["appDB"]["system.profile"].find().sort([("millis", -1)]).limit(5)

print("=== MongoDB Performance Summary ===")
print(f"Uptime: {server_status['uptime']} seconds")
print(f"Connections: {server_status['connections']}")
print(f"Memory: {server_status['mem']}")
print("\nTop Slow Queries:")
for q in profiled_queries:
    pprint.pprint(q)
```

✔ Run this weekly to generate automatic tuning reports.

□ Summary: Key Performance Wins

✔ Use **explain()** to analyze query plans
✔ Maintain **compound indexes** on frequently filtered fields
✔ Keep working set **in memory (RAM)**
✔ Use **profiling** to detect bottlenecks
✔ Avoid **collection scans** and **unbounded regex**
✔ Apply **schema patterns** (embedding, bucket, subset)
✔ Optimize app-level connection pools

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Backup & Restore Automation in MongoDB, one of the most vital areas for any Database Administrator.
In this module, you'll master **logical backups**, **physical snapshots**, **point-in-time recovery**, and **automating backups** using scripts and schedulers — both for **on-prem** and **MongoDB Atlas** deployments.

Module 11: MongoDB Backup, Restore & Automation
Objective: Learn how to perform full, incremental, and automated backups of MongoDB databases — and restore them with zero data loss.

1 Backup Approaches Overview

Type	Tool	Description	Use Case
Logical Backup	mongodump / mongorestore	Exports BSON files	Migration, versioned backups
Physical Backup	File-system level or LVM snapshot	Copies data directory	Fast restore for large data
Cloud Snapshot	MongoDB Atlas / Ops Manager	Automated backups	Production-grade HA
Oplog-based (Incremental)	mongodump + oplog.bson	Point-in-time recovery	Replica sets

2 Full Database Backup using mongodump
Basic Syntax:
mongodump --host localhost --port 27017 --out /backups/mongodump-$(date +%F)
Backup a specific database:
mongodump --db salesDB --out /backups/sales-$(date +%F)
Backup a specific collection:
mongodump --db salesDB --collection orders --out /backups/orders-$(date +%F)
✔ **Output Folder Example:**

```
/backups/sales-2025-10-14/
└── salesDB/
    ├── orders.bson
    └── orders.metadata.json
```

3 Backup with Authentication
mongodump --uri "mongodb://adminUser:StrongPassword#123@localhost:27017/salesDB?authSource=admin" --out /backups/secure-sales

4 Backup Replica Set with Oplog (For Point-in-Time Restore)
Use --oplog flag:
mongodump --host replica1:27017 --out /backups/replica-dump-$(date +%F) --oplog
✔ This captures **oplog.bson**, enabling **consistent snapshots** across the replica set.

5 Restore MongoDB Backups with mongorestore
Restore Entire Dump:
mongorestore /backups/mongodump-2025-10-14/
Restore Specific Database:
mongorestore --db salesDB /backups/sales-2025-10-14/salesDB/
Restore Collection:
mongorestore --collection orders --db salesDB /backups/orders-2025-10-14/salesDB/orders.bson
Overwrite Existing Data:
mongorestore --drop /backups/mongodump-2025-10-14/

6 Restore with Authentication
mongorestore --uri "mongodb://adminUser:StrongPassword#123@localhost:27017/salesDB?authSource=admin" /backups/secure-sales

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

7️⃣ Validate Backup & Restore

Verify backup folder

ls -lh /backups/mongodump-2025-10-14

Compare document counts:

db.orders.countDocuments()

After restore, check:

db.orders.countDocuments()

✔ Counts should match pre-backup snapshot.

⚙️ 8️⃣ Automating Backups with Cron Jobs (Linux)

Create script /scripts/mongo_backup.sh:

```
#!/bin/bash
BACKUP_DIR="/backups/mongodump-$(date +%F)"
mongodump --uri "mongodb://adminUser:StrongPassword#123@localhost:27017/?authSource=admin" --out "$BACKUP_DIR"
find /backups/* -type d -mtime +7 -exec rm -rf {} \;
echo "MongoDB backup completed on $(date)" >> /var/log/mongo_backup.log
```

Make it executable:

chmod +x /scripts/mongo_backup.sh

Add cron job:

crontab -e

0 2 * * * /scripts/mongo_backup.sh

✔ Runs every night at 2:00 AM.

☁️ 9️⃣ MongoDB Atlas Backup Automation

Type	Feature	Description
Continuous Cloud Backup	Point-in-time restore	Automated snapshots every few minutes
Snapshot Frequency	Configurable	Hourly, daily, weekly
Restore Targets	Same cluster or new	Easy clone or rollback

Access via:

Atlas UI → Cluster → Backup → Snapshots

Or CLI:

atlas backups snapshots list --clusterName Cluster0

atlas backups restore start --snapshotId 123456789 --clusterName Cluster0

📦 🔟 Physical (File-Level) Backups

Used for **large datasets** or **hot backups** in Enterprise setups.

Steps:

1. Stop MongoDB or use fsync lock
2. db.fsyncLock()
3. Copy data directory
4. rsync -av /var/lib/mongo /mnt/backup/mongo-data
5. Unlock database
6. db.fsyncUnlock()

🌤️ Use **LVM snapshots** for hot backups without downtime.

🗜️ 1️⃣1️⃣ Compression and Storage Optimization

Compress backup folder:

tar -czf /backups/sales-$(date +%F).tar.gz /backups/sales-$(date +%F)

Store in S3:

aws s3 cp /backups/sales-2025-10-14.tar.gz s3://mongodb-backups/

List S3 backups:

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

aws s3 ls s3://mongodb-backups/

🔟2️⃣ Backup & Restore Validation Checklist

Task	Command	Expected Result
Backup completed	echo $?	0 (success)
Folder exists	ls /backups/mongodump-*	Backup timestamped
Restore verified	db.orders.countDocuments()	Matches original
Oplog captured	ls /backups/*/oplog.bson	Exists
Cron job logs	cat /var/log/mongo_backup.log	Daily entries

1️⃣3️⃣ Sample Automation (Python Script)

```
import os, datetime
from pymongo import MongoClient

backup_dir = f"/backups/mongodump-{datetime.date.today()}"
os.system(f"mongodump --uri 'mongodb://adminUser:StrongPassword#123@localhost:27017/?authSource=admin' --out {backup_dir}")

# Retain 7 days
os.system("find /backups/* -type d -mtime +7 -exec rm -rf {} \;")

client = MongoClient("mongodb://adminUser:StrongPassword#123@localhost:27017/admin")
print("Backup verified:", os.path.exists(backup_dir))
print("Current DB Count:", len(client.list_database_names()))
```

1️⃣4️⃣ Advanced: Incremental Backups using Oplog Replay

Extract operations after last backup timestamp:

```
mongodump --oplog --oplogStart 2025-10-13T10:00:00Z --out /backups/incremental-$(date +%F)
```

Replay oplog to restore changes:

```
mongorestore --oplogReplay /backups/incremental-2025-10-14/
```

✔ Used for near **zero data loss recovery** in replica sets.

1️⃣5️⃣ Best Practices Summary

✔ Always use **--oplog** for replica backups
✔ Automate with cron or systemd timers
✔ Rotate & compress backups regularly
✔ Store copies offsite (S3, Azure Blob)
✔ Test restore quarterly — **a backup is only as good as its restore**
✔ Monitor /var/log/mongo_backup.log for failures
✔ Use **Atlas Snapshots** for production clusters

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Real-world MongoDB Monitoring, Logging & Alerting — the heartbeat of every DBA's operational toolkit.
Monitoring ensures you catch issues *before* they affect performance, replication, or availability. This module covers both **native tools** and **enterprise-grade platforms** like **Ops Manager**, **Atlas**, and **open-source integrations (Prometheus + Grafana)**.

Module 12: MongoDB Monitoring, Logging & Alerting
Objective: Set up and manage real-time monitoring, logging, and alerting for MongoDB across standalone, replica set, and sharded deployments.

1 Monitoring Overview

Category	Purpose	Common Tools
Database Metrics	Monitor performance, connections, ops/sec	mongostat, mongotop, Atlas, Ops Manager
System Metrics	CPU, Memory, Disk I/O	vmstat, iostat, sar
Query Profiling	Analyze slow queries	Profiler, explain()
Log Monitoring	Capture errors, restarts, replication events	/var/log/mongodb/mongod.log
Alerting	Notify on critical thresholds	Atlas alerts, Prometheus, Ops Manager

2 Native Monitoring Tools

a. mongostat — Real-Time MongoDB Performance

```
mongostat --rowcount 5
```

Sample Output:

```
insert query update delete getmore command dirty used flushes vsize res faults
   12     3     1     0      0    55|0     1%  80%       0  2.5G 1.1G      0
```

Metric	Meaning
insert, query, update	Operations per second
flushes	Disk writes
dirty, used	Cache usage
vsize, res	Memory footprint

b. mongotop — Collection-Level Activity

```
mongotop 5
```

Shows read/write time per collection every 5 seconds.
Example:

```
ns                  total   read  write
sales.orders          150ms  50ms  100ms
inventory.products     10ms  10ms    0ms
```

Use this to find **collections causing I/O hotspots.**

c. MongoDB Shell Commands

```
db.serverStatus().opcounters
db.serverStatus().connections
db.serverStatus().mem
db.currentOp()
```

These return:

- Current read/write/query operations
- Active connection count
- Memory usage summary
- Running operations (helpful for killing long-running queries)

3 MongoDB Logs

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Default log location (Linux):
/var/log/mongodb/mongod.log
Enable verbose logging (temporarily):
db.setLogLevel(2)
Return to default:
db.setLogLevel(0)
View last 20 lines:
tail -n 20 /var/log/mongodb/mongod.log
Look for:

- NETWORK — Connection issues
- STORAGE — Disk errors
- REPL — Replication lag
- QUERY — Slow or unindexed queries

4 Query Profiler for Deep Performance Monitoring

Enable profiler for slow queries:
db.setProfilingLevel(1, { slowms: 100 })
View profiled queries:
db.system.profile.find().sort({ ts: -1 }).limit(5).pretty()
✔ Use this alongside logs to **trace query latency.**

5 Ops Manager (On-Prem Monitoring & Backup Tool)

MongoDB Ops Manager (for enterprise or self-hosted clusters) provides:

- Real-time dashboards
- Performance metrics
- Alerts and backup management
- Visual schema & index analysis

Install & Configure:

1. Install Ops Manager packages
2. Create MongoDB Monitoring Agents
3. Register servers in Ops Manager UI
4. Configure dashboards for:
    - Operations per second
    - Cache usage
    - Replication lag
    - Disk I/O and memory usage

UI Sections:

- **Clusters:** View health status
- **Performance:** Query latency, CPU usage
- **Alerts:** Threshold-based triggers

6 MongoDB Atlas Monitoring & Alerts (Cloud)

MongoDB Atlas provides **built-in monitoring** with zero setup.
Metrics Include:

- **Operations:** Ops/sec, latency, cache hit ratio
- **Hardware:** CPU, memory, IOPS
- **Replication:** Lag and election metrics
- **Indexes:** Usage and efficiency

Enable Alerts in Atlas UI → Alert Settings
Example Alerts:

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Condition	Threshold	Action
Disk usage > 85%	85%	Email / Slack
Replication lag > 10s	10 seconds	SMS
CPU > 90% for 5m	90%	PagerDuty trigger

You can export metrics to:

- **CloudWatch**
- **Datadog**
- **PagerDuty**
- **Opsgenie**

7️⃣ Open Source Monitoring (Prometheus + Grafana)

Step 1: Install MongoDB Exporter

```
sudo apt install prometheus-mongodb-exporter
```

Step 2: Configure Prometheus target

/etc/prometheus/prometheus.yml

```
scrape_configs:
  - job_name: 'mongodb'
    static_configs:
      - targets: ['localhost:9216']
```

Step 3: Start services

```
sudo systemctl restart prometheus
sudo systemctl restart grafana-server
```

Step 4: Import MongoDB Dashboard in Grafana

- Use **dashboard ID: 2583** from Grafana.com

✔ Get live visual metrics like:

- Query latency
- Connections
- Replication lag
- Disk utilization

8️⃣ Alerting & Automation Examples

Email Alert for Replica Lag (Shell Script)

```
#!/bin/bash
LAG=$(mongo --quiet --eval "rs.printSlaveReplicationInfo()" | grep "sec behind" | awk '{print $4}')
if [ "$LAG" -gt 10 ]; then
  echo "MongoDB replication lag detected: $LAG seconds" | mail -s "MongoDB Alert" admin@company.com
fi
```

Schedule:

```
crontab -e
*/5 * * * * /scripts/check_replication_lag.sh
```

9️⃣ Health Check Summary Commands

Area	Command	Description
Connections	db.serverStatus().connections	Active & available connections
Replication	rs.status()	Lag, election, member health
CPU & Mem	db.serverStatus().mem	Memory usage
I/O	iostat -xm 1	Disk latency
Ops Count	db.serverStatus().opcounters	Query & insert rates
Top Collections	mongotop 5	Collection-level I/O time

⚡ 🔟 Advanced: Custom Metrics with Aggregation

Create monitoring collection:

```
db.createCollection("monitoring")
```

Insert periodic metrics:

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
db.monitoring.insert({
  ts: new Date(),
  mem: db.serverStatus().mem,
  connections: db.serverStatus().connections,
  opcounters: db.serverStatus().opcounters
})
```

Query dashboard metrics:

```
db.monitoring.find().sort({ ts: -1 }).limit(3).pretty()
```

🔟1️⃣ Log Rotation and Retention

Manual rotation:

```
logrotate /etc/logrotate.d/mongod
```

Example config:

```
/etc/logrotate.d/mongod
/var/log/mongodb/mongod.log {
  daily
  rotate 7
  compress
  missingok
  notifempty
  create 640 mongod adm
  postrotate
    /bin/kill -USR1 `pidof mongod`
  endscript
}
```

✔ Keeps 7 days of compressed logs automatically.

1️⃣2️⃣ Validation Checklist

Task	Validation Command	Expected
mongostat output	mongostat --rowcount 3	Shows live metrics
Log access	tail -f /var/log/mongodb/mongod.log	No permission errors
Profiler enabled	db.getProfilingStatus()	was = 1
Prometheus metrics	curl localhost:9216/metrics	HTTP 200
Grafana dashboard	Web UI	Metrics visible
Atlas alerts	Atlas UI	Active & configured

1️⃣3️⃣ Best Practices Summary

✔ Use **Profiler** and **mongostat** regularly
✔ Enable **Atlas or Ops Manager alerts**
✔ Store logs on separate disks or S3
✔ Automate **log rotation**
✔ Monitor **replication lag** continuously
✔ Integrate **Prometheus + Grafana** for visualization
✔ Use **email/SMS alerts** for production events
✔ Keep monitoring agents updated

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

One of the *most critical* MongoDB DBA areas: **High Availability (HA)** and **Disaster Recovery (DR)**.
This is where you learn to keep MongoDB online, resilient, and recoverable — even if hardware fails, data centers go down, or someone deletes data by mistake.

MongoDB High Availability & Disaster Recovery (HA/DR)

🎯 **Objective:** Design, implement, and test MongoDB HA & DR strategies using replica sets, backups, and failover simulations to ensure 99.9 %+ uptime and no data loss.

1️⃣ What Is High Availability (HA)?

High Availability ensures **continuous database service** even during:

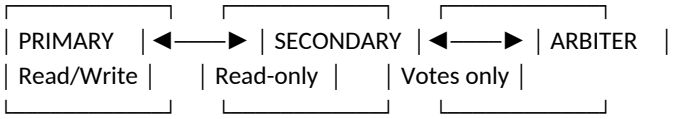
- Server or hardware failure
- Network partition
- Maintenance or upgrades

MongoDB achieves HA through **Replica Sets**.

Concept	Description
Primary Node	Handles reads and writes
Secondary Node(s)	Replicates data from primary (async replication)
Arbiter	Participates in elections but holds no data
Automatic Failover	When the primary fails, a secondary becomes the new primary

2️⃣ Replica Set Architecture

Typical 3-Node Setup



Minimum Recommended:

- 3 voting members (odd number)
- Spread across **different availability zones / servers**

3️⃣ Setting Up a Replica Set

Step 1: Create Data Directories

```
mkdir -p /data/rs0/{node1,node2,node3}
```

Step 2: Start Each mongod

```
mongod --replSet "rs0" --port 27017 --dbpath /data/rs0/node1 --bind_ip localhost,host1 --fork --logpath /data/rs0/node1/mongod.log
mongod --replSet "rs0" --port 27018 --dbpath /data/rs0/node2 --bind_ip localhost,host2 --fork --logpath /data/rs0/node2/mongod.log
mongod --replSet "rs0" --port 27019 --dbpath /data/rs0/node3 --bind_ip localhost,host3 --fork --logpath /data/rs0/node3/mongod.log
```

Step 3: Initialize Replica Set

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "host1:27017" },
    { _id: 1, host: "host2:27018" },
    { _id: 2, host: "host3:27019", arbiterOnly: true }
  ]
})
```

Check status:

```
rs.status()
```

4️⃣ Failover Process (Automatic)

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

When the **Primary** becomes unreachable:

1. Secondary members detect heartbeat timeout (~10 s).
2. Election is triggered.
3. Secondary with highest priority becomes **Primary**.
4. Clients automatically reconnect using **MongoDB connection string**.

Example:
mongodb://host1,host2,host3/?replicaSet=rs0
✔ Drivers handle failover transparently.

🧯 Disaster Recovery (DR) Overview

DR Concept	Purpose
Backups	Protect data from corruption or accidental deletion
Offsite Copy	Store backups in another region or S3
Point-in-Time Recovery	Restore data to specific timestamps
Testing	Regular restore validation
Automation	Cron jobs, Ops Manager, or Atlas Backups

💾 Backup & Restore Strategies

Option A: mongodump / mongorestore (Logical Backup)
mongodump --host host1 --port 27017 --out /backups/$(date +%F)
mongorestore --drop /backups/2025-10-14
✔ Good for small–medium DBs
⚠ No point-in-time recovery

Option B: filesystem Snapshot (Physical Backup)
Stop MongoDB briefly or use **fsyncLock**:
db.fsyncLock()
Then back up the data directory (/var/lib/mongo) using:
tar -czf /backups/mongo-data-$(date +%F).tgz /var/lib/mongo
After backup:
db.fsyncUnlock()
✔ Fast & consistent snapshot
⚠ Requires storage-level tools (EBS, LVM, SAN)

Option C: MongoDB Atlas Backups (Cloud)

- Automated daily snapshots
- Continuous cloud backup (oplog-based)
- Point-in-time restore (PITR)
- Encryption + multi-region redundancy

Restore directly from the Atlas UI.

Option D: Ops Manager Backups (On-Prem Enterprise)
Features:

- Continuous incremental backup
- PITR
- Restore any timestamp or cluster
- Compression & deduplication
- Monitoring integration

🧪 Validating Backups

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Validation Step	Command	Expected
Check dump size	du -sh /backups/latest	Sufficient data size
List collections	mongorestore --dryRun	No errors
Test restore	mongorestore --db testrestore /backups/latest/testdb	Successful import
Compare doc count	db.stats().count	Matches original

8 Cross-Region or DR Site Replication

Option 1: Geo-Distributed Replica Set

Deploy one secondary in another region:

rs.add({ host: "us-east-dr.mongodb.net:27017", priority: 0, hidden: true })

✔ Keeps an up-to-date copy for DR

✔ Hidden node avoids client queries

Option 2: Backup Copy to Cloud Storage

Automate:

aws s3 cp /backups/2025-10-14 s3://mongo-dr-backups/

9 Testing Failover & Recovery

Test 1: Simulate Primary Failure

sudo systemctl stop mongod

Observe new primary election with:

rs.status()

Test 2: Data Recovery Test

1. Drop a test collection.
2. Restore from backup.
3. Verify data integrity.

10 DR Runbook Example (Production SOP)

Step	Action	Tool/Command
1	Confirm failure	rs.status()
2	Identify last healthy node	rs.printSlaveReplicationInfo()
3	Promote secondary manually (if needed)	rs.stepUp()
4	Restore from backup if corruption	mongorestore /backups/...
5	Validate replication resumed	rs.status()
6	Notify stakeholders	Email/Slack Ops Channel
7	Post-mortem documentation	Jira / Confluence

11 Key Metrics to Monitor

Metric	Description	Threshold
Replication Lag	Delay in syncing secondaries	< 10 seconds
Oplog Window	Time range of replication log	> 24 hours
Disk I/O	Write latency	< 10 ms
Node Availability	Node uptime	99.9 %
Backup Size	Backup growth trend	Track weekly

12 Hands-On Validation Checklist

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Task	Validation Command	Expected Result
Replica Set Status	rs.status()	PRIMARY + SECONDARY healthy
Failover	Stop primary	Secondary elected
Backup Test	mongodump + mongorestore	Successful
DR Site Sync	rs.printSlaveReplicationInfo()	Lag < 10 s
Restore Test	mongorestore --dryRun	No errors

🔟3️⃣ Best Practices Summary

✔ Always deploy **odd number of voting nodes** (3, 5, 7)
✔ Keep one **hidden secondary** in a different region
✔ Enable **authentication & TLS** for secure replication
✔ Regularly **test failover & restore drills**
✔ Store backups **offsite or in cloud**
✔ Automate **alerts** for replication lag
✔ Document **DR SOPs** and review quarterly
✔ Ensure **backup encryption + retention policy**

1️⃣4️⃣ Real-World HA/DR Topologies

Environment	Recommended Topology	Notes
Production (On-Prem)	3× Replica Set + Hidden DR Secondary	Local + Remote resilience
Cloud (Atlas)	Multi-Region Cluster with Continuous Backup	Automated
Hybrid (On-Prem + Cloud)	Replica across DCs + Cloud Backup	Best of both worlds

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

One of the *most critical* MongoDB DBA areas: **High Availability (HA)** and **Disaster Recovery (DR)**.
This is where you learn to keep MongoDB online, resilient, and recoverable — even if hardware fails, data centers go down, or someone deletes data by mistake.

MongoDB High Availability & Disaster Recovery (HA/DR)

🎯 **Objective:** Design, implement, and test MongoDB HA & DR strategies using replica sets, backups, and failover simulations to ensure 99.9 %+ uptime and no data loss.

1️⃣ What Is High Availability (HA)?

High Availability ensures **continuous database service** even during:

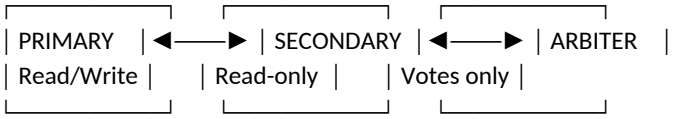
- Server or hardware failure
- Network partition
- Maintenance or upgrades

MongoDB achieves HA through **Replica Sets**.

Concept	Description
Primary Node	Handles reads and writes
Secondary Node(s)	Replicates data from primary (async replication)
Arbiter	Participates in elections but holds no data
Automatic Failover	When the primary fails, a secondary becomes the new primary

2️⃣ Replica Set Architecture

Typical 3-Node Setup



Minimum Recommended:

- 3 voting members (odd number)
- Spread across **different availability zones / servers**

3️⃣ Setting Up a Replica Set

Step 1: Create Data Directories

```
mkdir -p /data/rs0/{node1,node2,node3}
```

Step 2: Start Each mongod

```
mongod --replSet "rs0" --port 27017 --dbpath /data/rs0/node1 --bind_ip localhost,host1 --fork --logpath /data/rs0/node1/mongod.log
mongod --replSet "rs0" --port 27018 --dbpath /data/rs0/node2 --bind_ip localhost,host2 --fork --logpath /data/rs0/node2/mongod.log
mongod --replSet "rs0" --port 27019 --dbpath /data/rs0/node3 --bind_ip localhost,host3 --fork --logpath /data/rs0/node3/mongod.log
```

Step 3: Initialize Replica Set

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "host1:27017" },
    { _id: 1, host: "host2:27018" },
    { _id: 2, host: "host3:27019", arbiterOnly: true }
  ]
})
```

Check status:

```
rs.status()
```

4️⃣ Failover Process (Automatic)

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

When the **Primary** becomes unreachable:

1. Secondary members detect heartbeat timeout (~10 s).
2. Election is triggered.
3. Secondary with highest priority becomes **Primary**.
4. Clients automatically reconnect using **MongoDB connection string**.

Example:
mongodb://host1,host2,host3/?replicaSet=rs0
✔ Drivers handle failover transparently.

🧯 Disaster Recovery (DR) Overview

DR Concept	Purpose
Backups	Protect data from corruption or accidental deletion
Offsite Copy	Store backups in another region or S3
Point-in-Time Recovery	Restore data to specific timestamps
Testing	Regular restore validation
Automation	Cron jobs, Ops Manager, or Atlas Backups

💾 Backup & Restore Strategies

Option A: mongodump / mongorestore (Logical Backup)
mongodump --host host1 --port 27017 --out /backups/$(date +%F)
mongorestore --drop /backups/2025-10-14
✔ Good for small–medium DBs
⚠ No point-in-time recovery

Option B: filesystem Snapshot (Physical Backup)
Stop MongoDB briefly or use **fsyncLock**:
db.fsyncLock()
Then back up the data directory (/var/lib/mongo) using:
tar -czf /backups/mongo-data-$(date +%F).tgz /var/lib/mongo
After backup:
db.fsyncUnlock()
✔ Fast & consistent snapshot
⚠ Requires storage-level tools (EBS, LVM, SAN)

Option C: MongoDB Atlas Backups (Cloud)

- Automated daily snapshots
- Continuous cloud backup (oplog-based)
- Point-in-time restore (PITR)
- Encryption + multi-region redundancy

Restore directly from the Atlas UI.

Option D: Ops Manager Backups (On-Prem Enterprise)
Features:

- Continuous incremental backup
- PITR
- Restore any timestamp or cluster
- Compression & deduplication
- Monitoring integration

🧪 Validating Backups

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Validation Step	Command	Expected
Check dump size	du -sh /backups/latest	Sufficient data size
List collections	mongorestore --dryRun	No errors
Test restore	mongorestore --db testrestore /backups/latest/testdb	Successful import
Compare doc count	db.stats().count	Matches original

8 Cross-Region or DR Site Replication

Option 1: Geo-Distributed Replica Set

Deploy one secondary in another region:

rs.add({ host: "us-east-dr.mongodb.net:27017", priority: 0, hidden: true })

✔ Keeps an up-to-date copy for DR

✔ Hidden node avoids client queries

Option 2: Backup Copy to Cloud Storage

Automate:

aws s3 cp /backups/2025-10-14 s3://mongo-dr-backups/

9 Testing Failover & Recovery

Test 1: Simulate Primary Failure

sudo systemctl stop mongod

Observe new primary election with:

rs.status()

Test 2: Data Recovery Test

1. Drop a test collection.
2. Restore from backup.
3. Verify data integrity.

10 DR Runbook Example (Production SOP)

Step	Action	Tool/Command
1	Confirm failure	rs.status()
2	Identify last healthy node	rs.printSlaveReplicationInfo()
3	Promote secondary manually (if needed)	rs.stepUp()
4	Restore from backup if corruption	mongorestore /backups/...
5	Validate replication resumed	rs.status()
6	Notify stakeholders	Email/Slack Ops Channel
7	Post-mortem documentation	Jira / Confluence

11 Key Metrics to Monitor

Metric	Description	Threshold
Replication Lag	Delay in syncing secondaries	< 10 seconds
Oplog Window	Time range of replication log	> 24 hours
Disk I/O	Write latency	< 10 ms
Node Availability	Node uptime	99.9 %
Backup Size	Backup growth trend	Track weekly

12 Hands-On Validation Checklist

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Task	Validation Command	Expected Result
Replica Set Status	rs.status()	PRIMARY + SECONDARY healthy
Failover	Stop primary	Secondary elected
Backup Test	mongodump + mongorestore	Successful
DR Site Sync	rs.printSlaveReplicationInfo()	Lag < 10 s
Restore Test	mongorestore --dryRun	No errors

🔟3️⃣ Best Practices Summary

✔ Always deploy **odd number of voting nodes** (3, 5, 7)
✔ Keep one **hidden secondary** in a different region
✔ Enable **authentication & TLS** for secure replication
✔ Regularly **test failover & restore drills**
✔ Store backups **offsite or in cloud**
✔ Automate **alerts** for replication lag
✔ Document **DR SOPs** and review quarterly
✔ Ensure **backup encryption + retention policy**

1️⃣4️⃣ Real-World HA/DR Topologies

Environment	Recommended Topology	Notes
Production (On-Prem)	3× Replica Set + Hidden DR Secondary	Local + Remote resilience
Cloud (Atlas)	Multi-Region Cluster with Continuous Backup	Automated
Hybrid (On-Prem + Cloud)	Replica across DCs + Cloud Backup	Best of both worlds

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Security, Authentication, and Role-Based Access Control (RBAC) — the foundation of a **secure MongoDB production environment**. In this module, you'll learn **how to protect MongoDB data from unauthorized access**, **enable authentication**, **manage users and roles**, and **implement best practices** for auditing and encryption.

Module 14: MongoDB Security, Authentication, and RBAC

Objective: Configure secure MongoDB environments with authentication, authorization, encryption, auditing, and access control using built-in security mechanisms.

1 Security Overview

Security Area	Description
Authentication	Verifies who the user or client is
Authorization (RBAC)	Controls what actions they can perform
Network Security	Restricts access to trusted sources
Encryption	Protects data at rest and in transit
Auditing	Tracks administrative and user actions
Backup Security	Ensures backups are protected and encrypted

2 Authentication Modes in MongoDB

Mode	Description
None (Default)	No authentication; anyone can connect
SCRAM-SHA-256 / 1	Default username-password mechanism
x.509	Certificate-based authentication
LDAP / Kerberos	Enterprise centralized authentication

3 Enabling Authentication (SCRAM)

Step 1: Start MongoDB Without Auth (initial setup)

```
mongod --port 27017 --dbpath /var/lib/mongo --bind_ip 0.0.0.0 --fork --logpath /var/log/mongodb/mongod.log
```

Step 2: Create the Admin User

In the shell:

```
use admin
db.createUser({
  user: "siteAdmin",
  pwd: "StrongPassw0rd!",
  roles: [ { role: "userAdminAnyDatabase", db: "admin" }, "readWriteAnyDatabase" ]
})
```

Step 3: Enable Authentication

Edit /etc/mongod.conf:

```
security:
  authorization: enabled
```

Restart MongoDB:

```
sudo systemctl restart mongod
```

Step 4: Connect with Authentication

```
mongo -u "siteAdmin" -p "StrongPassw0rd!" --authenticationDatabase "admin"
```

✔ You now have authentication enabled system-wide.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

4 Role-Based Access Control (RBAC) Overview

RBAC ensures each user can only perform authorized actions.

Role Category	Description
Database User Roles	Read, Write, ReadWrite
Database Admin Roles	dbAdmin, dbOwner
Cluster Roles	clusterAdmin, clusterManager
Backup/Restore Roles	backup, restore
Custom Roles	Created by DBAs for specific needs

5 Common Built-in Roles

Role	Permissions	Scope
read	Read data only	Database
readWrite	Read and modify data	Database
dbAdmin	Manage indexes, stats, profiles	Database
userAdmin	Create/modify users and roles	Database
clusterAdmin	Manage replication, sharding, clusters	Cluster
backup	Perform backups	Cluster
restore	Restore data from backups	Cluster
root	Full privileges (superuser)	Cluster

6 Creating Users and Assigning Roles

Example: Create App User with Limited Access

```
use sales
db.createUser({
  user: "appUser",
  pwd: "AppUser#123",
  roles: [ { role: "readWrite", db: "sales" } ]
})
```

Example: Create Backup User

```
use admin
db.createUser({
  user: "backupUser",
  pwd: "Backup@2025",
  roles: [ { role: "backup", db: "admin" } ]
})
```

Example: Create Custom Role

```
use admin
db.createRole({
  role: "readAnalytics",
  privileges: [
    { resource: { db: "analytics", collection: "" }, actions: [ "find" ] }
  ],
  roles: []
})
```

Assign the custom role:

```
db.grantRolesToUser("analyst1", [ { role: "readAnalytics", db: "admin" } ])
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

7️⃣ Network Access & Bind Configuration

Default Configuration

In /etc/mongod.conf:

```
net:
  bindIp: 127.0.0.1
  port: 27017
```

✔ This restricts access to **localhost only**.

Change to:

```
bindIp: 127.0.0.1,192.168.1.10
```

for internal network access.

Recommended Firewall Rules

Allow	Deny
Trusted internal servers	All external IPs
Application servers	Public internet
Monitoring tools	Unknown hosts

Use firewalld / iptables to restrict:

```
sudo firewall-cmd --permanent --add-source=192.168.1.0/24
sudo firewall-cmd --reload
```

8️⃣ Encryption

a) Encryption in Transit (TLS/SSL)

Generate a self-signed certificate:

```
openssl req -newkey rsa:4096 -new -x509 -days 365 -keyout mongo.key -out mongo.crt
cat mongo.key mongo.crt > mongo.pem
chmod 600 mongo.pem
```

Update /etc/mongod.conf:

```
net:
  ssl:
    mode: requireSSL
    PEMKeyFile: /etc/ssl/mongo.pem
```

Restart MongoDB:

```
sudo systemctl restart mongod
```

Now connect:

```
mongo --ssl --sslCAFile /etc/ssl/mongo.crt --host dbserver
```

b) Encryption at Rest

MongoDB Enterprise & Atlas support **Encrypted Storage Engines**.

In /etc/mongod.conf:

```
security:
  enableEncryption: true
  encryptionKeyFile: /etc/mongodb-keyfile
```

Generate key:

```
openssl rand -base64 32 > /etc/mongodb-keyfile
chmod 600 /etc/mongodb-keyfile
chown mongod:mongod /etc/mongodb-keyfile
```

9️⃣ Auditing in MongoDB (Enterprise)

Enable Auditing:

```
setParameter:
  auditAuthorizationSuccess: false
auditLog:
  destination: file
  path: /var/log/mongodb/audit.log
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

format: JSON

Audits include:

- User logins
- Role assignments
- Schema changes
- Query executions

Example record:

```
{
  "atype": "authenticate",
  "param": { "user": "admin", "db": "admin" },
  "timestamp": { "$date": "2025-10-14T10:35:00Z" },
  "result": 0
}
```

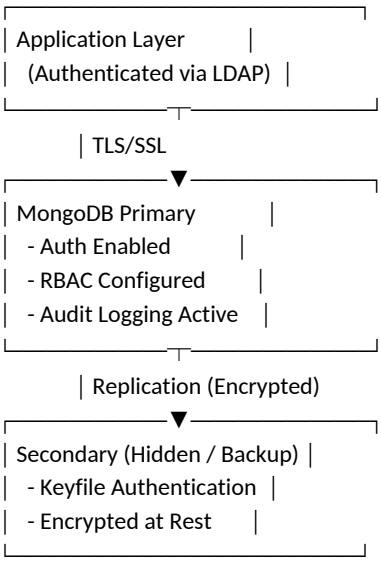
🔟 Security Best Practices

✔ Always enable **authorization**
✔ Restrict **network access** (bind IPs & firewall)
✔ Use **strong passwords / keyfiles**
✔ Encrypt data **in transit & at rest**
✔ Use **custom roles** for principle of least privilege
✔ Regularly **rotate keys and passwords**
✔ Enable **auditing** in production
✔ Restrict **local file access** (no world-readable logs)
✔ Keep **MongoDB patched & updated**

1️⃣1️⃣ Hands-On Validation Checklist

Task	Command	Expected Result
Authentication Enabled	db.isMaster()	Access denied if no login
Role Verification	db.getUser("siteAdmin")	Displays roles
Connection Secure	mongo --ssl	Successful TLS handshake
Encrypted Data	Inspect /var/lib/mongo	Binary, not readable
Audit Logs	tail -f /var/log/mongodb/audit.log	Auth records visible

1️⃣2️⃣ Real-World Production Security Architecture



1️⃣3️⃣ Incident Response Checklist

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Step	Action	Tool/Command
1	Identify unauthorized login	Review audit logs
2	Revoke access	db.revokeRolesFromUser()
3	Rotate passwords	db.updateUser()
4	Check for data anomalies	Application-level validation
5	Restore from backup if needed	mongorestore
6	Document incident	Security report

🔟4️⃣ Summary of Key Learnings

✅ Enable **Authentication & Authorization**
✅ Use **RBAC with least privileges**
✅ Secure MongoDB using **TLS + Encryption at rest**
✅ Enforce **network isolation & firewall rules**
✅ Implement **audit trails** for compliance
✅ Regularly **review users and roles**
✅ Keep **keys, passwords, and backups** safe and rotated

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Crucial one for every MongoDB DBA: **Backup, Restore, and Point-in-Time Recovery (PITR & Snapshot Strategies)**.
In this module, you'll master how to **protect MongoDB data from corruption or loss**, implement **backup automation**, and perform **reliable restores** — both for full databases and specific collections.

MongoDB Backup, Restore, and Point-in-Time Recovery (PITR)

🎯 **Objective:** Learn how to design, configure, and validate MongoDB backup and recovery strategies for standalone, replica set, and sharded deployments — including PITR, snapshots, and automation.

1️⃣ Why Backups Matter

MongoDB's replication ensures availability — **but not recoverability**.
Backups are essential for:

- Data corruption
- Accidental deletions
- Ransomware or human error
- Migration or environment cloning

A **complete backup plan** includes:
✔ Regular automated backups
✔ Offsite copies
✔ PITR (Point-in-Time Recovery)
✔ Validation & test restores

⚙️ 2️⃣ Types of MongoDB Backups

Type	Method	Use Case	Notes
Logical Backup	mongodump / mongorestore	Small to medium DBs	Portable (BSON/JSON)
Physical Backup	File system or LVM snapshots	Large production DBs	Fast restore
Oplog / PITR Backup	Continuous oplog capture	Replica sets	Enables time-based restore
Atlas Backup	Managed snapshots	Cloud clusters	Fully automated
Ops Manager Backup	Enterprise on-prem backup	Large-scale environments	Continuous & incremental

3️⃣ Logical Backups (mongodump / mongorestore)

Backup Example:

```
mongodump --host localhost --port 27017 \
  --out /backups/mongobkp_$(date +%F)
```

Restore Example:

```
mongorestore --drop /backups/mongobkp_2025-10-14
```

Option	Description
--db testdb	Backup specific database
--collection users	Backup specific collection
--gzip	Compress output
--archive	Combine into single file

Example (Compressed Single File Backup):

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

mongodump --db sales --gzip --archive=/backups/sales_$(date +%F).gz
Restore:
mongorestore --gzip --archive=/backups/sales_2025-10-14.gz

4 Physical Backups (File System Snapshots)

Ideal for **large production databases** or **low-latency recovery**.

Steps:

1. **Lock writes** (for consistency):
2. db.fsyncLock()
3. **Take snapshot** (LVM / EBS / SAN):
4. lvcreate --snapshot --name mongosnap /dev/vg0/mongodata
5. **Unlock writes**:
6. db.fsyncUnlock()
7. **Copy snapshot data** to backup location.

✔ *Fast, consistent, block-level copy.*
⚠ *Requires admin access & storage-level control.*

5 Oplog-Based Incremental & PITR Backups

MongoDB's **oplog.rs** collection stores all operations applied to the primary.
You can capture it periodically for **continuous backup**.

Dump oplog:

mongodump --oplog --out /backups/oplog_$(date +%F)

Restore to a Specific Point in Time:

1. Restore base snapshot.
2. Apply oplog replay:
3. mongorestore --oplogReplay /backups/oplog_2025-10-14/oplog.bson

✔ Recovers DB up to exact moment before data loss.

6 Automating Backups with Cron Jobs

Example Script: /scripts/mongo_backup.sh

```
#!/bin/bash
BACKUP_PATH="/backups/$(date +%F)"
mkdir -p $BACKUP_PATH
mongodump --gzip --archive=$BACKUP_PATH/mongobkp.gz
find /backups/* -mtime +7 -exec rm -rf {} \;
```

Schedule with Cron:

```
crontab -e
0 2 * * * /scripts/mongo_backup.sh
```

🕑 Runs daily at 2 AM and retains 7 days of backups.

7 MongoDB Atlas Backups (Cloud)

MongoDB Atlas provides:

- **Daily snapshots**
- **Continuous cloud backup (PITR)**
- **Encrypted backups**
- **Restore to any timestamp**

Steps:

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

1. Go to **Atlas Console → Backup → Settings**.
2. Choose between **Snapshot Backup** or **Continuous Backup**.
3. For PITR:
    - Enable *"Continuous Cloud Backup"*
    - Choose restore timestamp under **"Restore from point in time"**

✔ Fully managed — no scripts, encryption included.

8 Ops Manager Backup (Enterprise On-Prem)

Ops Manager provides:

- Continuous incremental backups
- PITR
- Snapshot compression & deduplication
- Automated restore via UI or API

Workflow:

1. Deploy Backup Daemon + Agents.
2. Schedule snapshots (e.g., hourly).
3. Configure PITR window (e.g., 72 hours).
4. Validate with restore test weekly.

9 Restoring Specific Collections or Databases

```
mongorestore --nsInclude "sales.orders" /backups/mongobkp_2025-10-14/
```

✔ Restore only selected namespace.

Restore to Different Database:

```
mongorestore --nsFrom "sales.*" --nsTo "sales_restore.*" /backups/mongobkp_2025-10-14/
```

10 Validating Backups

Validation Task	Command	Expected Result
List contents	bsondump --help	Validate BSON
Test restore	mongorestore --dryRun	No errors
Compare counts	db.collection.countDocuments()	Match source
Backup integrity	gzip -t file.gz	"OK"
PITR check	Restore before & after time	Data consistent

11 Backup Storage Strategy

Storage Type	Recommended Use	Retention
Local Disk	Quick recovery	7 days
NFS / NAS	Shared backups	14 days
S3 / Azure Blob	Offsite DR	30+ days
Glacier / Deep Archive	Long-term compliance	90+ days

Backup Directory Naming Example:

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

/backups/mongo/
├── full_2025-10-14/
├── incr_2025-10-14T06/
└── incr_2025-10-14T12/

12 Backup Security & Encryption

✔ Use **OS-level permissions**:

chmod 700 /backups
chown mongod:mongod /backups

✔ Encrypt archives:

gpg -c /backups/mongobkp.gz

✔ Store encryption keys securely (Vault / KMS).

13 Recovery Testing Checklist

Scenario	Test Type	Expected Outcome
Full restore	Restore all DBs	DB operational
Single collection	Targeted restore	Partial data restored
PITR restore	Replay oplog	Data consistent
DR restore	From remote site	Successful

 Validate every quarter by restoring a random backup.

14 Example End-to-End PITR Workflow

1. Take a **full base backup** nightly.
2. Continuously capture **oplog.rs** changes.
3. To recover:
   - Restore base dump
   - Apply oplog until target timestamp
   - Verify via record counts and checksums

✔ Example Command:

mongorestore --oplogReplay --oplogLimit "2025-10-14T10:35:00Z" /backups/full_2025-10-14

15 Hands-On Validation Commands

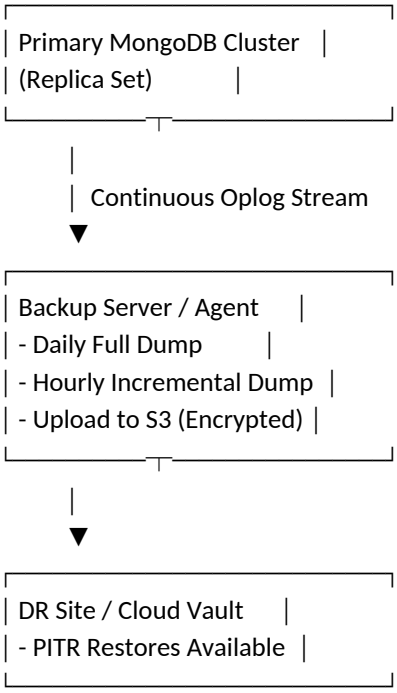
Step	Command	Expected Output
Create backup	mongodump --gzip --archive	Completed without error
Restore test	mongorestore --dryRun	Validation passed
Backup size check	du -sh /backups/*	Non-zero
Oplog capture	mongodump --oplog	BSON file created
PITR validation	Restore + timestamp	Data up-to-date

16 Best Practices Summary

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

✔ Always encrypt backups
✔ Automate with **cron / Ops Manager / Atlas**
✔ Use **naming conventions + timestamps**
✔ Store copies **offsite & in multiple regions**
✔ Test restores regularly
✔ Document your **RTO** (Recovery Time Objective) & **RPO** (Recovery Point Objective)
✔ Use **PITR** for business-critical systems

🏗️ Real-World MongoDB Backup Architecture



Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Performance Tuning, Indexing, and Query Optimization.
This is one of the most critical topics for DBAs and Developers — it directly affects how fast queries run, how efficiently resources are used, and how well your clusters scale under load.

MongoDB Performance Tuning, Indexing, and Query Optimization

🎯 **Objective:** Learn how to analyze, troubleshoot, and optimize MongoDB performance through indexing, query analysis, system resource tuning, and best practices for high-throughput environments.

1️⃣ Understanding MongoDB Performance Bottlenecks

MongoDB performance depends on multiple layers:

Layer	Focus Area	Example
Hardware / OS	CPU, Memory, Disk I/O	IOPS, swap usage
MongoDB Config	Cache size, journaling	WiredTiger cache
Query Design	Query filters, indexes	$regex, $in, $lookup
Schema Design	Document structure	Embedding vs referencing
Concurrency	Locking, write concern	Replication lag

⚙️ 2️⃣ Key Performance Metrics to Monitor

Metric	Description	Command
opcounters	Reads/writes per sec	db.serverStatus().opcounters
connections	Active client connections	db.serverStatus().connections
mem	Memory usage	db.serverStatus().mem
locks	Global lock percentage	db.serverStatus().globalLock
page faults	Disk reads due to cache miss	mongostat
indexMissRatio	% of queries missing index	db.serverStatus().indexCounters
wiredTiger cache	Cache usage ratio	db.serverStatus().wiredTiger.cache

✅ **Goal:** Keep **cache utilization < 80%** and **lock ratio < 10%**.

3️⃣ WiredTiger Engine Tuning

MongoDB (since 3.2+) uses **WiredTiger** by default — a highly concurrent storage engine.

Important Tunables:

Parameter	Description	Recommended
storage.wiredTiger.engineConfig.cacheSizeGB	Cache size	50% of RAM (max 256 GB)
storage.journal.enabled	Journaling for durability	Enabled
storage.syncPeriodSecs	Journal flush interval	60s (default)
net.maxIncomingConnections	Max connections	Tune for workload

Example (/etc/mongod.conf):

```
storage:
  wiredTiger:
    engineConfig:
      cacheSizeGB: 16
```

🔍 4️⃣ Query Performance Analysis

MongoDB provides two key tools:

1. explain() — query plan and index usage
2. profiler — query performance logs

Using explain():

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

db.orders.find({ status: "Delivered" }).explain("executionStats")
Key fields:

- winningPlan → The chosen index
- executionTimeMillis → Query time
- nReturned vs totalDocsExamined → Efficiency ratio

✔ **Goal:** totalDocsExamined ≈ nReturned
⚠ If large difference → missing or inefficient index.

5 MongoDB Profiler (Query Logging)

Enable profiler:
db.setProfilingLevel(2, { slowms: 100 })
View slow queries:
db.system.profile.find().sort({ ts: -1 }).limit(5).pretty()
Disable profiler:
db.setProfilingLevel(0)
✔ Store logs under /var/log/mongodb/mongod.log
✔ Use slowms to define slow query threshold.

6 Indexing Strategies

Common Index Types

Index Type	Example	Use Case
Single field	{ name: 1 }	Basic lookups
Compound	{ status: 1, date: -1 }	Multi-key filters
Multikey	{ tags: 1 }	Array fields
Text index	{ description: "text" }	Full-text search
Hashed	{ user_id: "hashed" }	Sharding
TTL	{ createdAt: 1 }	Auto-expiring data

Create example:

```
db.orders.createIndex({ status: 1, date: -1 })
```

List indexes:

```
db.orders.getIndexes()
```

Drop index:

```
db.orders.dropIndex("status_1_date_-1")
```

7 Compound Index Best Practices

✔ Place **most selective field first**
✔ Use **sort order** in index for queries with sort()
✔ Avoid redundant indexes (MongoDB may still use only one per query)
✔ Always **analyze with explain()**
Example:
db.sales.find({ region: "US" }).sort({ date: -1 }).explain("executionStats")
Matching index: { region: 1, date: -1 }

8 Index Optimization Tips

- Avoid over-indexing (extra indexes slow writes)
- Regularly check unused indexes:
-

```
db.collection.aggregate([
 { $indexStats: {} },
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
  { $match: { accesses: { ops: { $eq: 0 } } } }
])
```

- Monitor index size:
  db.collection.stats().indexSizes
- Rebuild large fragmented indexes periodically:
  db.collection.reIndex()

9 Schema Design for Performance

MongoDB's flexibility can hurt performance if misused.
Choose between **embedding** and **referencing** wisely.

Design	Description	When to Use
Embedding	Nest related data in one document	1:1 or 1:few relationships
Referencing	Use ObjectIds to link collections	1:many or large sub-documents

Example (Embedding):

```
{
  order_id: 101,
  customer: { name: "Noel", city: "NY" },
  items: [ { product: "Laptop", qty: 2 } ]
}
```

✔ Fewer joins (fast)
⚠ Document size limit: **16MB**

10 Query Pattern Tuning

Problem	Solution
Scanning too many docs	Add index
Regex without prefix	Use anchored regex ^abc
$in with large list	Replace with $lookup or bulk query
$lookup too slow	Pre-aggregate with $merge
$sort high latency	Create compound index matching sort

11 Aggregation Pipeline Optimization

Aggregation pipelines ($match, $group, $sort) can be heavy.
Optimize by:
✔ Using $match and $project early
✔ Avoiding $group on large datasets without indexes
✔ Using $merge or $out to persist intermediate results
✔ Checking performance with:

```
db.orders.aggregate([
  { $match: { status: "Delivered" } },
  { $group: { _id: "$region", total: { $sum: "$amount" } } }
]).explain("executionStats")
```

12 Caching and Connection Pooling

- Use **connection pooling** (default in drivers)
- Cache frequently used lookups in app layer (Redis / memory)
- Adjust connection pool size:
- maxPoolSize=100

✔ Fewer connection creations = lower latency.

13 OS-Level Tuning Tips

Area	Setting	Recommendation
Disk I/O	Use SSDs	≥10k IOPS

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Area	Setting	Recommendation
Swap	Disable or minimize	Avoid swapping
File System	XFS preferred	Journaling enabled
NUMA	Disable NUMA for mongod	Use numactl --interleave=all
Transparent Huge Pages	Disable	Avoid memory stalls

🔟4️⃣ Performance Testing Tools

Tool	Purpose	Command Example
mongostat	Real-time stats	mongostat --host localhost
mongotop	Track read/write per collection	mongotop 5
Atlas Performance Advisor	Index suggestions	Built-in
Profiler	Query performance	db.system.profile.find()

1️⃣1️⃣5️⃣ Practical Performance Tuning Checklist

Task	Tool / Command	Expected Result
Identify slow queries	Profiler / explain()	Execution < 100ms
Find unused indexes	$indexStats	Remove unused
Check disk I/O	iostat	Low IOWAIT
Analyze cache	db.serverStatus().wiredTiger.cache	<80% usage
Monitor connections	mongostat	Within limits

🧪1️⃣6️⃣ Hands-On Lab Checklist

Exercise	Validation
1️⃣ Create compound index on orders	db.orders.getIndexes()
2️⃣ Run explain() before/after index	Compare executionTimeMillis
3️⃣ Enable profiler for 100ms	Capture slow queries
4️⃣ Tune WiredTiger cache size	Verify in db.serverStatus()
5️⃣ Disable THP / enable NUMA interleave	Validate with numactl
6️⃣ Benchmark with mongostat, mongotop	Ensure stable performance

1️⃣7️⃣ Best Practices Summary

✔ Use explain() and profiler for every critical query
✔ Create **compound indexes** based on query filters + sort
✔ Avoid over-indexing (write-heavy workloads suffer)
✔ Tune **cache, IOPS, and connection pool size**
✔ Regularly review **slow queries and unused indexes**
✔ Design schema aligned with access patterns
✔ Disable **THP** and **NUMA** for production Linux
✔ Benchmark before and after tuning

1️⃣8️⃣ Example Real-World Optimization Scenario

Problem: Orders query slow (>2s)

```
db.orders.find({ region: "US", status: "Delivered" }).sort({ order_date: -1 })
```

Solution Steps:

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

1. Analyze plan:
2. .explain("executionStats")
3. Create index:
4. db.orders.createIndex({ region: 1, status: 1, order_date: -1 })
5. Re-run explain — verify index used
6. Result: query time ↓ from **2.4s → 70ms**

✔ **Performance improved by 34x**

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Scaling and Sharding Strategy (Horizontal Scalability & Balancing) — one of the most advanced and impactful topics for any MongoDB Database Administrator. This module explains **how MongoDB scales horizontally**, how to design **efficient shard keys**, and how to monitor and balance data across shards for consistent performance and high availability.

MongoDB Scaling and Sharding Strategy (Horizontal Scalability & Balancing)

🎯 **Objective:**

Learn to design, configure, and manage a **sharded cluster** in MongoDB for large-scale, high-throughput workloads — including shard key design, balancing, autoscaling, and best practices for distributed performance.

1️⃣ Why Sharding?

MongoDB supports **horizontal scaling** via **sharding** — distributing data across multiple servers (shards) to handle:

- Huge datasets (terabytes+)
- High write/read throughput
- Geographic distribution (regional data)

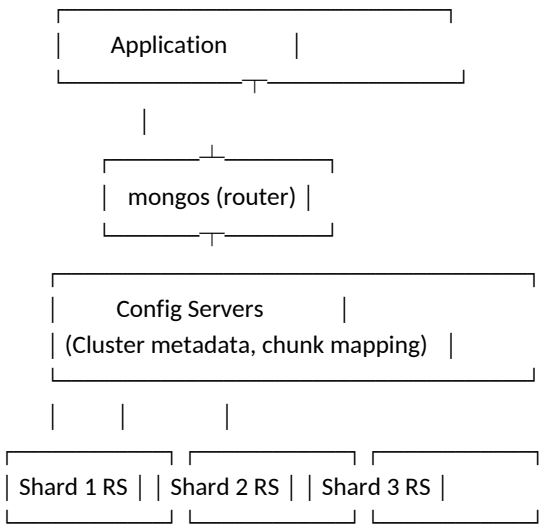
Key benefits

✔ Linear scalability
✔ Load distribution
✔ Fault isolation
✔ Online data growth without downtime

2️⃣ Components of a Sharded Cluster

Component	Description	Example
Shard	A MongoDB replica set that stores subset of data	shard01, shard02, …
Config Server	Stores cluster metadata (chunk mapping)	configReplSet
Query Router (mongos)	Front-end router that directs queries	mongos --configdb configReplSet/…

Architecture Diagram



3️⃣ Sharding Key (The Heart of Sharding)

The **shard key** determines how data is distributed across shards.

Type	Description	Best For
Ranged Sharding	Based on sequential ranges of a key	Time-series, ordered inserts
Hashed Sharding	Based on hashed value of key	Uniform distribution
Zone Sharding	Based on key ranges + zones	Geo / region-based workloads

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Shard Key Best Practices

✔ Choose a **high-cardinality field** (many unique values)

✔ Ensure **even data distribution** (avoid "hot shards")

✔ Use a field that appears in **most queries**

✔ Avoid monotonically increasing keys (e.g., timestamps)

4 Setting Up a Sharded Cluster

Step 1 — Start Config Servers

```
mongod --configsvr --replSet configReplSet --dbpath /data/configdb --port 27019
```

Step 2 — Start Shard Replica Sets

```
mongod --shardsvr --replSet shard01 --dbpath /data/shard01 --port 27018
```

Step 3 — Start mongos Router

```
mongos --configdb configReplSet/localhost:27019 --port 27017
```

Step 4 — Connect and Initialize

```
mongo --port 27017
sh.addShard("shard01/localhost:27018")
sh.addShard("shard02/localhost:27020")
```

Step 5 — Enable Sharding

```
sh.enableSharding("salesDB")
sh.shardCollection("salesDB.orders", { region: "hashed" })
```

✔ MongoDB will now distribute documents automatically across shards.

5 Verifying Sharded Cluster Status

```
sh.status()
```

Sample output:

```
--- Sharding Status ---
 sharding version: { "_id": 1, "version": 5 }
 shards:
   { "_id": "shard01", "host": "shard01/localhost:27018" }
   { "_id": "shard02", "host": "shard02/localhost:27020" }
 databases:
   { "_id": "salesDB", "primary": "shard01", "partitioned": true }
```

6 Chunking and Data Balancing

MongoDB splits data into **chunks** (default 128MB).

Chunks are automatically balanced among shards by the **balancer process**.

View chunks:

```
use config
db.chunks.find({ ns: "salesDB.orders" }).pretty()
```

Enable/Disable balancer:

```
sh.startBalancer()
sh.stopBalancer()
```

✔ Monitor balancing activity:

```
sh.isBalancerRunning()
```

7 Hashed vs Ranged Sharding

Type	Advantages	Disadvantages
Hashed	Uniform distribution	Range queries inefficient
Ranged	Efficient range scans	Possible hot shards
Zone	Geo/location awareness	Complex configuration

Example

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

sh.shardCollection("analytics.logs", { log_id: "hashed" })
→ Evenly spreads data across shards.

⚙️8 Zonal (Tag-Aware) Sharding

You can assign **zones** to shards for region-based control.

Steps

```
sh.addShardTag("shard01", "US-East")
sh.addShardTag("shard02", "EU-West")
sh.addTagRange("salesDB.orders",
        { region: "A" },
        { region: "M" },
        "US-East")
```

✔ MongoDB ensures U.S. data stays in U.S. shard.
Used for **data locality** or **compliance** (GDPR, etc.).

📈9 Scaling Read and Write Operations

- Reads are distributed automatically via mongos.
- Writes go to the shard containing the key range.

Best Practices

✔ Use **hashed shard key** for uniform writes
✔ Place **read-heavy data** on faster shards (SSD)
✔ Avoid scatter-gather queries (target specific shards)
✔ Use **compound shard keys** for multi-attribute lookups
Example:

```
sh.shardCollection("salesDB.transactions", { customer_id: "hashed", region: 1 })
```

🔟 Monitoring & Troubleshooting Sharding

Tool	Purpose	Example
sh.status()	Cluster overview	Check chunk count
db.printShardingStatus()	Shard info	CLI summary
mongostat	Realtime metrics	Throughput per node
mongotop	Collection I/O	Identify hot collections
Atlas Performance Advisor	Shard balancing & slow queries	Automated tuning

🚚11 Rebalancing and Chunk Migration

Chunks may become uneven due to inserts/deletes.

Manual migration:

```
sh.moveChunk("salesDB.orders",
  { region: "TX" },
  "shard02")
```

✔ Verify with:

```
db.chunks.find({ ns: "salesDB.orders", "min.region": "TX" })
```

🔄12 Resharding (MongoDB 5.0+)

MongoDB supports **live resharding** — changing shard keys **without downtime**.

```
reshardCollection: "salesDB.orders",
key: { customer_id: "hashed" }
```

✔ MongoDB creates temporary collection, migrates data, then swaps.
⚠ Requires extra disk and careful monitoring.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🔧 13 Sharded Cluster Maintenance

Task	Command	Frequency
Check shard health	rs.status()	Daily
Monitor chunk imbalance	sh.status()	Weekly
Validate zone mappings	sh.getZones()	Monthly
Backup config servers	mongodump --db config	Regularly

🧪 14 Hands-On Lab Checklist

Exercise	Validation
1️⃣ Deploy 3-node sharded cluster	All mongod/mongos running
2️⃣ Enable sharding on DB	sh.status() shows partitioned
3️⃣ Create hashed shard key	Data balanced across shards
4️⃣ Add new shard dynamically	New shard appears in cluster
5️⃣ Move chunk manually	db.chunks.find() reflects new shard
6️⃣ Perform resharding	Cluster stable post-operation

🧠 15 Best Practices Summary

✔ Always use **replica sets** as shards (not standalone nodes)
✔ Choose shard key carefully — **irreversible decision**
✔ Monitor **chunk imbalance and balancer activity**
✔ Avoid cross-shard transactions unless required
✔ Plan **zones** early for data locality
✔ Automate scaling with scripts or MongoDB Atlas APIs
✔ Regularly test **resharding** and **failover scenarios**

🧩 16 Real-World Sharded Cluster Example

Scenario:
E-commerce database handling 10M+ orders/month.

Component	Description
Shards	3 replica sets (shard01, shard02, shard03)
Shard key	{ customer_id: "hashed" }
Config servers	3-node replica set
Query routers	2 mongos routers (load-balanced)

✔ **Result:**

- Queries distributed evenly
- Writes scale horizontally
- Zero downtime scaling via adding new shards

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Maintenance & Automation (Backups, Updates, Index Optimization, and Routine Tasks)

Objective:

Learn how to automate MongoDB administrative routines — backups, index maintenance, log rotation, version upgrades, and scheduled housekeeping — ensuring a healthy, high-performing, and resilient environment.

1️⃣ Importance of Maintenance & Automation

Proactive maintenance prevents downtime, corruption, and performance degradation.

Key Goals:

- Maintain **data integrity**
- Reduce **manual errors**
- Automate **repetitive admin tasks**
- Ensure **consistent performance**
- Improve **operational visibility**

2️⃣ Core Maintenance Activities

Category	Maintenance Task	Frequency	Tool/Command
Data Integrity	Backup validation, restore test	Weekly	mongodump, mongorestore
Performance	Index rebuild, compact collections	Monthly	db.collection.reIndex(), compact
Logs	Log rotation and cleanup	Daily / Weekly	logRotate, OS cron job
Disk Health	Check free space and IOPS	Weekly	df -h, iostat
Security	Key rotation, TLS validation	Quarterly	openssl, mongo --tls
Upgrade	Minor/patch upgrade	Quarterly	yum / apt / manual
Replication	Check sync delay, heartbeat	Daily	rs.printSlaveReplicationInfo()

⚙️3️⃣ Automating Routine Tasks Using CRON

You can use **Linux cron jobs** to automate maintenance tasks.

🧩 Example: Automated Backup Script

File: /scripts/mongo_backup.sh

```
#!/bin/bash
DATE=$(date +%F_%H-%M)
BACKUP_DIR="/backups/mongodb/$DATE"
mkdir -p $BACKUP_DIR

mongodump --host localhost --port 27017 --out $BACKUP_DIR

# Keep only last 7 days of backups
find /backups/mongodb/ -type d -mtime +7 -exec rm -rf {} \;

echo "Backup completed at $DATE" >> /var/log/mongo_backup.log
```

Add cron job:

```
crontab -e
0 2 * * * /scripts/mongo_backup.sh
```

🕑 → Runs daily at 2 AM.

🧩 Example: Log Rotation via CRON

mongosh --eval "db.adminCommand({ logRotate: 1 })"

Add to cron:

0 0 * * 0 /usr/bin/mongosh --eval "db.adminCommand({ logRotate: 1 })"

🕛 → Weekly rotation every Sunday at midnight.

4️⃣ Automating Index Maintenance

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Indexes can bloat or become fragmented due to frequent writes/deletes.

Step 1 — Identify Large/Unused Indexes

```
db.collection.stats({ indexDetails: true })
db.collection.getIndexes()
```

Step 2 — Rebuild Indexes

```
db.collection.reIndex()
```

Step 3 — Drop Unused Indexes (Carefully)

```
db.collection.dropIndex("index_name")
```

You can automate stats collection via a script that emails metrics weekly.

5 Automating Alerts & Monitoring

Combine MongoDB metrics with **Prometheus + Grafana** or **MongoDB Cloud Manager / Ops Manager** for alerting.

Automate alert triggers for:

- Oplog replication lag
- Disk space < 15%
- Connections > threshold
- Memory usage > 85%
- Backup failures

6 Automating Version Upgrades (Minor/Patch)

Sample script:

```
#!/bin/bash
# Stop MongoDB
systemctl stop mongod

# Backup current binaries
cp -r /usr/bin/mongo* /usr/bin/mongo_backup_$(date +%F)

# Update MongoDB
yum update -y mongodb-org

# Start MongoDB
systemctl start mongod

# Validate version
mongosh --eval "db.version()"
```

Add version validation logs and email summary to DBA team.

7 Compacting Collections

Reclaims unused space and optimizes storage.

```
db.runCommand({ compact: "collection_name" })
```

⚠ Requires exclusive access — do it on **secondary nodes** first.

8 Disk & Memory Health Automation

Monitor filesystem usage:

```
df -h /var/lib/mongo
```

Monitor IO performance:

```
iostat -x 1 5
```

Use cron to alert if free space < 20%:

```
df -h | awk '$5 > 80 {print $0}' | mail -s "MongoDB Disk Alert" dba_team@example.com
```

9 Maintenance Validation Checklist

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Validation Task	Command	Expected Result
Backup runs successfully	ls /backups/mongodb	Latest timestamped folder
Restore test	mongorestore	Successful
Index rebuilds	db.collection.reIndex()	"OK"
Logs rotated	/var/log/mongodb/mongod.log	New file created
Disk space check	df -h	< 80% usage
Monitoring alerts	Check Prometheus/Grafana	No critical alerts
Replica sync delay	rs.printSlaveReplicationInfo()	< 5 sec delay

🔟 Best Practices for MongoDB Maintenance

✔ Always test automation scripts in a **non-prod environment first**
✔ Keep **odd-number replica sets** for HA
✔ Schedule **backups during low I/O periods**
✔ Maintain **at least one hidden backup node**
✔ Monitor **backup and restore logs** regularly
✔ Document all scheduled jobs and owners
✔ Enable **email or Slack alerts** for job failures

🧰 1️⃣1️⃣ Recommended Tools

Tool	Purpose
crontab / systemd timers	Schedule tasks
Prometheus + Grafana	Monitoring and alerts
MongoDB Ops Manager	Automated maintenance and deployment
Ansible / Puppet / Chef	Config management & rollout
Percona PMM	MongoDB performance monitoring
Bash + mailx / sendmail	Notification scripting

🚀 Summary

With these maintenance and automation routines:

- You ensure **consistent performance**
- Reduce **human error**
- Guarantee **recoverability**
- Maintain **operational transparency**
- Achieve **enterprise-grade MongoDB uptime**

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

☁ MongoDB Integration with Cloud Services (AWS, Azure, GCP)

Objective:

Learn how to deploy, integrate, and manage MongoDB across major cloud platforms — **AWS**, **Microsoft Azure**, and **Google Cloud Platform (GCP)** — including connectivity, backups, scaling, and cost optimization.

1️⃣ Cloud Deployment Models for MongoDB

Deployment Type	Description	Examples
Self-managed	You install MongoDB on cloud VMs (EC2, Azure VM, GCE).	mongod on EC2
Managed Service	Cloud vendor or MongoDB Inc manages setup, scaling, patching.	**MongoDB Atlas**, Azure Cosmos DB (Mongo API)
Hybrid	Mix of on-prem + cloud for DR, backup, or analytics workloads.	On-prem primary, cloud secondary

⚙ 2️⃣ MongoDB on AWS

Option 1 — Self-Managed on EC2

Architecture:

```
EC2 (Primary) ---- EC2 (Secondary) ---- EC2 (Arbiter)
     |                 |
  EBS Volumes       EBS Volumes
```

Setup Steps:

1. Launch 3 EC2 instances (Amazon Linux 2 / Ubuntu 22.04)
2. Install MongoDB:
3. sudo yum install -y mongodb-org
4. Configure /etc/mongod.conf with replSetName: rs0
5. Attach **EBS volumes** for /data/db
6. Use **Elastic IPs** or **Private DNS** for replica members
7. Enable **security group rules:**
    - TCP 27017 between members
    - SSH (22) for admin access only

Backup Options:

- **EBS Snapshots** (block-level)
- mongodump to **S3**
- mongodump --archive | aws s3 cp - s3://my-mongo-backups/dump.archive
- **Automated Lambda Backup Trigger**

Option 2 — Managed via MongoDB Atlas on AWS

Features:

- Auto-scaling clusters
- Point-in-time restore
- Multi-region replication
- IAM & VPC Peering integration

Integration Steps:

1. Sign in to MongoDB Atlas
2. Choose **AWS** as cloud provider
3. Create project → Build Cluster
4. Configure **region**, **instance size**, **replica count**
5. Set **IP Access list**
6. Connect via:
7. mongosh "mongodb+srv://cluster0.mongodb.net" --username admin

Optional Integrations:

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

- AWS CloudWatch for performance metrics
- AWS Lambda for data-triggered functions
- S3 for archival storage

3 MongoDB on Microsoft Azure

Option 1 — Self-Managed MongoDB on Azure VMs

1. Create **3 Ubuntu VMs** in same region (Availability Set recommended)
2. Install MongoDB
3. Configure **replica set** or **sharded cluster**
4. Use **Azure Managed Disks** for /data/db
5. Enable **Azure Backup** for snapshots
6. Open ports in **NSG**:
    - Allow 27017 internally
    - Restrict external IPs

Backup Options:

- Azure Blob Storage:
- mongodump --archive | az storage blob upload --container mongo-backup --file dump.archive
- Azure Automation for scheduled scripts

Option 2 — Azure Cosmos DB (MongoDB API)

If you prefer a **fully managed, Mongo-compatible** service:

- No installation required
- Supports MongoDB connection strings
- Auto scaling and global replication
- Built-in **backup**, **encryption**, and **RBAC**

Connection Example:

```
mongosh "mongodb://<cosmos_account>.mongo.cosmos.azure.com:10255/?ssl=true" \
  --username myuser --password "MyPassword!"
```

4 MongoDB on Google Cloud (GCP)

Option 1 — Self-managed on Compute Engine

1. Launch 3 VMs (Ubuntu)
2. Configure **firewall rules**:
    - Allow internal 27017
3. Attach **Persistent Disks**
4. Install MongoDB and configure **replica set**
5. Use **Cloud Storage (GCS)** for backups:
6. mongodump --archive | gsutil cp - gs://mongo-backups/dump.archive

Monitoring:

- Stackdriver (Cloud Monitoring) integration
- Log-based alerts for slow queries, replication lag

Option 2 — MongoDB Atlas on GCP

Identical to AWS version, except:

- Choose **GCP region**
- Integrate with **Cloud Logging & IAM**
- Use **Private Service Connect (PSC)** for secure access

5 Cloud Security Integration

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Cloud	Security Options
AWS	IAM roles, KMS encryption, VPC Peering
Azure	Managed Identity, Key Vault, Private Link
GCP	IAM roles, CMEK encryption, VPC Service Controls

MongoDB Security Best Practices (all clouds):

- Enforce **TLS/SSL**
- Enable **SCRAM-SHA-256 authentication**
- Restrict inbound IPs
- Use **encrypted storage**
- Rotate keys regularly

6 Cloud Backup & DR Strategy

Cloud	Backup Method	Description
AWS	EBS Snapshots, S3 + Lambda	Fast restore and retention
Azure	Blob backups, Automation runbooks	Native scheduling
GCP	GCS snapshots, Cloud Scheduler	Integrated monitoring
All (Atlas)	Continuous cloud backup + PITR	Built-in management console

7 Cost Optimization Strategies

✔ Use **spot/preemptible instances** for non-prod clusters
✔ Enable **auto-scaling** in Atlas
✔ Compress backups before upload (gzip)
✔ Use **tiered storage** (S3/Blob/GCS Nearline)
✔ Monitor with **AWS Cost Explorer / Azure Cost Management / GCP Billing**

8 Hands-On Validation Checklist

Validation Step	Command/Action	Expected Result
Connect to MongoDB on cloud VM	mongosh --host <IP>	Connection successful
Backup to S3/Blob/GCS	mongodump + upload	File visible in storage
Restore backup	mongorestore	Data restored
Replication test	rs.status()	All members healthy
Atlas connection test	mongosh "mongodb+srv://..."	Authenticated successfully
Failover test	Stop primary node	Secondary promoted

9 Tools for Cloud MongoDB Administration

Tool	Purpose
MongoDB Atlas	Managed cloud clusters
Terraform / Ansible	Automated provisioning
AWS CloudWatch / Azure Monitor / GCP Operations	Monitoring
Ops Manager / Cloud Manager	Backup + automation
mongodump / mongorestore	Data backup/restore
Vault / Key Management Services	Secrets management

10 Summary

After completing this module, you'll be able to:

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

- Deploy MongoDB in **AWS, Azure, and GCP**
- Integrate **backup, security, and monitoring**
- Use **managed solutions (Atlas, Cosmos DB)** effectively
- Implement **DR and scaling** in cloud-native ways
- Optimize **cost and performance**

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Troubleshooting & Common DBA Issues (Diagnostics, Repairs, and Recovery Scenarios)

Objective: Learn to diagnose, troubleshoot, and resolve performance, replication, and storage issues in MongoDB production systems — using both MongoDB utilities and Linux-level tools.

1 Understanding MongoDB Troubleshooting Framework

When troubleshooting:

1. **Identify the symptom** (slow queries, replication lag, crash)
2. **Collect evidence** (logs, metrics, server stats)
3. **Analyze root cause**
4. **Apply fix**
5. **Validate and document**

2 Key MongoDB Diagnostic Tools

Tool	Command	Use
mongostat	mongostat --host <server>	Real-time stats (ops/sec, memory, connections)
mongotop	mongotop 2	Shows read/write activity by collection
serverStatus()	db.serverStatus()	General health metrics
currentOp()	db.currentOp()	Active operations
explain()	db.collection.find().explain("executionStats")	Query plan details
log files	/var/log/mongodb/mongod.log	Core troubleshooting info
dbStats()	db.stats()	Database-level usage
collStats()	db.collection.stats()	Collection performance & index info

3 Common Problem Categories

Issue Category	Examples
Performance	Slow queries, high CPU, memory pressure
Replication	Oplog lag, sync failures
Storage	Disk full, journal corruption
Connections	Too many connections, connection timeout
Authentication	User/role issues, keyFile mismatch
Startup/Crash	Missing data files, config corruption

4 Performance Troubleshooting

A. Identify Slow Queries

```
db.system.profile.find().sort({ millis: -1 }).limit(5)
```

or enable profiling:

```
db.setProfilingLevel(2)
```

B. Use explain() to See Query Plan

```
db.orders.find({ customer_id: 101 }).explain("executionStats")
```

Look for:

- COLLSCAN (collection scan → missing index)
- IXSCAN (index scan → optimal)
- High nReturned vs nExamined

C. Fix

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

- Create index:
- db.orders.createIndex({ customer_id: 1 })
- Use projections to limit fields returned
- Optimize schema (avoid large embedded docs)

D. Check Server Resource Usage

top
free -m
iostat -x 2 5

E. MongoDB-Level Metrics

db.serverStatus().metrics.operation.scanAndOrder
db.serverStatus().metrics.queryExecutor.scannedObjects

5 Replication Troubleshooting

A. Check Replication Status

rs.status()
rs.printSlaveReplicationInfo()

Symptoms:

- "SECONDARY lagging behind PRIMARY"
- "Replication Oplog too small"
- "Initial sync failed"

Fixes:

- Increase Oplog size:
- mongod --oplogSize 4096
- Resync secondary:
- rs.syncFrom("mongo1:27017")
- Verify network latency between nodes:
- ping mongo2

B. Node Out of Sync

db.printReplicationInfo()
If too far behind, resync by:
stop mongod
rm -rf /data/db/*
start mongod
Then automatically performs **initial sync**.

6 Storage & Disk Troubleshooting

A. Disk Full / WiredTiger Cache Full

Check storage usage:
df -h
du -sh /var/lib/mongo
Reduce WiredTiger cache if memory pressure is high:

```
storage:
  wiredTiger:
    engineConfig:
      cacheSizeGB: 2
```

Restart service:
systemctl restart mongod

B. Corrupt Collection or Journal

If mongod fails to start:

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

mongod --repair --dbpath /var/lib/mongo
⚠ Always back up data directory before running repair.

7️⃣ Connection Issues

A. Too Many Connections

db.serverStatus().connections
If "current" > "available", increase limit:
ulimit -n 65535
Or tune:
net:
maxIncomingConnections: 20000

B. Authentication Errors

- Verify users:
- db.getUsers()
- Check keyFile permissions (replica set):
- ls -l /etc/mongo-keyfile
- chmod 600 /etc/mongo-keyfile

8️⃣ Startup / Crash Recovery

A. Logs Indicate Port or PID Conflict

ps aux | grep mongod
sudo kill -9 <pid>
sudo systemctl start mongod

B. Missing Data Files

Restore from latest backup:
mongorestore --archive=/backups/dump.archive

C. Incomplete Oplog or WiredTiger Lock File

Delete mongod.lock and start again:
rm /var/lib/mongo/mongod.lock
mongod --repair

9️⃣ Advanced Diagnostic Commands

Command	Purpose
db.currentOp()	Identify long-running operations
db.killOp(<opid>)	Terminate stuck query
db.serverStatus().asserts	View internal MongoDB errors
db.getProfilingStatus()	View current profiling config
rs.printReplicationInfo()	Monitor replication state
db.stats()	Check collection count, index sizes

☐ 🔟 Hands-On Validation Checklist

Validation Step	Command	Expected Result
Check replication health	rs.status()	All nodes PRIMARY/SECONDARY
Detect slow queries	db.system.profile.find()	Top 5 slow ops
Fix query with index	db.collection.createIndex()	Reduced scan time
Repair startup issue	mongod --repair	Service starts cleanly
Restore data	mongorestore	DB restored successfully
Verify connections	db.serverStatus().connections	Below threshold

1️⃣1️⃣ Tools for Troubleshooting

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Tool	Purpose
mongostat / mongotop	Performance metrics
Atlas Performance Advisor	Query & index recommendations
Ops Manager / Cloud Manager	Deep performance insights
Perf / iostat / vmstat	OS-level performance
Wireshark / netstat	Network debugging
Prometheus + Grafana	Visual monitoring dashboards

🧠 Troubleshooting Best Practices

✔ Always analyze logs before restarting
✔ Document the *root cause* for each issue
✔ Enable **profiling** only temporarily
✔ Maintain **separate backups** for repair attempts
✔ Use **replica sets** to minimize downtime
✔ Periodically **test restore and failover**
✔ Review **alerts and logs daily**

🦅 Summary

After this module, you'll be able to:

- Diagnose and resolve performance bottlenecks
- Handle replication and disk corruption issues
- Manage connection limits and authentication failures
- Recover MongoDB from crashes safely
- Apply systematic troubleshooting under pressure

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Backup, Restore & Disaster Simulation (End-to-End Recovery Testing)

Objective: Learn complete strategies for **backing up, restoring, and simulating disaster recovery** in MongoDB — using both native utilities and enterprise-grade methods. This module ensures you can **recover any MongoDB deployment** quickly and confidently.

1️⃣ Why Backup & DR Validation Matter

Data loss or corruption can occur due to:

- Accidental deletions
- Hardware/OS failures
- Disk corruption
- Application bugs
- Ransomware or misconfiguration

Hence, every DBA must ensure:

✔ **Regular backups**
✔ **Offsite retention**
✔ **Restore validation**
✔ **Failover testing**

⚙️ 2️⃣ MongoDB Backup Types

Backup Type	Description	Tools / Command	Use Case
Logical Backup	Exports BSON/JSON of collections	mongodump, mongorestore	Portable, smaller datasets
Physical Backup	Backs up actual data files	FS Snapshot, LVM, EBS, Atlas Backup	Fast restore for large DBs
Oplog Backup	Continuously captures changes	Oplog tailing / Ops Manager	Point-in-time recovery
Cloud Backup	Managed, automated backup in Atlas or Ops Manager	Atlas Cloud Backups	Enterprise clusters

📦 3️⃣ Logical Backup (Using mongodump)

Syntax:

```
mongodump --host localhost --port 27017 --out /backups/mongo_backup_$(date +%F)
```

Backup single database:

```
mongodump --db testdb --out /backups/testdb_$(date +%F)
```

Backup specific collection:

```
mongodump --db testdb --collection users --out /backups/testdb_users
```

Compress the backup:

```
tar -czvf mongo_backup_$(date +%F).tar.gz /backups/mongo_backup_$(date +%F)
```

🔄 4️⃣ Restore Data (Using mongorestore)

Syntax:

```
mongorestore /backups/mongo_backup_2025-10-14
```

Restore to another DB name:

```
mongorestore --nsInclude testdb.users --nsFrom testdb.users --nsTo newdb.users
```

Restore from archive:

```
mongorestore --archive=dump.archive --gzip
```

✔ **Tip:** Always drop target collections before restore if you want a clean load:

```
mongorestore --drop --db testdb /backups/testdb
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

5️⃣ Physical Backup (File System Snapshot Method)

Used for **large databases** or **replica sets**.

Steps:

1️⃣ Stop writes (or use a **secondary node**)

2️⃣ Lock the database:

```
db.fsyncLock()
```

3️⃣ Copy data files:

```
cp -r /var/lib/mongo /mnt/backup/mongo_data_$(date +%F)
```

4️⃣ Unlock after snapshot:

```
db.fsyncUnlock()
```

✔ For **replica sets**, always take snapshots from **SECONDARY nodes**.

6️⃣ Cloud Backups (MongoDB Atlas)

Atlas offers **automated continuous backups** with **Point-in-Time Restore (PITR)**.

Features:

- Hourly snapshots
- PITR up to 5–30 days
- Cross-region restore
- UI and API-based restores

Restore Example:

1. Go to *Atlas → Backup → Snapshots*
2. Select desired snapshot
3. Choose “Restore” or “Download dump”

For scripting:

atlas backups snapshots list --projectId <PID> --clusterName <CLUSTER>

7️⃣ Automating Backups

Example: Daily Cron Job

```
#!/bin/bash
DATE=$(date +%F_%H-%M)
OUT_DIR="/backups/mongo/$DATE"
mkdir -p $OUT_DIR
mongodump --host localhost --out $OUT_DIR
find /backups/mongo/ -type d -mtime +7 -exec rm -rf {} \;
```

Add to crontab:

```
0 2 * * * /scripts/mongo_backup.sh
```

🕑 → Runs daily at 2 AM, keeps 7 days’ history.

8️⃣ Point-in-Time Recovery (Oplog Restore)

Used for restoring **to a specific timestamp** after a backup.

Step 1 — Take full backup:

```
mongodump --oplog --out /backups/full_$(date +%F)
```

Step 2 — Restore with oplog:

```
mongorestore --oplogReplay /backups/full_2025-10-14
```

You can restore **to an exact timestamp** using:

```
bsondump oplog.bson | grep "ts" | less
```

Then replay oplog entries up to that time.

9️⃣ Backup Validation (Restore Testing)

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Every backup is **useless unless tested**.

Validation Steps:

1. Restore backup to test environment:
   mongorestore --db restore_test /backups/latest
2. Verify document count:
   db.users.countDocuments()
3. Validate indexes:
   db.collection.getIndexes()
4. Check referential integrity (if using embedded docs)
5. Run smoke tests on restored DB

🔟 Disaster Recovery Simulation (DR Drill)

Goal: Test how quickly and completely you can recover from a total failure.

Scenario 1 — Data Center Failure

1. Shutdown all primary nodes
2. Promote secondary (automatic failover)
3. Measure RTO (Recovery Time Objective)

Scenario 2 — Data Loss

1. Drop a collection:
   db.orders.drop()
2. Restore from last valid backup:
   mongorestore --db sales /backups/sales_backup_2025-10-14
3. Measure RPO (Recovery Point Objective)

Scenario 3 — Corrupted Primary Node

1. Stop damaged node
2. Initialize new node and add back to replica set:
   rs.add("mongo-new:27017")
3. Validate replication sync

1️⃣1️⃣ DR Validation Checklist

Validation Step	Command	Expected Result
Verify backup integrity	mongorestore --dryRun	No corruption
Restore to test DB	mongorestore --db restore_test	Data matches
Verify index recovery	db.collection.getIndexes()	All indexes restored
Measure RTO	Timer	< 30 minutes
Measure RPO	Compare timestamps	Data loss < 5 min
Oplog replay success	mongorestore --oplogReplay	Data consistent
Cloud restore test	Atlas UI/API	Snapshot restored

1️⃣2️⃣ Best Practices

✔ Backup **from secondary** to reduce load
✔ Keep at least **3 copies** (local, remote, cloud)
✔ Test restore **monthly**
✔ Encrypt backups (use gpg or S3 SSE)
✔ Automate cleanup and retention
✔ Document **restore steps** and timing
✔ Store backup metadata (size, checksum, time)

1️⃣3️⃣ Tools for Backup & Recovery

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Tool	Function
mongodump / mongorestore	Logical backups
LVM / EBS Snapshots	Physical volume snapshots
MongoDB Ops Manager / Atlas	Continuous backups, PITR
Percona Backup for MongoDB (PBM)	Advanced open-source backup automation
cron / Ansible	Scheduling and orchestration
AWS S3 / Azure Blob / GCS	Offsite storage

Summary

By the end of this module, you can:

- Implement **end-to-end backup and restore** procedures
- Perform **disaster simulations** confidently
- Validate **RTO/RPO objectives**
- Automate **backup retention** and reporting
- Ensure **data recoverability** across environments

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Indexing Deep Dive (Types, Optimization & Query Planner Behavior)

Objective:

Master MongoDB's indexing mechanisms — how indexes work internally, types of indexes, how to analyze query execution plans, and how to optimize queries for real-world performance. By the end, you'll know how to **tune any slow query** using the right index strategy.

1 Why Indexes Matter

Indexes in MongoDB are like **roadmaps** for your queries — without them, MongoDB must scan every document (collection scan).

- **Without index:** COLLSCAN → full collection scan
- **With index:** IXSCAN → only relevant documents

Indexes dramatically reduce query latency, improve throughput, and optimize resource usage.

2 How Indexes Work Internally

- MongoDB uses **B-Tree** data structures for most indexes.
- Each key in the index points to the **physical location** of documents in the collection.
- Queries use these trees to locate results efficiently.
- Indexes are updated automatically on insert, update, or delete.

3 Index Types in MongoDB

Index Type	Description	Example
Single Field Index	Basic index on one field	db.users.createIndex({name: 1})
Compound Index	Combines multiple fields	db.orders.createIndex({cust_id: 1, order_date: -1})
Multikey Index	For array fields	db.products.createIndex({tags: 1})
Text Index	For full-text search	db.blog.createIndex({content: "text"})
Hashed Index	For sharding hash keys	db.logs.createIndex({user_id: "hashed"})
Wildcard Index	Indexes dynamic / unknown fields	db.inventory.createIndex({"$**": 1})
TTL Index	Auto-delete documents after expiry	db.sessions.createIndex({createdAt: 1}, {expireAfterSeconds: 3600})
Partial Index	Index subset of documents	db.orders.createIndex({status: 1}, {partialFilterExpression: {status: "PENDING"}})

4 Create & Manage Indexes

Create a new index:

```
db.collection.createIndex({field: 1})
```

List all indexes:

```
db.collection.getIndexes()
```

Drop an index:

```
db.collection.dropIndex("field_1")
```

Drop all indexes:

```
db.collection.dropIndexes()
```

5 Understanding Index Direction

- 1 = ascending
- -1 = descending

For **single-field indexes**, direction doesn't matter much.

For **compound indexes**, order and direction matter for query optimization.

Example:

```
db.sales.createIndex({region: 1, sales_amount: -1})
```

✔ Efficient for queries like:

```
db.sales.find({region: "EU"}).sort({sales_amount: -1})
```

6 Compound Index Rules

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Rule 1 — Prefix Principle

MongoDB can use the **leftmost prefix** of a compound index.

Example:

```
db.orders.createIndex({cust_id: 1, order_date: -1})
```

Queries that use:

- {cust_id: X}
- {cust_id: X, order_date: Y}

will use the index ✔

But {order_date: Y} alone ✖ will not.

7 Multikey Indexes (Arrays)

If your documents contain arrays, MongoDB creates one index entry for **each array element**.

Example:

```
{ _id: 1, tags: ["db", "mongo", "nosql"] }
db.articles.createIndex({tags: 1})
```

Query:

```
db.articles.find({tags: "mongo"})
```

✔ Uses index efficiently.

⚠ **Note:** You cannot create compound multikey indexes with more than one array field.

8 Text Indexes

Used for **searching words or phrases**.

```
db.blog.createIndex({title: "text", content: "text"})
```

Search example:

```
db.blog.find({$text: {$search: "mongodb performance"}})
```

Score-based sorting:

```
db.blog.find(
 {$text: {$search: "replication"}},
 {score: {$meta: "textScore"}}
).sort({score: {$meta: "textScore"}})
```

9 TTL Indexes (Time to Live)

Automatically deletes documents after a set duration.

Example:

```
db.sessions.createIndex({createdAt: 1}, {expireAfterSeconds: 3600})
```

🕒 → Deletes documents 1 hour after createdAt.

Used in:

- Logs
- Temporary sessions
- Cache collections

10 Hashed Indexes

Used mainly for **sharded clusters** (hashed shard keys).

```
db.users.createIndex({user_id: "hashed"})
```

Ensures **even distribution** across shards.

11 Query Optimization & Index Selection

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Analyze query plan:

```
db.orders.find({cust_id: 123}).explain("executionStats")
```

Key metrics:

Metric	Meaning
winningPlan.stage	How the query is executed (IXSCAN, COLLSCAN, etc.)
nReturned	Number of docs returned
totalKeysExamined	Keys scanned in index
totalDocsExamined	Docs fetched
executionTimeMillis	Total execution time

✔ **Goal:**
totalDocsExamined ≈ nReturned
That means the index is selective and efficient.

12 Covered Queries

If all queried fields are in the index, MongoDB does not need to fetch documents.
Example:

```
db.users.createIndex({name: 1, age: 1})
db.users.find({name: "John"}, {name: 1, age: 1, _id: 0})
```

✔ Query is **covered** → faster.

13 Index Hints

You can manually force MongoDB to use a specific index:

```
db.orders.find({cust_id: 1}).hint({cust_id: 1})
```

Useful for testing or tuning query plans.

14 Performance Validation Steps

Test	Command	Expected Result
Check if index used	.explain("executionStats")	IXSCAN in plan
Compare query time	executionTimeMillis	< 10ms ideal
Validate selectivity	Compare nReturned vs docsExamined	Close match
Validate index coverage	Use projection	No COLLSCAN
Validate compound order	.sort() queries	Use matching index order

15 Index Maintenance

Task	Frequency	Command
Review slow queries	Weekly	db.system.profile
Drop unused indexes	Monthly	db.collection.dropIndex()
Rebuild indexes	As needed	db.collection.reIndex()
Monitor index size	Ongoing	db.collection.stats()
Archive old data	Periodic	Move to cold storage

16 Advanced: Index Intersection

MongoDB can combine multiple single-field indexes when no compound index exists.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Example:

```
db.users.createIndex({age: 1})
db.users.createIndex({city: 1})
db.users.find({age: 25, city: "NY"})
```

✔ Uses **index intersection** of both.

⚠ Less efficient than compound indexes.

🧪 Hands-On Lab Exercises

Exercise	Steps	Validation
Create single-field index	createIndex({name: 1})	Query uses IXSCAN
Create compound index	createIndex({cust_id: 1, order_date: -1})	Query optimized
Test text search	createIndex({content: "text"})	$text returns ranked docs
Create TTL index	createIndex({createdAt: 1}, {expireAfterSeconds: 60})	Doc auto-deletes
Explain plan test	find().explain("executionStats")	Shows index usage
Covered query	Include only indexed fields	No COLLSCAN

🕊️ Summary

By the end of this module, you can:

✔ Identify and create optimal index types

✔ Understand query planner decisions

✔ Interpret execution plans correctly

✔ Use TTL, text, hashed, and partial indexes

✔ Validate and maintain index health

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Query Optimization & Performance Tuning (Profiler, Query Plans & Real-World Scenarios)

Objective: Gain mastery over diagnosing and tuning **slow queries** in MongoDB using real-time tools such as **explain plans, profiling, and index optimization**. You'll learn to identify bottlenecks, interpret query stages, and apply proven performance-tuning strategies.

1 Why Query Optimization Matters

Performance in MongoDB depends on:

- Proper **indexing**
- Efficient **query structure**
- Balanced **schema design**
- Optimal **hardware and configuration**

80% of performance issues come from **poorly indexed queries** or **inefficient filters**.

2 MongoDB Performance Workflow

1 Identify slow queries
2 Analyze query plan (.explain("executionStats"))
3 Optimize query and/or schema
4 Add or adjust indexes
5 Validate with profiler and execution stats

3 Enable the Profiler (Built-in Query Tracer)

The **database profiler** collects information about query execution time, scans, and resource usage.

Enable profiler globally:

```
db.setProfilingLevel(2)
```

- Level 0 — Off
- Level 1 — Slow operations only
- Level 2 — All operations

Enable profiler for slow queries (>100ms):

```
db.setProfilingLevel(1, {slowms: 100})
```

View profiling status:

```
db.getProfilingStatus()
```

4 View Profiler Data

Profiler data is stored in:

```
db.system.profile.find().sort({ts: -1}).limit(5).pretty()
```

Key fields:

Field	Description
op	Type of operation (query, insert, update)
ns	Namespace (database.collection)
keysExamined	Index keys scanned
docsExamined	Docs scanned
millis	Execution time
planSummary	Plan type (COLLSCAN, IXSCAN)
ts	Timestamp

5 Identify Slow Queries

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Find all queries slower than 200ms:

```
db.system.profile.find({millis: {$gt: 200}}).sort({millis: -1})
```

Group by collection:

```
db.system.profile.aggregate([
 {$match: {millis: {$gt: 200}}},
 {$group: {_id: "$ns", avgTime: {$avg: "$millis"}, count: {$sum: 1}}},
 {$sort: {avgTime: -1}}
])
```

6 Use explain() to Analyze Queries

Example:

```
db.orders.find({status: "PAID"}).explain("executionStats")
```

Key stages:

Stage	Meaning
COLLSCAN	Full collection scan
IXSCAN	Index scan
FETCH	Retrieves docs from disk
SORT	Sort stage (can be optimized by index)
LIMIT / SKIP	Pagination operators
PROJECTION	Limits returned fields

7 Important Execution Metrics

Metric	Description	Ideal Outcome
nReturned	Docs returned	Matches expected
totalDocsExamined	Docs scanned	≈ nReturned
totalKeysExamined	Keys scanned in index	Small number
executionTimeMillis	Query time	< 10ms for normal ops
winningPlan	Plan chosen by optimizer	Should include IXSCAN

8 Example — Slow Query Analysis

Query:

```
db.orders.find({customer_id: 12345, status: "SHIPPED"})
```

Plan output:

```
"winningPlan": {
  "stage": "COLLSCAN",
  "filter": { "customer_id": 12345, "status": "SHIPPED" }
}
```

✕ Full scan detected → no index used.

Fix:

```
db.orders.createIndex({customer_id: 1, status: 1})
```

✓ After optimization:

```
"winningPlan": { "stage": "IXSCAN" }
"executionTimeMillis": 3
```

9 Optimize Sorts and Projections

Avoid in-memory sorts:

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
db.orders.find().sort({order_date: -1})
```

✕ If no index on order_date, it performs an expensive sort.

✓ Fix:

```
db.orders.createIndex({order_date: -1})
```

Optimize projection to reduce data transfer:

```
db.users.find({}, {name: 1, email: 1, _id: 0})
```

🔟 Aggregation Pipeline Optimization

Example slow aggregation:

```
db.sales.aggregate([
 {$match: {region: "US"}},
 {$sort: {amount: -1}}
])
```

✓ Fix:

```
db.sales.createIndex({region: 1, amount: -1})
```

Optimize $lookup:

- Use indexed join fields
- Use $project early to reduce data size
- Limit output using $limit or $match

1️⃣1️⃣ Real-World Query Tuning Examples

Scenario	Problem	Solution
Searching by field not indexed	COLLSCAN	Add single-field index
Sorting large datasets	Memory sort	Create compound index matching sort
Filtering + Sorting	Two separate indexes	Create compound index {filter, sort}
Array search slow	Multikey scan	Use $elemMatch
Aggregation heavy	CPU bottleneck	Use $project, $match early

1️⃣2️⃣ Using Cursor Batches for Performance

Large result sets? Use **batching**:

cursor = db.logs.find().batchSize(1000)

Reduces memory usage on both client and server.

For pagination:

```
db.logs.find().skip(1000).limit(100)
```

1️⃣3️⃣ Caching and Query Reuse

MongoDB caches:

- Query plans
- Index pages
- Frequently used documents

To benefit:

✓ Keep working set in RAM

✓ Use **read-heavy nodes** for analytical workloads

1️⃣4️⃣ Avoiding Common Performance Pitfalls

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

✕ Unbounded queries → always filter with indexed fields
✕ Over-indexing → each index slows writes
✕ Large $in filters → prefer $or or batch logic
✕ $regex without prefix → full scan
✕ Aggregations without $limit → memory overhead

⚙️15 Server-Level Performance Tuning

Component	Parameter	Recommendation
WiredTiger Cache	--wiredTigerCacheSizeGB	50–60% of RAM
Connections	maxIncomingConnections	Limit excessive clients
Journaling	journalCommitInterval	Tune for latency
Storage	Use SSDs	Faster I/O
NUMA / Swappiness	Set swappiness=1	Reduce memory swapping

🧪16 Hands-On Lab Exercises

Exercise	Task	Validation
Enable profiler	db.setProfilingLevel(2)	system.profile populated
Identify slow queries	find({millis: {$gt: 100}})	List top slow ops
Analyze query plan	.explain("executionStats")	View IXSCAN vs COLLSCAN
Create optimal index	Compound {cust_id, status}	Query <10ms
Tune aggregation	Add $match early	Execution time reduced
Validate tuning	Re-run .explain()	DocsExamined ≈ nReturned

🏁17 Summary

By completing this module, you can:
✔ Identify, analyze, and tune slow queries
✔ Interpret explain plans confidently
✔ Use profiling and aggregation optimization
✔ Apply schema and index strategies to improve performance
✔ Maintain consistently low query latency in production

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Storage Engine Deep Dive (WiredTiger Internals, Compression & Cache Tuning)

Objective: Understand MongoDB's **WiredTiger storage architecture**, including how data is stored, compressed, cached, and checkpointed. Learn how to tune performance, optimize memory, and troubleshoot I/O bottlenecks in production environments.

🧩 1 What is the WiredTiger Storage Engine

Since MongoDB **v3.2**, **WiredTiger** has been the **default storage engine**.
It offers:

- Document-level concurrency (no global lock)
- Compression for data & indexes
- Write-ahead journaling
- In-memory caching
- Checkpoint-based persistence

⚙️ 2 WiredTiger Architecture Overview

Components:

1. **Cache Layer (In-Memory Data Store)**
   Stores hot data pages and indexes for fast access.
2. **Data Files (On-Disk Storage)**
   Each collection and index has its own .wt file.
3. **Journal (Write-Ahead Log)**
   Ensures durability — changes written before commit.
4. **Checkpoints**
   Snapshots ensuring data consistency at certain points.
5. **Compression Engine**
   Reduces disk footprint using Snappy, Zstd, or Zlib.

🔧 Flow of a Write Operation

1 Data written → goes to **cache** (memory)
2 Change recorded in **journal** (for durability)
3 Later written to disk during **checkpoint**
4 Journal truncated after checkpoint completes
✔ Guarantees **Durability (D)** in ACID

🗂️ 3 WiredTiger File Structure

Typical database directory:

```
/var/lib/mongo/
│
├── collection-0--1234567890123456789.wt   → Collection data
├── index-1--9876543210987654321.wt        → Index data
├── WiredTiger.turtle                      → Metadata about config
├── WiredTiger.wt                          → Internal catalog
├── WiredTiger.lock                        → Process lock
└── journal/                               → Write-ahead logs
```

🧠 4 Checkpointing Mechanism

- MongoDB periodically takes **checkpoints** (by default every 60 seconds).
- Checkpoints flush **dirty pages** from memory to disk.
- They ensure **crash recovery** without replaying the entire journal.

Checkpoints are triggered:

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

- Every 60 seconds
- When journal size exceeds threshold
- On clean shutdown

5️⃣ Compression in WiredTiger

Compression Type	Applies To	Default	Use Case
Snappy	Data & Index	✔ Default	Fast & moderate compression
Zlib	Data	✖	Better compression, slower writes
Zstd	Data	Optional	High compression with good speed
None	Data & Index	Optional	For in-memory / low-latency

Configure Compression (mongod.conf):

```
storage:
  wiredTiger:
    collectionConfig:
      blockCompressor: zstd
    indexConfig:
      prefixCompression: true
```

6️⃣ WiredTiger Cache Management

MongoDB's **WiredTiger cache** holds frequently accessed data.

If your working set fits in cache → performance skyrockets 🚀

Default Cache Size

≈ **50% of system RAM** (up to 10 GB minimum).

View current size:

```
db.serverStatus().wiredTiger.cache["maximum bytes configured"]
```

Monitor usage:

```
db.serverStatus().wiredTiger.cache
```

Tune Cache Size

In mongod.conf:

```
storage:
  wiredTiger:
    engineConfig:
      cacheSizeGB: 16
```

Or via command line:

```
mongod --wiredTigerCacheSizeGB 16
```

7️⃣ Key Cache Metrics

Metric	Description	Healthy Range
bytes currently in cache	Memory in use	< 80% of limit
tracked dirty bytes in cache	Unflushed writes	< 20%
pages evicted because they exceeded the in-memory cache	Indicates cache pressure	Low count ideal
pages read into cache	Disk reads	Should be stable
pages written from cache	Disk writes	Should match checkpoint cycles

8️⃣ Eviction Process

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

When cache is full, WiredTiger **evicts pages** (cold data) to disk.
Triggers:

- Cache > 80% full
- Write pressure high

You can monitor eviction activity:

```
db.serverStatus().wiredTiger.cache["pages evicted by application threads"]
```

High eviction = potential memory bottleneck.

⚙️ 9 Journaling and Durability

- Journal ensures **no data loss** during crashes.
- Uses **write-ahead logging (WAL)**.

Journal Directory:

```
/var/lib/mongo/journal/
```

Journal Tuning Options:

```
storage:
  journal:
    enabled: true
    commitIntervalMs: 100
```

🔄 Lower interval → higher durability, more I/O.
Raise interval for throughput-heavy systems.

🔟 Disk & I/O Tuning Recommendations

Area	Recommendation
Disk Type	Use SSD (NVMe preferred)
Filesystem	XFS (recommended)
Mount Options	noatime, nodiratime
RAID	RAID10 for balance of performance and redundancy
IO Scheduler	deadline or noop
Swap	Disable or set swappiness=1
Journal Partition	Separate from data for heavy write loads

1️⃣1️⃣ Backup Considerations with WiredTiger

- Always backup using **mongodump** or **filesystem snapshots**.
- For file-level backup:
  - Use **fsyncLock()** to flush and lock writes.
  - Snapshot data + journal directories.
  - Unlock after snapshot.

```
db.fsyncLock()
# Take snapshot
db.fsyncUnlock()
```

1️⃣2️⃣ WiredTiger Statistics Commands

Check WiredTiger stats:

```
db.serverStatus().wiredTiger
```

Check collection-level stats:

```
db.collection.stats()
```

Check file-level usage:

```
ls -lh /var/lib/mongo/*.wt
```

🐞 1️⃣3️⃣ Common WiredTiger Issues

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Symptom	Root Cause	Fix
High eviction count	Cache too small	Increase cache size
Slow writes	Compression overhead	Switch to Snappy
Journal I/O bottleneck	Disk contention	Move journal to separate volume
“Data too large for cache”	Large working set	Scale vertically or shard
Long checkpoint times	Huge dirty page count	Tune write intervals

🔢 Hands-On Lab Exercises

Exercise	Task	Validation
Check cache usage	db.serverStatus().wiredTiger.cache	Cache < 80% full
Change compression	Edit mongod.conf → zstd	collection.stats() shows new compression
Tune cache size	cacheSizeGB: 8	Validate via serverStatus()
Simulate eviction	Load large dataset	Observe pages evicted rise
Measure I/O throughput	iostat -x	High util% → I/O bound
Backup test	db.fsyncLock() snapshot	Restore successful

🔢 Best Practices for WiredTiger Tuning

✔ Keep working set within cache
✔ Use Snappy unless compression is critical
✔ Monitor eviction + dirty pages regularly
✔ Use SSDs for high write throughput
✔ Isolate journal files on separate disks
✔ Tune checkpoints if latency spikes
✔ Automate performance metrics collection (Prometheus / MMS)

🔢 Summary

By the end of this module, you can:
✔ Explain WiredTiger internals and file structure
✔ Configure compression and caching for optimal performance
✔ Interpret WiredTiger metrics and troubleshoot bottlenecks
✔ Perform advanced cache tuning and journal optimization
✔ Build production-grade storage strategies for MongoDB clusters

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Schema Design & Data Modeling (Normalization, Embedding, Referencing, and Best Practices)

Objective: Understand how to design efficient MongoDB schemas — how to structure collections, choose between embedding and referencing, and optimize for queries, writes, and scaling.

1️⃣ SQL vs NoSQL Schema Mindset

Concept	SQL	MongoDB
Data Model	Normalized, relational tables	Flexible, document-based
Joins	Handled by SQL engine	Avoided or done in application
Schema Enforcement	Strict (DDL)	Dynamic (JSON schema optional)
Data Integrity	Foreign keys	Application-driven or references
Scaling	Vertical (scale up)	Horizontal (sharding, replica sets)

MongoDB schema design is **query-driven**, not normalization-driven.

2️⃣ BSON Document Structure

MongoDB stores data in **BSON** (Binary JSON) format.
Example:

```
{
  "_id": ObjectId("5f9a..."),
  "name": "John Doe",
  "email": "john@example.com",
  "orders": [
    { "order_id": 1, "total": 250 },
    { "order_id": 2, "total": 480 }
  ]
}
```

- BSON supports embedded arrays, sub-documents, and binary data.
- Enables **rich hierarchical data models**.

⚙️3️⃣ Schema Design Strategies

MongoDB offers two primary modeling techniques:

A. Embedding (Denormalization)

Store related data **inside the same document**.

Example:

```
{
  "_id": 1,
  "customer": "Alice",
  "orders": [
    { "order_id": 101, "amount": 500 },
    { "order_id": 102, "amount": 800 }
  ]
}
```

✅ **Advantages:**

- Fast reads (single document)
- Atomic updates
- Fewer joins / lookups

✕ **Disadvantages:**

- Large documents (>16 MB)
- Repeated data
- Difficult partial updates

B. Referencing (Normalization)

Store related data in **separate collections**.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Example:

// customers
{ "_id": 1, "name": "Alice" }

// orders
{ "_id": 101, "customer_id": 1, "amount": 500 }

✔ **Advantages:**

- Reusable data
- Smaller documents
- Easier updates

✕ **Disadvantages:**

- Multiple queries or $lookup joins
- Slightly slower reads

⚖️ 4 Choosing Between Embedding vs Referencing

Use Case	Recommended Model
One-to-One	Embed
One-to-Few	Embed
One-to-Many (small)	Embed
One-to-Many (large or growing)	Reference
Many-to-Many	Reference
High read frequency	Embed
High update frequency	Reference
Data reused across entities	Reference

🧱 5 Schema Design Example

Scenario:

E-commerce application with customers, orders, and products.

Option 1 — Embedded Orders

```
{
  "_id": 1,
  "name": "John",
  "orders": [
    { "product": "Phone", "qty": 1 },
    { "product": "Laptop", "qty": 2 }
  ]
}
```

Option 2 — Referenced Orders

// customers
{ "_id": 1, "name": "John" }

// orders
{ "_id": 1001, "customer_id": 1, "product": "Phone", "qty": 1 }

Decision:

- If customers make <20 orders → embed.
- If customers have thousands → reference.

🧠 6 MongoDB Schema Design Rules

✔ Design around **application queries**, not ER diagrams.

✔ Keep frequently accessed data **in the same document**.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

✔ Avoid deep nesting (>3 levels).
✔ Use **indexes** on query filters and sort keys.
✔ Avoid unbounded arrays.
✔ Use **schema validation** for critical collections.
✔ Consider **document growth patterns** to avoid padding.

7️⃣ Schema Validation Example

MongoDB supports JSON schema validation:

```
db.createCollection("users", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "email"],
      properties: {
        name: { bsonType: "string" },
        email: { bsonType: "string" },
        age: { bsonType: "int", minimum: 0 }
      }
    }
  }
})
```

Validation helps enforce consistency without rigid schema migrations.

8️⃣ Data Modeling Patterns

Pattern	Description	Use Case
Subset Pattern	Store partial embedded arrays	Only recent comments/orders
Outlier Pattern	Separate very large documents	User profiles with rare large sections
Extended Reference Pattern	Include key fields from referenced doc	Faster lookup with partial embedding
Computed Pattern	Store precomputed aggregates	Dashboard metrics
Bucket Pattern	Group time-series data	IoT, logs, metrics
Attribute Pattern	Dynamic fields stored as key-value pairs	Variable data per user

9️⃣ Example — Bucket Pattern

Time-Series Logs Example:

```
{
  "_id": ObjectId(),
  "sensor_id": "A1",
  "timestamp_minute": "2025-10-14T10:00:00Z",
  "readings": [
    { "t": "10:00:01", "v": 22.1 },
    { "t": "10:00:05", "v": 22.4 }
  ]
}
```

✔ Reduces write amplification
✔ Efficient for range queries
✔ Common in monitoring and IoT systems

⚙ 🔟 Indexing Considerations in Schema Design

- Design schema **and** indexes together.
- Use compound indexes on frequent queries:
- db.orders.createIndex({ customer_id: 1, order_date: -1 })

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

- Avoid too many indexes (each consumes RAM).
- Use partial indexes to reduce size.
- For nested fields:
- db.users.createIndex({ "address.city": 1 })

11 Schema Evolution Techniques

MongoDB supports flexible evolution:

- Add new fields dynamically.
- Use $unset to remove deprecated ones.
- Write migration scripts with updateMany().
- Maintain backward compatibility in your application.

Example:

```
db.users.updateMany({}, { $set: { status: "active" } })
```

12 Hands-On Lab Checklist

Exercise	Task	Validation
Create sample schema	Build customers & orders collections	Verify document embedding
Apply validation rules	Create JSON schema	Insert invalid doc → rejected
Apply bucket pattern	Group IoT readings	Confirm grouped documents
Compare read times	Query embedded vs referenced	Analyze query stats
Run explain plans	db.collection.find().explain("executionStats")	Ensure indexed scans

13 Performance Optimization Tips

✔ Use **embedding** for read-heavy workloads
✔ Use **referencing** for write-heavy or frequently updated data
✔ Avoid unbounded arrays — consider bucketing
✔ Use **capped collections** for rolling logs
✔ Minimize document size growth
✔ Cache computed values (denormalization)

14 Schema Design Anti-Patterns

Anti-Pattern	Description	Impact
Massive documents (>16MB)	Over-embedding	Write failures
Deep nesting (>3 levels)	Hard to query	Slow access
Large unbounded arrays	Growing data	Performance issues
Overuse of $lookup	Excessive joins	High latency
Too many indexes	Write amplification	Increased storage
Ignoring access patterns	Poor design	Inefficient reads/writes

15 Schema Review Checklist

✔ Data model fits query patterns
✔ No unbounded arrays
✔ Indexes support filters & sorts
✔ Document growth within limits

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

✔ Schema validation rules defined
✔ Compression and TTL as needed
✔ Scalable with sharding

🏁 Summary

By completing this module, you can:

- Design schemas tailored to your application queries
- Decide when to embed or reference
- Implement data modeling patterns for scalability
- Use validation and indexing for integrity
- Avoid schema anti-patterns and bottlenecks

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Query Optimization & Index Tuning (Explain Plans, Index Types, and Query Patterns)

Objective: Learn to diagnose, tune, and optimize MongoDB queries using indexes, explain plans, and performance best practices.

1 Why Query Optimization Matters

MongoDB performance depends on:

- Query pattern efficiency
- Index usage
- Data volume and working set size
- Storage engine behavior

A single missing index can turn a **milliseconds query** into **seconds or minutes**.

2 Query Lifecycle in MongoDB

1 Client issues a query
2 Query planner evaluates possible index paths
3 Best plan selected (cached for reuse)
4 Data fetched from cache or disk
5 Results returned to client

MongoDB automatically caches **query plans** for frequently executed queries.

3 Explain Plan Basics

explain() shows how MongoDB executes a query.

Example:

```
db.orders.find({ customer_id: 1001 }).explain("executionStats")
```

Key Output Sections:

Field	Description
queryPlanner	Query plan, index used, stage types
winningPlan	Actual plan used
rejectedPlans	Other considered plans
executionStats	Docs scanned, returned, time taken
totalDocsExamined	Count of documents scanned
nReturned	Number of results
executionTimeMillis	Query runtime

Example Output (simplified)

```
{
  "winningPlan": {
    "stage": "IXSCAN",
    "keyPattern": { "customer_id": 1 }
  },
  "executionStats": {
    "nReturned": 1,
    "totalDocsExamined": 1,
    "executionTimeMillis": 2
  }
}
```

✔ Index used
✔ Only 1 doc scanned → efficient

4 Detecting Slow Queries

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Symptoms:

- High executionTimeMillis
- COLLSCAN in explain plan (collection scan)
- High CPU / disk I/O
- Many documents examined vs returned

Enable **slow query logging**:

```
db.setProfilingLevel(1, { slowms: 100 })
```

View slow queries:

```
db.system.profile.find().sort({ ts: -1 }).limit(5).pretty()
```

5 Types of Indexes in MongoDB

Index Type	Description	Use Case
Single Field Index	One field only	Simple lookups
Compound Index	Multiple fields	Queries using multiple filters
Multikey Index	Arrays	For fields with array values
Text Index	Text search	Keyword searches
Hashed Index	Hash-based	Sharding keys, equality queries
Wildcard Index	Dynamic keys	Flexible schema collections
TTL Index	Auto-deletion	Expire old logs/sessions
Sparse Index	Excludes missing fields	Partial data indexing
Partial Index	Index subset of documents	Optimize for specific conditions

Example: Creating Different Indexes

```
db.customers.createIndex({ email: 1 })           // Single
db.orders.createIndex({ customer_id: 1, date: -1 }) // Compound
db.products.createIndex({ tags: 1 })             // Multikey
db.logs.createIndex({ createdAt: 1 }, { expireAfterSeconds: 86400 }) // TTL
```

6 Index Selection Rules

✔ Index the fields used in:

- Filters (find conditions)
- Sorts (sort)
- Join conditions ($lookup)
- Group keys ($group)
- Frequent aggregations

✔ Keep indexes small — they live in RAM.

✔ Avoid overlapping redundant indexes.

7 Compound Index Design

Order matters!

```
db.orders.createIndex({ customer_id: 1, order_date: -1 })
```

Query:

```
db.orders.find({ customer_id: 1001 }).sort({ order_date: -1 })
```

✔ Fully optimized (prefix rule matched)

The Prefix Rule:

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

If index = { A: 1, B: 1, C: 1 }

Query	Uses Index?
{ A: 1 }	✔
{ A: 1, B: 1 }	✔
{ B: 1 }	✖ (prefix missing)

8 Covered Queries

A **covered query** is one where all required fields are in the index — no collection scan needed.

Example:

```
db.users.createIndex({ email: 1, name: 1 })
db.users.find({ email: "test@example.com" }, { _id: 0, name: 1 })
```

✔ Query is fully covered by index

⚡ Very fast — no data fetch from disk

9 Index Cardinality & Selectivity

Cardinality = Number of unique values

Selectivity = How selective an index is

Example:

- Gender: Low selectivity (M/F)
- Email: High selectivity (unique)

👉 Always prefer indexes on **high-cardinality fields** for better filtering.

Check distinct values:

```
db.collection.distinct("field").length
```

10 Sorting and Indexing

Avoid in-memory sorts (high memory cost).

If sort field not indexed:

```
"stage": "SORT"
```

Fix it:

```
db.orders.createIndex({ status: 1, order_date: -1 })
```

Re-run explain — should show:

```
"stage": "IXSCAN"
```

11 Query Plan Caching

MongoDB caches **winning plans** for repeated queries.

View cache entries:

```
db.runCommand({ planCacheListPlans: "orders" })
```

Clear cache (if plan changed):

```
db.runCommand({ planCacheClear: "orders" })
```

12 Aggregation Pipeline Optimization

Use $match and $project early in pipeline:

```
db.orders.aggregate([
 { $match: { status: "PAID" } },
 { $project: { customer_id: 1, total: 1 } },
 { $group: { _id: "$customer_id", totalSpent: { $sum: "$total" } } }
])
```

✔ Filters early → fewer docs downstream

✔ Reduce CPU/memory cost

13 Query Optimization Tools

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Tool	Purpose
explain()	Execution plan
serverStatus()	Server metrics
$indexStats	Index usage frequency
system.profile	Slow query log
Atlas Performance Advisor	Cloud index suggestions
Compass Query Profiler	Visual query latency

Example: Index Usage Stats

```
db.orders.aggregate([{ $indexStats: {} }])
```

Output:

```
{ "name": "customer_id_1", "accesses": { "ops": 1200 } }
```

Shows index hit count — helps identify unused indexes.

🧪 Hands-On Lab Checklist

Exercise	Task	Validation
Create slow query	Run find without index	Explain shows COLLSCAN
Add index	createIndex()	Explain shows IXSCAN
Create compound index	On filter + sort fields	Query uses index for both
Analyze performance	Compare executionTimeMillis	Reduced after indexing
Test covered query	Query returns only indexed fields	No collection scan
Enable profiling	db.setProfilingLevel(1, {slowms: 100})	Profile logs created

🧭 Best Practices for Index & Query Optimization

✔ Create indexes based on query frequency, not guesswork
✔ Keep indexes under 20% of dataset size
✔ Use compound indexes wisely
✔ Drop unused or redundant indexes
✔ Monitor indexStats regularly
✔ Keep your working set in RAM
✔ Review explain plans periodically

🚨 Common Query Performance Pitfalls

Problem	Root Cause	Fix
COLLSCAN	Missing index	Create proper index
Slow $lookup	No index on join key	Add index on foreign field
High CPU	Unoptimized aggregation	Filter early, use $project
Large I/O	Huge documents	Use projection fields
Slow pagination	Skip/limit overhead	Use range queries or _id filters
Many unused indexes	Write amplification	Drop with dropIndex()

🧠 Advanced Indexing Features

- **Partial Indexes**

```
db.orders.createIndex({ status: 1 }, { partialFilterExpression: { status: "PAID" } })
```

- **Sparse Indexes**

```
db.users.createIndex({ phone: 1 }, { sparse: true })
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

- **Wildcard Indexes**

```
db.inventory.createIndex({ "$**": 1 })
```

- **TTL Indexes**

```
db.sessions.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 })
```

🏁 **Summary**

By completing this module, you can:

- Interpret MongoDB explain plans
- Detect and fix slow queries
- Design and tune indexes effectively
- Monitor query and index performance
- Apply optimization best practices for production-grade workloads

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Query Optimization & Index Tuning (Explain Plans, Index Types, and Query Patterns)

Objective: Learn to diagnose, tune, and optimize MongoDB queries using indexes, explain plans, and performance best practices.

1 Why Query Optimization Matters

MongoDB performance depends on:

- Query pattern efficiency
- Index usage
- Data volume and working set size
- Storage engine behavior

A single missing index can turn a **milliseconds query** into **seconds or minutes**.

2 Query Lifecycle in MongoDB

1 Client issues a query
2 Query planner evaluates possible index paths
3 Best plan selected (cached for reuse)
4 Data fetched from cache or disk
5 Results returned to client

MongoDB automatically caches **query plans** for frequently executed queries.

3 Explain Plan Basics

explain() shows how MongoDB executes a query.

Example:

```
db.orders.find({ customer_id: 1001 }).explain("executionStats")
```

Key Output Sections:

Field	Description
queryPlanner	Query plan, index used, stage types
winningPlan	Actual plan used
rejectedPlans	Other considered plans
executionStats	Docs scanned, returned, time taken
totalDocsExamined	Count of documents scanned
nReturned	Number of results
executionTimeMillis	Query runtime

Example Output (simplified)

```
{
  "winningPlan": {
    "stage": "IXSCAN",
    "keyPattern": { "customer_id": 1 }
  },
  "executionStats": {
    "nReturned": 1,
    "totalDocsExamined": 1,
    "executionTimeMillis": 2
  }
}
```

✔ Index used
✔ Only 1 doc scanned → efficient

4 Detecting Slow Queries

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Symptoms:

- High executionTimeMillis
- COLLSCAN in explain plan (collection scan)
- High CPU / disk I/O
- Many documents examined vs returned

Enable **slow query logging**:

```
db.setProfilingLevel(1, { slowms: 100 })
```

View slow queries:

```
db.system.profile.find().sort({ ts: -1 }).limit(5).pretty()
```

5 Types of Indexes in MongoDB

Index Type	Description	Use Case
Single Field Index	One field only	Simple lookups
Compound Index	Multiple fields	Queries using multiple filters
Multikey Index	Arrays	For fields with array values
Text Index	Text search	Keyword searches
Hashed Index	Hash-based	Sharding keys, equality queries
Wildcard Index	Dynamic keys	Flexible schema collections
TTL Index	Auto-deletion	Expire old logs/sessions
Sparse Index	Excludes missing fields	Partial data indexing
Partial Index	Index subset of documents	Optimize for specific conditions

Example: Creating Different Indexes

```
db.customers.createIndex({ email: 1 })           // Single
db.orders.createIndex({ customer_id: 1, date: -1 }) // Compound
db.products.createIndex({ tags: 1 })             // Multikey
db.logs.createIndex({ createdAt: 1 }, { expireAfterSeconds: 86400 }) // TTL
```

6 Index Selection Rules

✔ Index the fields used in:

- Filters (find conditions)
- Sorts (sort)
- Join conditions ($lookup)
- Group keys ($group)
- Frequent aggregations

✔ Keep indexes small — they live in RAM.

✔ Avoid overlapping redundant indexes.

7 Compound Index Design

Order matters!

```
db.orders.createIndex({ customer_id: 1, order_date: -1 })
```

Query:

```
db.orders.find({ customer_id: 1001 }).sort({ order_date: -1 })
```

✔ Fully optimized (prefix rule matched)

The Prefix Rule:

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

If index = { A: 1, B: 1, C: 1 }

Query	Uses Index?
{ A: 1 }	✔
{ A: 1, B: 1 }	✔
{ B: 1 }	✖ (prefix missing)

8 Covered Queries

A **covered query** is one where all required fields are in the index — no collection scan needed.

Example:

```
db.users.createIndex({ email: 1, name: 1 })
db.users.find({ email: "test@example.com" }, { _id: 0, name: 1 })
```

✔ Query is fully covered by index

⚡ Very fast — no data fetch from disk

9 Index Cardinality & Selectivity

Cardinality = Number of unique values

Selectivity = How selective an index is

Example:

- Gender: Low selectivity (M/F)
- Email: High selectivity (unique)

👉 Always prefer indexes on **high-cardinality fields** for better filtering.

Check distinct values:

```
db.collection.distinct("field").length
```

10 Sorting and Indexing

Avoid in-memory sorts (high memory cost).

If sort field not indexed:

```
"stage": "SORT"
```

Fix it:

```
db.orders.createIndex({ status: 1, order_date: -1 })
```

Re-run explain — should show:

```
"stage": "IXSCAN"
```

11 Query Plan Caching

MongoDB caches **winning plans** for repeated queries.

View cache entries:

```
db.runCommand({ planCacheListPlans: "orders" })
```

Clear cache (if plan changed):

```
db.runCommand({ planCacheClear: "orders" })
```

12 Aggregation Pipeline Optimization

Use $match and $project early in pipeline:

```
db.orders.aggregate([
 { $match: { status: "PAID" } },
 { $project: { customer_id: 1, total: 1 } },
 { $group: { _id: "$customer_id", totalSpent: { $sum: "$total" } } }
])
```

✔ Filters early → fewer docs downstream

✔ Reduce CPU/memory cost

13 Query Optimization Tools

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Tool	Purpose
explain()	Execution plan
serverStatus()	Server metrics
$indexStats	Index usage frequency
system.profile	Slow query log
Atlas Performance Advisor	Cloud index suggestions
Compass Query Profiler	Visual query latency

Example: Index Usage Stats

```
db.orders.aggregate([{ $indexStats: {} }])
```

Output:

```
{ "name": "customer_id_1", "accesses": { "ops": 1200 } }
```

Shows index hit count — helps identify unused indexes.

🧪 Hands-On Lab Checklist

Exercise	Task	Validation
Create slow query	Run find without index	Explain shows COLLSCAN
Add index	createIndex()	Explain shows IXSCAN
Create compound index	On filter + sort fields	Query uses index for both
Analyze performance	Compare executionTimeMillis	Reduced after indexing
Test covered query	Query returns only indexed fields	No collection scan
Enable profiling	db.setProfilingLevel(1, {slowms: 100})	Profile logs created

🧭 Best Practices for Index & Query Optimization

✔ Create indexes based on query frequency, not guesswork
✔ Keep indexes under 20% of dataset size
✔ Use compound indexes wisely
✔ Drop unused or redundant indexes
✔ Monitor indexStats regularly
✔ Keep your working set in RAM
✔ Review explain plans periodically

🚨 Common Query Performance Pitfalls

Problem	Root Cause	Fix
COLLSCAN	Missing index	Create proper index
Slow $lookup	No index on join key	Add index on foreign field
High CPU	Unoptimized aggregation	Filter early, use $project
Large I/O	Huge documents	Use projection fields
Slow pagination	Skip/limit overhead	Use range queries or _id filters
Many unused indexes	Write amplification	Drop with dropIndex()

🧠 Advanced Indexing Features

- **Partial Indexes**

```
db.orders.createIndex({ status: 1 }, { partialFilterExpression: { status: "PAID" } })
```

- **Sparse Indexes**

```
db.users.createIndex({ phone: 1 }, { sparse: true })
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

- **Wildcard Indexes**

```
db.inventory.createIndex({ "$**": 1 })
```

- **TTL Indexes**

```
db.sessions.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 })
```

🏁 **Summary**

By completing this module, you can:

- Interpret MongoDB explain plans
- Detect and fix slow queries
- Design and tune indexes effectively
- Monitor query and index performance
- Apply optimization best practices for production-grade workloads

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Transactions & Concurrency Control (ACID, Locks, and Write Conflicts)

Objective: Understand how MongoDB ensures data consistency and integrity through **transactions**, **locking**, and **isolation mechanisms** — critical for production-grade database reliability.

1️⃣ Understanding MongoDB Transaction Model

MongoDB started as a **document-level atomic** database — each single document write was atomic.
Since version **4.0**, MongoDB supports **multi-document ACID transactions** on replica sets, and from **4.2**, on **sharded clusters** as well.

⚙️ 2️⃣ MongoDB and the ACID Properties

Property	Description	MongoDB Implementation
Atomicity	All or nothing	Each document update is atomic; transactions extend it across multiple docs
Consistency	Valid state after transaction	Enforced via schema validation, write concerns
Isolation	Transactions don't affect each other	Snapshot isolation using MVCC
Durability	Changes persist after commit	Journaled writes, writeConcern: majority

3️⃣ Single vs Multi-Document Transactions

Single Document Example

```
db.accounts.updateOne(
  { _id: 1001 },
  { $inc: { balance: -500 } }
);
```

✔ Automatically atomic — no explicit transaction required.

Multi-Document Example (ACID Transaction)

```
const session = db.getMongo().startSession();
session.startTransaction();

try {
  const accounts = session.getDatabase("bank").accounts;
  accounts.updateOne({ _id: 1001 }, { $inc: { balance: -500 } });
  accounts.updateOne({ _id: 1002 }, { $inc: { balance: +500 } });

  session.commitTransaction();
  print("✔ Transaction committed.");
} catch (e) {
  session.abortTransaction();
  print("✖ Transaction aborted due to: " + e);
} finally {
  session.endSession();
}
```

✔ Ensures debit and credit both succeed or both fail — like SQL BEGIN TRAN / COMMIT.

4️⃣ Transactions in Sharded Clusters

- Supported since MongoDB 4.2.
- Transactions can span **multiple shards**.
- **Mongos router** coordinates across shards.
- Commit protocol: **Two-phase commit (2PC)**.

Example:

```
session.startTransaction();
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
db.orders.insertOne({ orderId: 100, amount: 250 }, { session });
db.payments.insertOne({ orderId: 100, status: "PAID" }, { session });
session.commitTransaction();
```

5 Write Concerns and Read Concerns

- **Write Concern**

  Defines the level of acknowledgment requested from MongoDB for write operations.

Write Concern	Behavior
w: 1	Acknowledged by primary only
w: majority	Acknowledged by majority of nodes
journal: true	Ensures write is committed to journal

- **Read Concern**

  Defines isolation level for read operations.

Read Concern	Description
local	Reads from primary (no guarantee)
majority	Reads confirmed by majority
snapshot	Used inside transactions
linearizable	Guarantees strict ordering (highest consistency)

6 Locking and Concurrency Internals

MongoDB uses **fine-grained locking** mechanisms:

- **Global Lock** → (rarely used now)
- **Database Lock**
- **Collection Lock**
- **Document-Level Lock**

Each operation holds locks **only as long as needed**.

Example:

- Two different documents → can be updated concurrently.
- Two updates to same document → second waits until first releases lock.

Result: **High concurrency with safe consistency.**

7 Snapshot Isolation (MVCC)

MongoDB uses **Multi-Version Concurrency Control (MVCC)** to ensure:

- Reads see a consistent snapshot of data.
- Writes do not block reads.
- Readers never wait for writers.

Similar to PostgreSQL's snapshot isolation level.

8 Handling Write Conflicts

If two transactions modify the same document concurrently:

WriteConflict error: WriteConflict detected

MongoDB automatically retries **some** internal operations, but for user transactions, you must handle retries manually.

Example Retry Logic

```
function runTransactionWithRetry(txnFunc, session) {
  while (true) {
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
  try {
    txnFunc(session);
    break;
  } catch (error) {
    if (error.hasErrorLabel("TransientTransactionError")) {
      print("⚠ Retrying transaction...");
      continue;
    } else {
      throw error;
    }
  }
 }
}
```

9️⃣ Transaction Lifetime and Timeout

Setting	Description
transactionLifetimeLimitSeconds	Default = 60 seconds
maxCommitTimeMS	Limit on commit duration

Idle sessions auto-abort after timeout.

Use db.adminCommand({ getParameter: 1, transactionLifetimeLimitSeconds: 1 }) to check configuration.

🔟 Transactions in Replica Sets

- Writes are always done on **primary**.
- **Replication lag** affects read-your-writes consistency.
- Use readConcern: "majority" for consistent reads.
- During primary failover → in-progress transactions abort.

1️⃣1️⃣ Transactions in Sharded Clusters (Advanced)

- **Router (mongos)** splits the transaction across shards.
- **Coordinator shard** drives the commit.
- Uses **two-phase commit protocol (2PC)**:
  1. Prepare phase — shards confirm readiness.
  2. Commit phase — all shards finalize changes.

🌩️ Log entries ensure durability even if crash occurs mid-commit.

1️⃣2️⃣ Monitoring Transaction Performance

Command	Description
db.currentOp({ "transaction": { $exists: true } })	View running transactions
db.serverStatus().transactions	Transaction metrics
db.killOp(opid)	Terminate long transactions
db.system.profile.find({ "transaction": { $exists: true } })	Profiler view

Metrics to Monitor:

- transactions.totalCommitted
- transactions.totalAborted
- transactions.currentActive
- transactions.retryCommitted

1️⃣3️⃣ Common Errors and Fixes

Error	Cause	Fix
WriteConflict	Two writes on same doc	Implement retry logic

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Error	Cause	Fix
TransientTransactionError	Network/replica lag	Retry transaction
NoSuchTransaction	Timeout	Increase transactionLifetimeLimitSeconds
LockTimeout	Resource contention	Optimize index, reduce batch size

🔟4️⃣ Hands-On Lab Checklist

Exercise	Task	Validation
Basic transaction	Debit/Credit accounts	Balances remain consistent
Retry logic	Simulate WriteConflict	Transaction retries successfully
Timeout testing	Lower transactionLifetimeLimitSeconds	Transaction aborts as expected
Multi-shard transaction	Insert into 2 shards	Data consistent in both
Monitoring	Use db.serverStatus().transactions	Metrics reflect commit counts

1️⃣1️⃣5️⃣ Best Practices Summary

✔ Keep transactions **short** (few documents, quick commits)
✔ Always handle **TransientTransactionError**
✔ Use writeConcern: "majority" for durability
✔ Use readConcern: "snapshot" inside transactions
✔ Monitor serverStatus().transactions regularly
✔ Avoid long-running read/write locks inside transactions
✔ Break complex updates into smaller atomic batches

1️⃣1️⃣6️⃣ Transaction vs Non-Transaction Design

Use Case	Transaction Needed?	Why
Multi-doc banking transfers	✔ Yes	Atomic debit/credit
Single document update	✖ No	Already atomic
Logging events	✖ No	Append-only
Multi-shard consistency	✔ Yes	Needs cross-shard ACID
Caching / analytics writes	✖ No	Performance > atomicity

🏁 Summary

By the end of this module, you can:
✔ Implement multi-document ACID transactions
✔ Configure read and write concerns correctly
✔ Understand locking and MVCC
✔ Handle conflicts and retries safely
✔ Monitor and tune transaction performance

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Backup, Recovery, and Automation — the foundation of any **Enterprise-Grade MongoDB DR strategy**.

MongoDB Backup Strategies, Point-in-Time Recovery (PITR), and Automation Using Ops Manager / Atlas Backup
Objective Learn how to design, perform, verify, and automate **MongoDB backup and recovery** processes — ensuring high data availability and compliance for production environments.

1️⃣ Why Backups Are Critical

Even with replication and sharding, **backups are irreplaceable** for:

- Accidental deletions or schema corruption
- Data migrations or testing environments
- Regulatory compliance (retention policies)
- Disaster Recovery (DR) readiness

💡 *Replication ≠ Backup* — replication only copies live state, not history.

2️⃣ MongoDB Backup Types

Type	Description	Tool/Method
Logical Backup	BSON dump of collections	mongodump, mongoexport
Physical Backup	File system-level snapshot of data files	LVM, fsyncLock, EBS snapshot
Cloud Backup	Managed continuous backup in Atlas	Atlas Backups
Automated / Scheduled	Managed by Ops Manager	Ops Manager / MMS
Oplog-based Incremental Backup	Continuous PITR (Point-in-Time Recovery)	Ops Manager or Atlas Continuous Backup

3️⃣ Logical Backup — mongodump

Basic Command

```
mongodump --host localhost --port 27017 --out /backups/mongobackup_$(date +%F)
```

Backup Specific Database

```
mongodump --db sales --out /backups/sales_backup
```

Backup Specific Collection

```
mongodump --db sales --collection orders --out /backups/sales_orders
```

Authentication Example

```
mongodump --uri "mongodb://admin:<password>@localhost:27017/sales?authSource=admin" --out /data/backup
```

4️⃣ Restore — mongorestore

Restore Entire Backup

```
mongorestore /backups/mongobackup_2025-10-14
```

Restore Specific DB

```
mongorestore --db sales /backups/sales_backup/sales
```

Drop Before Restore

```
mongorestore --drop --db sales /backups/sales_backup/sales
```

Authentication Example

```
mongorestore --uri "mongodb://admin:<password>@localhost:27017/" /backups/sales_backup
```

✔ **Validation:**

```
db.stats()
db.orders.count()
```

5️⃣ Physical Backup — File System Snapshot

Used for **faster restoration** of very large databases (100GB+).

◆ **Steps:**

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

1. **Lock writes**
2. db.fsyncLock()
3. **Copy Data Files**
4. cp -r /var/lib/mongo /mnt/backup/
5. **Unlock**
6. db.fsyncUnlock()

⚠ **Note:**

- Data must reside on a consistent filesystem (e.g., LVM, EBS, ZFS snapshot).
- Journaling ensures crash consistency.

☁ MongoDB Atlas Backup Options

MongoDB Atlas offers **two** types of managed backups:

Type	Description	PITR Support
Continuous Backup	Streams oplog for continuous restore	✔ Yes
Snapshot Backup	Periodic full snapshots	✖ No (only point snapshots)

Continuous Backup Example

- Backups stored in cloud storage.
- PITR up to 35 days retention.
- Restore from **any timestamp** within retention window.

Restore Process in Atlas

1. Navigate to **Backups** → **Continuous Backup** tab.
2. Click **Restore** → choose point in time.
3. Restore to:
    - New cluster
    - Existing cluster
    - Downloadable snapshot (optional)

⚙ Ops Manager Backup Automation

For **on-prem deployments**, **Ops Manager** provides:

- Scheduled backups
- Incremental backups using oplog
- Automated restore jobs
- Backup retention policies
- PITR recovery window

Key Features:

✔ Compression & Encryption

✔ Oplog-based incremental backups

✔ Full cluster restore

✔ DR Region replication

Configuration Steps:

1. Deploy **Backup Daemon** on Ops Manager host.
2. Register MongoDB clusters.
3. Configure **Snapshot Schedule** (e.g., every 6 hrs).
4. Enable **PITR window** (default 24h).
5. Monitor jobs via Ops Manager UI.

🕒 Point-in-Time Recovery (PITR)

Allows restoring data **to an exact timestamp** — e.g., just before accidental deletion.

Concept:

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

1. Perform periodic **full snapshot**.
2. Record **oplog entries** continuously.
3. Reapply oplog entries up to desired timestamp.

 Example Command (Ops Manager)

```
om-cli restore create \
  --clusterName "ProdCluster" \
  --snapshotId 62d4f... \
  --pointInTime "2025-10-14T09:30:00Z"
```

9 Backup Validation

Always test restore regularly.

Validation Checklist:

Check	Command	Expected Result
DB exists	show dbs	Appears
Collection count	db.orders.count()	Matches source
Data checksum	md5sum / db.hashes	Identical
Indexes	db.orders.getIndexes()	Present
Backup logs	cat /var/log/mongodb/mongodump.log	No errors

 Follow "restore test" every month as part of DR drills.

⚡ 10 Automation Using Cron Jobs (for mongodump)

```
#!/bin/bash
BACKUP_PATH="/data/backups/$(date +%F)"
mkdir -p $BACKUP_PATH
mongodump --host localhost --out $BACKUP_PATH
find /data/backups/* -mtime +7 -exec rm -rf {} \;
echo "MongoDB backup completed: $BACKUP_PATH"
```

✔ Schedule with:

```
crontab -e
0 2 * * * /scripts/mongo_backup.sh
```

Runs daily at **2 AM** and keeps **7 days of history**.

11 Hands-On Lab Checklist

Exercise	Task	Validation
Full logical backup	mongodump entire DB	Files generated
Restore single DB	mongorestore sales	DB recovered
Physical snapshot	fsyncLock, copy, unlock	No corruption
Atlas PITR restore	Choose timestamp	Cluster restored
Ops Manager backup schedule	Configure & verify	Backups appear in UI
Automated cron job	Daily backup rotation	Verified via logs

12 Best Practices Summary

✔ Always test restores (not just backups)
✔ Store backups offsite or multi-region
✔ Use --gzip for large backups

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

✔ Enable journaling for physical consistency
✔ Encrypt backup files (use gpg or kms)
✔ Automate rotation & retention
✔ Use Atlas/Ops Manager for production-grade PITR
✔ Maintain monthly DR simulation schedule

🔟3️⃣ Troubleshooting Common Issues

Error	Cause	Fix
Error: couldn't connect	Mongod not running	Start service
BSONTooLarge	Document >16MB	Use --archive
Permission denied	Backup dir access	chown or sudo
Oplog missing	No replication	Enable replica set
Restore duplicate key	Existing data conflict	Use --drop

1️⃣4️⃣ Comparison Summary

Backup Method	Speed	PITR	Automation	Use Case
mongodump	Slow	✖	Manual	Small DBs, dev/test
Physical snapshot	Fast	✖	Scripted	Large DBs
Ops Manager	Fast	✔	Full	On-prem enterprise
Atlas Backup	Fast	✔	Fully managed	Cloud clusters

🏁 Summary

By the end of this module, you can:
✔ Perform logical & physical backups
✔ Restore data accurately
✔ Automate backups with cron or Ops Manager
✔ Perform point-in-time recovery
✔ Validate and audit backup integrity
✔ Design a production-grade MongoDB DR plan

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

How to design the database schema itself to ensure scalability, speed, and maintainability.

MongoDB Schema Design & Data Modeling Best Practices
(Embedding vs Referencing, Normalization, and Scaling Patterns)

🎯 **Objective:**
Learn how to design an optimal MongoDB schema that supports large-scale applications, balances read/write performance, and aligns with real-world access patterns.

1️⃣ **SQL vs NoSQL Schema Philosophy**

Aspect	SQL Databases	MongoDB
Data Model	Tables, Rows, Columns	Collections, Documents, Fields
Schema	Rigid (predefined)	Flexible (schema-less or dynamic)
Relationships	Joins	Embedded or Referenced documents
Normalization	High (to reduce redundancy)	Denormalized (to optimize reads)
Scaling	Vertical (bigger servers)	Horizontal (sharding across nodes)

💡 In MongoDB, **data modeling starts from access patterns**, not from normalization rules.

2️⃣ **MongoDB Document Structure Example**

```
{
  "_id": ObjectId("..."),
  "customer_id": 1001,
  "name": "John Doe",
  "email": "john@example.com",
  "orders": [
    {
      "order_id": 9876,
      "order_date": "2025-10-12",
      "total": 125.50,
      "items": [
        { "product": "Laptop", "qty": 1, "price": 1200 },
        { "product": "Mouse", "qty": 2, "price": 25 }
      ]
    }
  ]
}
```

✔ **Embedded model** — customer, orders, and items all within a single document.

🚀 Excellent for fast reads of one customer's data.

3️⃣ **Embedding vs Referencing**

Strategy	Description	Use Case	Example
Embedding	Store related data in the same document	One-to-few relationships	Customer with few orders
Referencing	Use ObjectIDs to link across collections	One-to-many or many-to-many	Products referenced in multiple orders
Hybrid	Combine both	Balance read/write & flexibility	Orders reference Products, but embed order items

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Example: Referencing

```
// orders collection
{
  "_id": ObjectId("..."),
  "customer_id": ObjectId("..."),
  "product_ids": [ObjectId("..."), ObjectId("...")]
}
```

Example: Embedding

```
// customers collection
{
  "_id": ObjectId("..."),
  "name": "Alice",
  "orders": [
    { "order_id": 1001, "amount": 250 },
    { "order_id": 1002, "amount": 400 }
  ]
}
```

4 When to Embed vs Reference

Consideration	Embed When	Reference When
Relationship Type	1-to-few	1-to-many / many-to-many
Data Size	Small subdocuments	Large or unbounded lists
Update Frequency	Read-heavy, few updates	Frequent updates to child docs
Atomicity Required	Yes (single document update)	Not needed
Access Pattern	Usually read together	Read separately

General Rule: **Embed for locality, Reference for scalability**

15 Data Modeling Patterns (Official MongoDB Patterns)

1 Subset Pattern

Store the most recent subset of related data inside the main document.

Example:

```
{
  "customer_id": 101,
  "name": "John",
  "recent_orders": [ { "order_id": 1 }, { "order_id": 2 } ]
}
```

➡ Keep top 5 orders embedded; older ones in separate collection.

2 Bucket Pattern

Group time-series data into buckets (useful for IoT or logs).

```
{
  "_id": "sensor_001_2025-10",
  "sensor_id": "001",
  "month": "2025-10",
  "readings": [
    { "timestamp": "2025-10-01T10:00Z", "value": 15.5 },
    { "timestamp": "2025-10-01T11:00Z", "value": 15.7 }
  ]
}
```

✔ Reduces document count and improves query performance.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

3️⃣ Outlier Pattern

Handle exceptional records separately.

Example:

- Most products have < 10 reviews → embed reviews.
- Some have > 10,000 reviews → store reviews separately.

4️⃣ Extended Reference Pattern

Store reference + essential data for faster reads.

```
{
  "order_id": 123,
  "product_id": ObjectId("..."),
  "product_name": "Laptop",
  "product_price": 900
}
```

✔ Avoids extra query while still maintaining reference.

5️⃣ Tree Pattern

Model hierarchical data (categories, org charts).

Parent Reference Pattern:

```
{ "_id": 3, "name": "Phones", "parent_id": 1 }
```

Materialized Path Pattern:

```
{ "_id": 3, "name": "Phones", "path": "1,2,3" }
```

6️⃣ Denormalization Techniques

MongoDB favors **denormalization** to reduce reads.

But denormalize **only as needed** to avoid data duplication.

Examples:

- Store customer name in the order document.
- Cache product info in order lines.

🔔 Always automate updates using triggers or application logic when data is duplicated.

7️⃣ Schema Design for Sharded Clusters

When sharding, pick a **good shard key**:

✔ Evenly distributes data

✔ Matches query filters

✔ Avoids hotspots

Good shard key example:

```
{ region: 1, customer_id: 1 }
```

Bad shard key example:

```
{ created_date: 1 } // sequential = hotspot
```

🔔 Use hashed sharding for random distribution:

```
sh.shardCollection("sales.orders", { customer_id: "hashed" })
```

8️⃣ Indexing and Schema Together

Design schema **and indexes in parallel**:

- Embed if you query subdocuments often
- Reference if you filter by _id or foreign keys
- Use **compound indexes** on frequent filters + sort fields
- Use **TTL** indexes for expiring data

9️⃣ Schema Evolution

MongoDB's flexible schema allows changes anytime:

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

- Add new fields anytime
- Use $set to update existing docs
- Use schema validation to enforce structure

Example:

```
db.createCollection("users", {
 validator: {
  $jsonSchema: {
   bsonType: "object",
   required: ["name", "email"],
   properties: {
    email: { bsonType: "string", pattern: "^.+@.+$" }
   }
  }
 }
})
```

🔟 Schema Validation & Consistency Checks

Use:

```
db.runCommand({ collMod: "users", validator: { ... } })
```

💡 MongoDB also supports **strict** and **moderate** validation levels.

1️⃣1️⃣ Hands-On Lab Checklist

Exercise	Task	Validation
Create embedded document	Embed orders in customers	Query embedded fields successfully
Create referenced collection	Store product details separately	Use $lookup to join
Apply subset pattern	Keep top 3 orders in each customer doc	Query recent orders
Build tree pattern	Create parent-child collection	Query using path
Enable schema validation	Add JSON schema	Insert invalid data → fails
Shard-aware design	Pick proper shard key	Verify balanced distribution

1️⃣2️⃣ Design Review Checklist

✔ Access pattern–based schema
✔ Choose embedding or referencing wisely
✔ Avoid unbounded arrays
✔ Denormalize only when beneficial
✔ Index design matches query filters
✔ Consider sharding early for scalability
✔ Validate schema in production
✔ Monitor collection/document size

1️⃣3️⃣ Summary

By the end of this module, you can:
✔ Design MongoDB collections for performance and scalability
✔ Choose between embedding and referencing based on use case
✔ Apply advanced data modeling patterns (subset, bucket, outlier, etc.)
✔ Enforce structure with JSON schema validation
✔ Plan schema with future sharding and indexing in mind

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Master how to work with **MongoDB Atlas**, the fully managed cloud service, including cluster provisioning, scaling, automation, and security.

☁ Module 32 — MongoDB Atlas Administration & Cloud Automation

(Clusters, Backup, and Security Policies)

🎯 Objective:

Gain practical, step-by-step expertise in **MongoDB Atlas** — from creating clusters to automating backups, enabling security, and integrating with CI/CD.

1️⃣ What Is MongoDB Atlas?

MongoDB Atlas is a **cloud-hosted, fully managed** version of MongoDB that automates:

- Cluster setup and scaling
- Backups and monitoring
- Security and compliance
- Global replication and sharding
- Integration with AWS, Azure, GCP

🔑 Key Features:

✔ Automated provisioning & patching
✔ Auto-scaling for compute/storage
✔ Global cluster deployment
✔ Built-in performance metrics & alerts
✔ Integrated encryption, auditing, and access control

2️⃣ Creating a Cluster (Step-by-Step)

◆ **Step 1: Sign In**

Go to 👉 https://cloud.mongodb.com
Create a free or paid Atlas account.

◆ **Step 2: Create a Project**

- Name your project (e.g., Prod-DBA-Cluster)
- Assign team members & roles

◆ **Step 3: Build a Cluster**

- Choose **Cloud Provider:** AWS / Azure / GCP
- Choose **Region:** Closest to application users
- Select **Cluster Tier:** M0 (free), M10–M700 (paid)
- Choose **MongoDB Version:** Always latest stable

◆ **Step 4: Configure Security**

- Add **IP whitelist** (your local IP or VPC CIDR)
- Create **Database User**
- username: admin
- password: <secure_password>
- role: atlasAdmin

◆ **Step 5: Connect to Cluster**

Copy your connection string:

```
mongodb+srv://admin:<password>@cluster0.abcd.mongodb.net/test
```

⚙ 3️⃣ Cluster Types in Atlas

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Cluster Type	Description	Use Case
Shared Cluster (M0–M5)	Multi-tenant, free or low-cost	Dev/Test
Dedicated Cluster (M10+)	Fully isolated hardware	Production
Serverless Instance	Pay-per-query model	Event-driven workloads
Multi-Cloud Cluster	Data distributed across providers	Compliance, global availability

4 Global Clusters & Zone Sharding

Atlas supports **Global Clusters** — data localized by geography using **Zone Sharding**.

Example:

```
sh.enableSharding("salesDB")
sh.shardCollection("salesDB.orders", { region: 1 })
```

Automatically directs users in “APAC” region to APAC data nodes.

5 Backup & Snapshot Management

Atlas automates backups using **Continuous Cloud Backups** or **Snapshot Backups**.

Continuous Backup

- Near-real-time backup
- Point-in-time recovery
- Storage in Atlas-managed cloud bucket

Snapshot Backup

- Periodic snapshot every X hours/days
- Retention policy configurable
- Option to download snapshot

Example Restore:

- From Atlas UI → “Backup” → “Restore”
- Choose point in time or snapshot
- Restore to existing or new cluster

CLI Example:

```
atlas backups restore create \
 --clusterName "ProdCluster" \
 --snapshotId 64a88d9b6789e543210aaaab
```

6 Security & Access Control in Atlas

Authentication Options:

Type	Description
SCRAM	Default username/password
X.509	Certificate-based auth
LDAP / SAML	Enterprise directory integration
Federated Identity (SSO)	Okta, Azure AD, etc.

Network Security

✔ IP Whitelisting
✔ VPC Peering / Private Link
✔ TLS/SSL enforced
✔ End-to-end encryption in transit and at rest

Example: Private Endpoint in AWS

```
aws ec2 create-vpc-endpoint --vpc-id vpc-12345 --service-name com.amazonaws.vpce.atlas
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🔐 Data Encryption

- **At Rest:** AES-256 via cloud KMS (AWS/GCP/Azure)
- **In Transit:** TLS 1.2+
- **Field-Level Encryption (FLE):**

```
encryptedField: {
  $encrypt: {
    keyId: UUID("..."),
    algorithm: "AEAD_AES_256_CBC_HMAC_SHA_512-Deterministic",
    value: "SensitiveData"
  }
}
```

📈 Monitoring and Performance

Atlas integrates built-in monitoring:

Metric	Purpose
Ops/sec	CRUD throughput
Connections	Connection pool status
Disk IOPS	Disk performance
CPU / Memory	Hardware utilization
Index Usage	Identify unused indexes

💡 Use **Performance Advisor** in Atlas to get auto-suggested indexes:
"Your query scanned 500k docs but used no index — create {order_date: 1}"

8️⃣ Autoscaling & Performance Optimization

Atlas can **auto-scale** cluster tiers based on usage.

- Enable **Auto-Scaling Storage** and **Cluster Tier Scaling**
- Configure thresholds for CPU/IOPS
- Review in UI: *Cluster → Settings → Auto-Scaling*

💡 Use **Serverless Atlas** for unpredictable workloads.

9️⃣ Atlas CLI & Automation

Install Atlas CLI:

```
brew install mongodb-atlas-cli
```

Authenticate:

```
atlas auth login
```

Create a cluster:

```
atlas clusters create ProdCluster --region US_EAST_1 --provider AWS --tier M10
```

Scale a cluster:

```
atlas clusters update ProdCluster --tier M30
```

Delete a cluster:

```
atlas clusters delete ProdCluster
```

🔟 Infrastructure as Code (IaC) Integration

Atlas integrates with:

- **Terraform Provider (official)**

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

- **AWS CloudFormation**
- **Azure Resource Manager (ARM)**
- **GitHub Actions / Jenkins**

Terraform Example:

```
resource "mongodbatlas_cluster" "prod" {
  project_id = var.project_id
  name       = "ProdCluster"
  provider_name = "AWS"
  region_name   = "US_EAST_1"
  cluster_type  = "REPLICASET"
  mongo_db_major_version = "7.0"
  auto_scaling_disk_gb_enabled = true
}
```

🔟1️⃣ Atlas Alerts and Notifications

Set up alerts for:

✔ High CPU usage

✔ Low disk space

✔ Slow queries

✔ Replica lag

✔ Unauthorized access attempts

Atlas → **Alerts → Create Alert → Conditions**

Example:

Trigger alert when "Disk IOPS > 80% for 5 minutes"

Send email or webhook to Slack/MS Teams

1️⃣2️⃣ Auditing & Compliance

Atlas meets major compliance standards:

- ISO 27001
- SOC 2 Type II
- HIPAA
- GDPR
- FedRAMP (for government workloads)

🔔 Enable **Database Auditing** for compliance:

```
atlas auditing enable --clusterName ProdCluster
```

1️⃣3️⃣ Hands-On Lab Checklist

Exercise	Task	Validation
Create Atlas Cluster	Build on AWS or Azure	Cluster appears as ACTIVE
Connect via SRV URI	Test with mongosh	Connection successful
Configure IP Whitelist	Add your public IP	Connection restricted
Enable Auto-Scaling	Turn ON in settings	Cluster scales automatically
Setup Continuous Backup	Enable in UI	Backup snapshot visible
Create Alert	Configure for CPU > 80%	Notification received
Enable Field Encryption	Encrypt field data	Query returns ciphertext

1️⃣4️⃣ Best Practices Summary

✔ Always use **dedicated clusters** for production

✔ Enable **auto-scaling** and **continuous backup**

✔ Use **VPC peering** for private traffic

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

✔ Configure **IAM / SSO** for enterprise security
✔ Monitor with **Atlas Metrics + Alerts**
✔ Automate via **CLI, Terraform, or API**
✔ Audit regularly for compliance

🏁 Summary
By completing this module, you can:
✔ Build and manage clusters in MongoDB Atlas
✔ Implement secure network and access controls
✔ Automate scaling, backups, and alerting
✔ Integrate Atlas into DevOps pipelines
✔ Maintain compliance and security posture

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

How to automate deployments, CI/CD pipelines, and manage infrastructure using **Terraform, Jenkins, and GitHub Actions** for MongoDB (both on-prem and Atlas).

MongoDB Automation, CI/CD Integration, and Infrastructure-as-Code (Terraform, GitHub Actions, Jenkins)

🎯 **Objective:**
Implement fully automated MongoDB environments — from cluster provisioning and schema migrations to backups and monitoring — using CI/CD pipelines and Infrastructure-as-Code tools.

1️⃣ **What Is MongoDB Automation?**
Automation = using code, scripts, and pipelines to handle:

- Cluster deployment
- Configuration changes
- Backups & restores
- Index creation
- Schema validation
- Patches & scaling

✔ Reduces manual effort
✔ Improves reliability
✔ Enables reproducibility

2️⃣ **Key Automation Tools Overview**

Tool	Purpose	Typical Use
MongoDB Atlas CLI / API	Manage Atlas clusters	Provision, backup, alert setup
Terraform	IaC provisioning	Create clusters, users, backups
Ansible	Configuration management	Install MongoDB, edit configs
Jenkins / GitHub Actions	CI/CD automation	Deploy, test, migrate
mongosh / Shell Scripts	Quick automation	Admin scripts, health checks

3️⃣ **Infrastructure-as-Code (IaC) with Terraform**
Terraform manages MongoDB infrastructure declaratively.
 Step 1: Install Terraform
brew install terraform
 Step 2: Configure MongoDB Atlas Provider
Create a file provider.tf:

```
terraform {
  required_providers {
    mongodbatlas = {
      source  = "mongodb/mongodbatlas"
      version = "~> 1.16.0"
    }
  }
}
provider "mongodbatlas" {
  public_key  = var.public_key
  private_key = var.private_key
}
```

 Step 3: Create Cluster

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
resource "mongodbatlas_cluster" "prod" {
  project_id            = var.project_id
  name                  = "ProdCluster"
  provider_name         = "AWS"
  region_name           = "US_EAST_1"
  cluster_type          = "REPLICASET"
  mongo_db_major_version  = "7.0"
  auto_scaling_disk_gb_enabled = true
}
```

Step 4: Apply

```
terraform init
terraform plan
terraform apply
```

✔ Terraform provisions your Atlas cluster automatically.

MongoDB Atlas API Automation

Use REST API to control Atlas operations.

Example — **Get all clusters:**

```
curl -u "<public_key>:<private_key>" \
 --digest \
 --request GET \
 "https://cloud.mongodb.com/api/atlas/v1.0/groups/<project_id>/clusters"
```

Example — **Pause Cluster:**

```
curl -u "<public_key>:<private_key>" \
 --digest \
 --request PATCH \
 --header "Content-Type: application/json" \
 --data '{ "paused" : true }' \
 "https://cloud.mongodb.com/api/atlas/v1.0/groups/<project_id>/clusters/ProdCluster"
```

Configuration Automation with Ansible (On-Prem or VM)

Example Playbook — mongodb-install.yml

```
- hosts: dbservers
  become: yes
  tasks:
    - name: Add MongoDB repo
      yum_repository:
        name: mongodb-org
        description: MongoDB Repository
        baseurl: https://repo.mongodb.org/yum/redhat/$releasever/mongodb-org/7.0/x86_64/
        gpgcheck: yes
        enabled: yes
        gpgkey: https://www.mongodb.org/static/pgp/server-7.0.asc

    - name: Install MongoDB
      yum:
        name: mongodb-org
        state: present

    - name: Start MongoDB


service:
        name: mongod
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
    state: started
    enabled: true
```

Run:

```
ansible-playbook mongodb-install.yml
```

✔ MongoDB gets installed and started across all target nodes automatically.

6 CI/CD Integration: MongoDB + Jenkins

Jenkins Pipeline Example (Jenkinsfile)

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps {
        git 'https://github.com/your-org/mongo-infra.git'
      }
    }
    stage('Provision Cluster') {
      steps {
        sh 'terraform init && terraform apply -auto-approve'
      }
    }
    stage('Deploy Schema Updates') {
      steps {
        sh 'mongosh --file scripts/update-schema.js'
      }
    }
    stage('Run Tests') {
      steps {
        sh 'npm run test:integration'
      }
    }
    stage('Notify') {
      steps {
        slackSend(channel: '#dba-alerts', message: 'MongoDB deployment complete ✔')
      }
    }
  }
}
```

✔ Jenkins automates provisioning, schema updates, and notifications.

7 MongoDB + GitHub Actions Automation

Create .github/workflows/mongodb-deploy.yml:

```
name: MongoDB Atlas Deploy
on:
  push:
    branches: [ main ]
```

```
jobs:
  deploy:
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
  runs-on: ubuntu-latest
  steps:
    - name: Checkout
      uses: actions/checkout@v3

    - name: Setup Terraform
      uses: hashicorp/setup-terraform@v2
      with:
        terraform_version: 1.8.0

    - name: Terraform Init & Apply
      run: |
        terraform init
        terraform plan
        terraform apply -auto-approve

    - name: Run Schema Migration
      run: mongosh --file migrations/schema_update.js

    - name: Send Notification
      uses: slackapi/slack-github-action@v1.23.0
      with:
        channel-id: "C02MONGOALERTS"
        slack-message: "✅ MongoDB schema update deployed successfully!"
```

🔄 Integrates MongoDB IaC automation with version control — any git push triggers a full deployment.

8️⃣ Common Automated Tasks

Task	Description	Example
Cluster Provisioning	Terraform or API	terraform apply
Backup Automation	Atlas Backup CLI	atlas backups create
Performance Monitoring	CLI or Prometheus exporter	mongostat, mongotop
Index Creation	Scripted using mongosh	createIndex()
Schema Validation	Automated JSON Schema	db.createCollection(...validator...)
Scaling	API/CLI driven	atlas clusters update --tier M30

9️⃣ MongoDB Automation Scripts

Backup Script Example (bash):

```
#!/bin/bash
DATE=$(date +%F)
mongodump --uri="mongodb+srv://admin:pwd@cluster0.abcd.mongodb.net/" --out /backup/mongo_$DATE
find /backup -type d -mtime +7 -exec rm -rf {} \;
```

Restore Script Example:

```
mongorestore --uri="mongodb+srv://admin:pwd@cluster0.abcd.mongodb.net/" /backup/mongo_2025-10-14
```

Schedule via cron:

```
0 1 * * * /scripts/mongo_backup.sh
```

📦 🔟 Integration with Monitoring Tools

Integrate MongoDB with DevOps Observability Stack:

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

- **Prometheus + Grafana** dashboards
- **Datadog / New Relic** integrations
- **Opsgenie / PagerDuty** for alerts

Atlas has native webhook support for alerting.

🧪 Hands-On Lab Checklist

Exercise	Task	Validation
Terraform IaC	Create Atlas cluster	Cluster appears “Active”
Ansible Config	Install MongoDB	mongod service running
Jenkins Pipeline	Automate schema deploy	Build succeeds in Jenkins
GitHub Action	Auto-provision cluster on push	Job runs successfully
Backup Script	Run automated mongodump	Backup folder created
Restore Test	Restore from dump	Data validated
Alerts	Trigger Atlas alert	Slack/email notification

🔐 Best Practices Summary

✔ Store IaC in version control (Git)
✔ Separate production & non-prod pipelines
✔ Use **Terraform + Jenkins** for infra, **GitHub Actions** for app integration
✔ Always use encrypted secrets (Vault, GitHub Secrets)
✔ Automate backups and cluster scaling
✔ Validate deployments via test stages
✔ Apply rollback logic for failed releases

🏁 Summary

By the end of this module, you can:
✔ Automate MongoDB provisioning with Terraform
✔ Integrate MongoDB into CI/CD workflows
✔ Use APIs, CLI, and scripts for operational automation
✔ Deploy and validate schema changes automatically
✔ Build DevOps-ready MongoDB environments

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

How to build a **complete monitoring and observability system** around MongoDB using **Atlas Metrics**, **Prometheus**, **Grafana**, and **Audit/Event Integrations**.

MongoDB Cloud Monitoring, Alerting & Observability (Atlas Metrics, Prometheus, Grafana, and Auditing Integration)

🎯 Objective

By the end of this module, you'll be able to:

- Configure MongoDB **monitoring and alerts** (both Atlas & on-prem).
- Integrate **Prometheus & Grafana** for metrics visualization.
- Enable **MongoDB audit logging** for compliance & security.
- Centralize logs using **ELK / OpenSearch** stacks.
- Build **end-to-end observability dashboards** for clusters and databases.

1️⃣ Understanding MongoDB Observability

MongoDB observability = combination of:

- **Metrics:** performance indicators (CPU, IOPS, locks, memory).
- **Logs:** events, slow queries, errors.
- **Traces:** execution paths (optional with APM tools).
- **Alerts:** proactive notifications for anomalies.

Goal → **Detect, diagnose, and prevent performance issues** before users notice.

2️⃣ MongoDB Atlas Monitoring Overview

If you're using **MongoDB Atlas**, monitoring is built-in.

🔍 **Atlas Dashboard Includes:**

- **Cluster metrics:** CPU, memory, IOPS, connections, cache hit ratio.
- **DB metrics:** ops/sec, slow queries, index usage, replication lag.
- **Atlas alerts:** integrated via email, Slack, PagerDuty, Opsgenie, etc.

⚙️ **Configure Alerts in Atlas UI**

1. Go to **Project → Alerts → Add Alert**
2. Choose a **Metric (e.g., Connections > 90%)**
3. Define a **Threshold & Duration**
4. Set **Notifications → Slack / Email / Webhook**
5. Save & Test

✅ Atlas automatically tracks and triggers alerts in real time.

3️⃣ Monitoring On-Prem MongoDB (Open Source)

For **self-managed clusters**, we can use:

- **MongoDB Exporter** → for Prometheus metrics.
- **mongostat / mongotop** → quick CLI metrics.
- **Grafana** → for dashboard visualization.
- **ELK / OpenSearch** → for log centralization.

4️⃣ Prometheus Integration for MongoDB

🔧 **Step 1: Install Prometheus**

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

sudo apt install prometheus -y

🔧 **Step 2: Configure MongoDB Exporter**

Download the exporter:

```
wget https://github.com/percona/mongodb_exporter/releases/download/v0.40.0/mongodb_exporter-0.40.0.linux-amd64.tar.gz
tar -xvzf mongodb_exporter-0.40.0.linux-amd64.tar.gz
cd mongodb_exporter
```

Run exporter:

```
./mongodb_exporter --mongodb.uri="mongodb://admin:password@localhost:27017"
```

🔧 **Step 3: Configure Prometheus to scrape it**

Edit /etc/prometheus/prometheus.yml:

```
scrape_configs:
 - job_name: 'mongodb'
   static_configs:
    - targets: ['localhost:9216']
```

🔧 **Step 4: Restart Prometheus**

```
sudo systemctl restart prometheus
```

✔ Prometheus now collects metrics from MongoDB every scrape interval (default 15s).

Visualizing MongoDB Metrics in Grafana

 Step 1: Install Grafana

```
sudo apt install -y grafana
sudo systemctl enable grafana-server --now
```

 Step 2: Add Prometheus as Data Source

- Go to **Settings → Data Sources → Add Prometheus**
- URL: http://localhost:9090

 Step 3: Import MongoDB Dashboard

You can use pre-built dashboards such as:
Percona MongoDB Dashboard ID: 2583

 Step 4: Monitor Key Metrics

- Ops/sec
- Query execution times
- Cache hit ratio
- Replication lag
- Index performance
- Connection count
- Page faults

✔ Grafana visualizes MongoDB cluster health beautifully in real time.

Logs & Audit Trail Management

🔍 **MongoDB Log Types:**

1. **mongod.log** — core server logs (queries, errors, slow ops).
2. **Audit logs** — user actions, authentication, schema changes.
3. **Profiler logs** — detailed query performance.

⚙ **Enable Profiling:**

```
db.setProfilingLevel(1, { slowms: 100 })
```

⚙ **Enable Audit Logging:**

Add to /etc/mongod.conf:

```
auditLog:
 destination: file
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
  format: JSON
  path: /var/log/mongodb/auditLog.json
  filter: '{ atype: { $in: ["authenticate", "createUser", "dropUser", "grantRolesToUser"] } }'
```

Restart service:

```
sudo systemctl restart mongod
```

✔ Now MongoDB logs all critical access & admin actions — ideal for compliance.

7️⃣ Centralized Logging with ELK / OpenSearch

Stack Components:

- **Filebeat** → collects MongoDB logs
- **Logstash** → processes & filters
- **Elasticsearch / OpenSearch** → stores logs
- **Kibana / OpenSearch Dashboards** → visualize

Example Filebeat config:

```
filebeat.inputs:
  - type: log
    paths:
      - /var/log/mongodb/mongod.log
      - /var/log/mongodb/auditLog.json
output.logstash:
  hosts: ["localhost:5044"]
```

🔍 You can create filters to highlight slow queries, auth failures, or replication events.

8️⃣ Integration with Alert Systems

Tool	Integration Type	Example
Slack	Webhook alerts	Atlas / Prometheus / Grafana
PagerDuty	Incident escalation	Atlas webhook
Opsgenie	Alert management	Atlas webhook
ServiceNow	ITSM integration	via Atlas or Prometheus bridge
Email / SMS	Native	Atlas or Alertmanager

Example Prometheus Alert Rule (alerts.yml):

```
groups:
- name: MongoDBAlerts
  rules:
  - alert: HighConnections
    expr: mongodb_connections_current > 1000
    for: 2m
    labels:
      severity: warning
    annotations:
      summary: "Too many MongoDB connections"
      description: "Connection count exceeds 1000 for more than 2 minutes"
```

9️⃣ Atlas API Monitoring & Automation

Use Atlas APIs to fetch real-time metrics:

```
curl -u "<public_key>:<private_key>" --digest \
--request GET \
"https://cloud.mongodb.com/api/atlas/v1.0/groups/<project_id>/processes?pretty=true"
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Fetch metrics for a specific node:

```
curl -u "<public_key>:<private_key>" --digest \
"https://cloud.mongodb.com/api/atlas/v1.0/groups/<project_id>/processes/<hostname>:27017/measurements?granularity=PT1M&period=PT1H"
```

✔ Useful for integrating Atlas metrics with Prometheus or Datadog.

🔟 Hands-On Lab Checklist

Exercise	Task	Validation
Atlas Monitoring	Enable alerts for high CPU	Notification received
Prometheus Setup	Install & configure exporter	Prometheus shows metrics
Grafana Dashboard	Import MongoDB dashboard	Visual metrics visible
Enable Profiling	Capture slow queries	Queries appear in log
Audit Logging	Track admin actions	Audit log entries verified
Log Forwarding	Filebeat → ELK	Logs visible in Kibana
Alert Rule	Trigger custom alert	Alert fired via Slack or Email

1️⃣1️⃣ Key Metrics to Monitor

Category	Metric	Description
Performance	ops/sec	Read/write throughput
Replication	replicationLag	Delay between nodes
Connections	currentConnections	Open client connections
Memory	residentMemory	RAM used by MongoDB
Storage	dataSize / indexSize	Space usage
Locks	lockRatio	% of time DB is locked
Cache	cacheHitRatio	WiredTiger cache efficiency
Errors	assertions, slow queries	Operational stability

1️⃣2️⃣ Best Practices Summary

✔ Monitor at **database, OS, and application layers**
✔ Use **Prometheus + Grafana** for open-source setups
✔ Use **Atlas built-in monitoring** for cloud
✔ Enable **slow query profiling**
✔ Keep **audit logs** for compliance (SOX, GDPR, HIPAA)
✔ Rotate and archive logs periodically
✔ Integrate alerting with **Slack or Opsgenie**
✔ Automate metric collection with **API scripts**

🏁 Summary

By completing this module, you can:
✔ Monitor MongoDB health using Atlas or Prometheus
✔ Visualize metrics in Grafana dashboards
✔ Configure and validate custom alerts

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

✔ Audit and centralize MongoDB logs for compliance
✔ Build a complete observability ecosystem

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Most **important and expert-level MongoDB DBA modules** — where true database performance mastery happens.
This module focuses on **analyzing, profiling, and optimizing MongoDB performance** through indexes, query plans, schema tuning, and cache efficiency.

MongoDB Advanced Performance Optimization & Query Profiling (Indexes, Query Plans, Caching, and Schema Tuning)

🎯 Objective

By the end of this module, you will:

- Identify slow queries and tune them using query plans and indexes.
- Analyze execution paths using **explain()** and **profiler**.
- Optimize schema design for performance and scalability.
- Tune WiredTiger cache and OS memory utilization.
- Apply real-world performance optimization best practices.

1️⃣ Understanding MongoDB Performance Layers

MongoDB performance depends on 4 key layers:

Layer	Focus Area	Optimization Example
Application	Query structure	Use projections and filters properly
Database	Indexes & schema	Create compound indexes
Storage Engine (WiredTiger)	Cache, compression	Tune cacheSizeGB
Infrastructure	CPU, Disk IOPS, Memory	Optimize hardware or cloud tier

✔ Each layer must be optimized holistically for peak throughput.

🔍2️⃣ Query Profiling & Diagnostics

Enable Profiling:

```
db.setProfilingLevel(1, { slowms: 100 })
```

- 0 → Off
- 1 → Logs slow queries (> slowms)
- 2 → Logs all queries

Check profiling results:

```
db.system.profile.find().sort({ ts: -1 }).limit(5).pretty()
```

Fields:

- ns → Namespace (DB + collection)
- op → Operation type (query, update)
- millis → Time taken
- planSummary → Index or COLLSCAN

✔ Identify slow queries easily.

3️⃣ Query Analysis using explain()

The explain() method shows **how MongoDB executes a query**.
Example:

```
db.orders.find({ customerId: 123 }).explain("executionStats")
```

Key fields:

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Field	Meaning
queryPlanner.winningPlan.stage	How query is executed (IXSCAN, COLLSCAN)
executionStats.executionTimeMillis	Total time
executionStats.nReturned	Documents returned
executionStats.totalKeysExamined	Index keys scanned
executionStats.totalDocsExamined	Documents scanned

If **totalDocsExamined >> nReturned**, query needs optimization.

🔧 Indexing Strategy & Best Practices

Create Index:

```
db.orders.createIndex({ customerId: 1 })
```

Compound Index:

```
db.orders.createIndex({ customerId: 1, orderDate: -1 })
```

Unique Index:

```
db.users.createIndex({ email: 1 }, { unique: true })
```

Covered Query:

A query that can be satisfied **entirely by an index** (no document fetch).

Example:

```
db.orders.find({ customerId: 101 }, { orderDate: 1, _id: 0 })
```

✔ If both filter & projection fields exist in the index → query is *covered*.

⚡ Common Indexing Tips

Scenario	Recommended Index
Range + equality filter	Equality fields first, range last ({user: 1, date: -1})
Sorting	Create index with same sort order
Frequent updates	Avoid too many indexes (writes slower)
Large arrays	Use **multikey indexes**
Text search	Use text indexes
Geo data	Use 2dsphere indexes

🧠 Using hint() for Query Plan Control

Force MongoDB to use a specific index:

```
db.orders.find({ customerId: 123 }).hint({ customerId: 1 })
```

Use this sparingly — it overrides the optimizer.

📊 Monitoring Index Usage

List all indexes:

```
db.orders.getIndexes()
Check index usage stats:
db.orders.aggregate([
  { $indexStats: {} }
])
```

Output example:

```
{
  "name": "customerId_1",
  "accesses": { "ops": 421 },
  "since": ISODate("2025-01-01T00:00:00Z")
}
```

✔ Identify unused or underutilized indexes.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

7️⃣ Schema Design Optimization

Schema should align with access patterns.

Embedding (One-to-Few)

```
{
  _id: 1,
  name: "John",
  addresses: [
    { city: "NYC", zip: "10001" },
    { city: "Boston", zip: "02101" }
  ]
}
```

✓ Pros: Fast reads, single document.

✗ Cons: Large document growth risk.

Referencing (One-to-Many)

users: { _id: 1, name: "John" }

orders: { user_id: 1, total: 120 }

✓ Pros: Independent growth, small documents.

✗ Cons: Requires $lookup for joins.

📈 **Rule of Thumb:**

- Use **embedding** for "few" related items (≤10s).
- Use **referencing** for "many" related items (≥100s).

8️⃣ WiredTiger Cache Optimization

Default WiredTiger cache = 50% of system RAM.

View cache usage:

```
db.serverStatus().wiredTiger.cache
```

Tune in /etc/mongod.conf:

```
storage:
  wiredTiger:
    engineConfig:
      cacheSizeGB: 4
```

✓ Adjust cache for optimal performance (usually 50–75% of available RAM).

9️⃣ Storage & Disk Optimization

- Use **SSD** for data & journal directories.
- Separate **data and logs** on different volumes.
- Mount with **noatime** to reduce disk I/O.
- Enable **journaling** for crash recovery.
- Run **compact** occasionally on low-usage windows:
- db.runCommand({ compact: "orders" })

🔟 Connection & Thread Tuning

- Use **connection pooling** in app (via driver).
- Monitor connections:
- db.serverStatus().connections

- Increase **ulimit -n** for concurrent connections if needed.
- Adjust net.maxIncomingConnections in config.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

11 Aggregation Pipeline Performance

Use $match and $project early in the pipeline.

Bad:

```
db.orders.aggregate([
 { $group: { _id: "$customerId", total: { $sum: "$total" } } },
 { $match: { total: { $gt: 100 } } }
])
```

Good:

```
db.orders.aggregate([
 { $match: { total: { $gt: 100 } } },
 { $group: { _id: "$customerId", total: { $sum: "$total" } } }
])
```

✔ Reduces document volume early → improves speed.

12 Performance Diagnostic Tools

Tool	Purpose
mongostat	Realtime ops stats
mongotop	Time spent per collection
explain()	Query execution analysis
serverStatus()	Server-level metrics
Atlas Performance Advisor	Suggests indexes
Atlas Profiler	Slow query logs
db.currentOp()	Active queries/locks

13 Hands-On Lab Checklist

Exercise	Task	Validation
Profiling	Enable and inspect slow queries	Profiler shows entries
Explain Plan	Analyze query plan	Identify COLLSCAN
Index Optimization	Create proper index	IXSCAN appears
Compound Index	Test range queries	Execution time reduced
Cache Tuning	Adjust WiredTiger cache	Memory usage stable
Aggregation Rewrite	Optimize pipeline	Faster runtime
Schema Review	Refactor embedding/referencing	Improved read speed

14 Best Practices Summary

✔ Use **compound indexes** for multi-field queries
✔ Avoid **unnecessary indexes** (hurts writes)
✔ Profile regularly and prune old indexes
✔ Embed small, static relationships
✔ Reference large or frequently updated subdocs
✔ Keep working set in RAM (cache efficiency)
✔ Analyze **slow queries weekly** using profiler logs
✔ Monitor **index fragmentation and cache hits**

Summary

After mastering this module, you can:
✔ Identify and fix slow queries

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

✔ Design indexes strategically
✔ Tune cache and I/O performance
✔ Optimize schemas for real-world workloads
✔ Build a repeatable performance-tuning framework

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Automation, Scripting, and DevOps Integration (CI/CD, Ansible, and Monitoring Pipelines)

🎯 **Objective** Automate MongoDB deployments, configurations, monitoring, and backups using DevOps tools such as **Ansible**, **Jenkins**, and **GitHub Actions** — enabling consistent, repeatable, and error-free database management workflows.

◆ 1. MongoDB Automation Concepts

- Automation ensures repeatable setup across environments (dev/test/prod).
- Key automation areas:
  - Installation and configuration management
  - Backup and restore automation
  - Replica set or sharded cluster scaling
  - Monitoring and alert integration
  - Patching and upgrades

Benefits:

- Reduces manual errors
- Improves consistency
- Enables CI/CD pipelines for database changes

◆ 2. Automation Tools Overview

Tool	Purpose	Example Usage
Ansible	Declarative configuration management	Automate setup of MongoDB nodes and clusters
Terraform	Infrastructure provisioning	Spin up servers and networking in AWS/Azure/GCP
Jenkins / GitHub Actions	CI/CD automation	Trigger DB backup, linting, or restore validation
Prometheus + Grafana	Monitoring automation	Visualize MongoDB metrics
Python / Bash Scripts	Task automation	Data archival, log rotation, and status checks

◆ 3. Ansible MongoDB Playbook Example

```
---
- name: MongoDB Replica Set Deployment
  hosts: mongo_nodes
  become: yes
  tasks:
    - name: Install MongoDB
      yum:
        name: mongodb-org
        state: present

    - name: Configure mongod.conf
      template:
        src: templates/mongod.conf.j2
        dest: /etc/mongod.conf
      notify:
        - restart mongod

  handlers:
    - name: restart mongod
      service:
        name: mongod
        state: restarted
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Template Variables:

- replicaSetName: rs0
- bindIp: 0.0.0.0
- dbPath: /data/db

◆ **4. Jenkins / GitHub Actions CI/CD Integration**

Example MongoDB Backup Pipeline (Jenkinsfile):

```
pipeline {
  agent any
  stages {
    stage('Backup MongoDB') {
      steps {
        sh 'mongodump --uri="mongodb://admin:pass@prod-db:27017" --out=/backups/$(date +%F)'
      }
    }
    stage('Upload to S3') {
      steps {
        sh 'aws s3 cp /backups/$(date +%F) s3://mongo-backups/'
      }
    }
  }
}
```

GitHub Actions Equivalent (.github/workflows/mongo-backup.yml):

```
name: MongoDB Backup Workflow
on:
  schedule:
    - cron: '0 3 * * *'   # Daily at 3 AM
jobs:
  backup:
    runs-on: ubuntu-latest
    steps:
      - name: Run MongoDB Backup
        run: mongodump --uri="${{ secrets.MONGO_URI }}" --out=/tmp/mongobackup
      - name: Upload to S3
        run: aws s3 cp /tmp/mongobackup s3://mongo-backups/
```

◆ **5. Automation for Monitoring & Alerting**

Prometheus Exporter Setup:

```
sudo apt install prometheus mongodb-exporter
systemctl enable mongodb-exporter
```

Grafana Dashboards:

- Import MongoDB dashboard from Grafana Marketplace
- Connect Prometheus as a data source
- Set alerts for slow queries or replication lag

◆ **6. Backup Automation Script Example**

```
#!/bin/bash
DATE=$(date +%F)
DEST="/backups/mongodb/$DATE"
mkdir -p $DEST
mongodump --uri="mongodb://backupuser:pass@localhost:27017" --out=$DEST
tar -czf $DEST.tar.gz $DEST
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
aws s3 cp $DEST.tar.gz s3://mongo-backups/
find /backups/mongodb -type f -mtime +7 -delete
```

✅ **Schedule with Cron:**

```
0 2 * * * /usr/local/bin/mongo-backup.sh >> /var/log/mongo-backup.log 2>&1
```

◆ **7. DevOps Validation Steps**

Task	Command / Tool	Expected Result
Check automation logs	journalctl -u mongod	No config or permission errors
Verify backup success	ls /backups/mongodb/	Backup files created daily
Verify restore integrity	mongorestore --dryRun	No data corruption
CI/CD job status	Jenkins dashboard / GitHub Actions logs	All steps succeeded
Infrastructure validation	ansible-inventory --list	Correct host mappings

◆ **8. Hands-On Lab Checklist**

Exercise	Description	Validation
Lab 1	Deploy MongoDB using Ansible	Confirm running instances and config
Lab 2	Schedule daily automated backups	Check cron job and logs
Lab 3	Integrate MongoDB exporter with Prometheus	View metrics in Prometheus
Lab 4	Create CI/CD job for automated backup	Validate execution via Jenkins
Lab 5	Automate restore testing in staging	Compare data counts and indexes

□ **Summary**

- Automate MongoDB operations with Ansible and CI/CD pipelines
- Ensure monitoring and backup tasks are repeatable and validated
- Integrate with DevOps tools for proactive database management
- Validate all jobs and logs to maintain database reliability

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

MongoDB Performance Tuning & Query Optimization

This module focuses on how to **analyze**, **optimize**, and **tune MongoDB performance** in real-world environments.
You'll learn how to handle **slow queries**, **index strategies**, **hardware tuning**, and **diagnostics**.

1 Understanding MongoDB Performance Bottlenecks

MongoDB performance issues usually fall into **four categories**:

Category	Common Causes	Focus Area
Query Inefficiency	Missing indexes, poor filters, large scans	Indexing & Query Optimization
Write Latency	Journaling, replication lag, small batch inserts	Write Concern & Batching
Hardware Constraints	Disk I/O bottleneck, low RAM, slow CPU	Server Tuning
Configuration Issues	Wrong cache size, small oplog	System Configuration

2 Tools for Performance Analysis

Tool	Command	Purpose
explain()	db.collection.find(query).explain("executionStats")	Shows how query executes
profiler	db.setProfilingLevel(2)	Logs slow queries
serverStatus	db.serverStatus()	Server health metrics
top	mongotop	Shows time spent per collection
stat	mongostat	Real-time DB performance stats
Atlas Performance Advisor	(Atlas UI)	Recommends indexes automatically

3 Query Optimization Techniques

Step 1 — Identify Slow Queries

Enable profiler (for queries slower than 100ms):

```
db.setProfilingLevel(1, { slowms: 100 })
```

Check slow queries:

```
db.system.profile.find().sort({ ts: -1 }).limit(5).pretty()
```

Step 2 — Analyze Query Execution Plan

```
db.orders.find({ customer_id: 123 }).explain("executionStats")
```

Focus on:

- stage (IXSCAN = uses index, COLLSCAN = full collection scan)
- nReturned
- executionTimeMillis
- totalDocsExamined

Step 3 — Create Proper Indexes

Indexes make lookups, filters, and sorts efficient.

```
db.orders.createIndex({ customer_id: 1 })
```

Compound index example:

```
db.orders.createIndex({ customer_id: 1, order_date: -1 })
```

Partial index example:

```
db.orders.createIndex({ status: 1 }, { partialFilterExpression: { status: "ACTIVE" } })
```

Step 4 — Use Projection to Reduce Data Transfer

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

db.customers.find({ city: "NYC" }, { name: 1, email: 1, _id: 0 })
Only returns specific fields → reduces I/O.

Step 5 — Optimize Aggregations

- Use $match and $project early in the pipeline.
- Avoid $lookup unless necessary.
- Use $merge instead of writing large temporary data.
- Index the join fields.

Example:

```
db.sales.aggregate([
 { $match: { region: "US" } },
 { $group: { _id: "$customer_id", total: { $sum: "$amount" } } },
 { $sort: { total: -1 } }
])
```

4 Indexing Best Practices

Type	Example	Use Case
Single Field	{ field1: 1 }	Simple lookups
Compound	{ field1: 1, field2: -1 }	Multi-field filtering
Multikey	{ tags: 1 }	Array fields
Text Index	{ description: "text" }	Full-text search
Hashed Index	{ _id: "hashed" }	Shard key for even distribution
TTL Index	{ createdAt: 1 }, { expireAfterSeconds: 3600 }	Auto-delete old docs

5 Write Performance Optimization

Technique	Command	Benefit
Bulk Inserts	db.collection.insertMany([...])	Reduces network overhead
Batch Writes	Use ordered: false	Continues even if some writes fail
Write Concern	{ w: 1, j: false } for non-critical	Faster writes
Disable Journaling (test only)	--nojournal	Boosts speed (not for production)
Pre-allocate IDs	Use application-side IDs	Avoids locking overhead

6 Hardware & Server-Level Tuning

Area	Recommendation
RAM	Keep working set (active data + indexes) in memory
Disk	Use SSDs for data & journal
CPU	Higher clock speed > more cores for OLTP
NUMA	Disable NUMA or configure interleave mode
Filesystem	Use XFS on Linux
WiredTiger Cache	Default = 50% of RAM; can tune via storage.wiredTiger.engineConfig.cacheSizeGB
Connections	Use connection pooling at app level

7 Replication Lag & Performance

- Monitor lag:

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

- rs.printSlaveReplicationInfo()
- Tune:
  - Increase oplog size for heavy write systems
  - Adjust write concern (w: "majority" vs w: 1)
  - Isolate backup and analytics workloads on secondaries

8 Sharding Performance Tuning

Aspect	Best Practice
Shard Key Choice	Evenly distribute data; avoid monotonically increasing fields
Pre-split Chunks	Avoid initial chunk migration
Balancer Window	Run balancing off-peak hours
Use Hashed Shard Keys	For random write distribution
Zone Sharding	Keep region-specific data close to users

9 Monitoring Commands Summary

Command	Purpose
db.serverStatus()	System-level health info
db.currentOp()	Shows running operations
db.killOp(opid)	Kill long-running operation
db.collection.stats()	Collection-level metrics
db.hostInfo()	Hardware details
db.stats()	Database-level storage info
rs.printReplicationInfo()	Replication log status
db.serverStatus().wiredTiger.cache	Cache utilization

10 Hands-On Validation Checklist

Validation Step	Command	Expected Result
Find slow queries	db.system.profile.find()	Queries >100ms logged
Analyze plan	.explain("executionStats")	IXSCAN instead of COLLSCAN
Create index	db.coll.createIndex({ field: 1 })	Index visible in db.coll.getIndexes()
Test improvement	Compare executionTimeMillis	Reduced time
Monitor performance	mongostat / mongotop	Balanced CPU & I/O usage
Check cache	db.serverStatus().wiredTiger.cache	Cache hit ratio > 95%